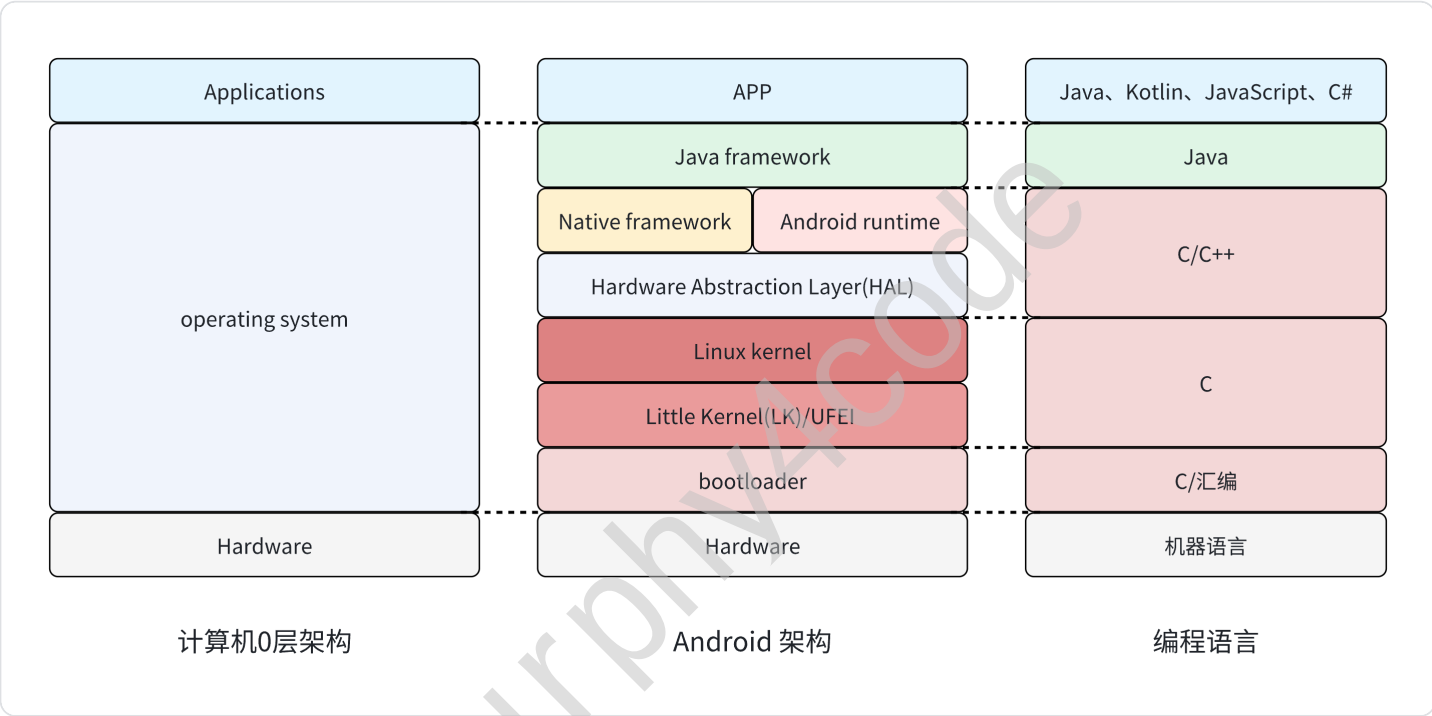


嵌入式C语言学习指南

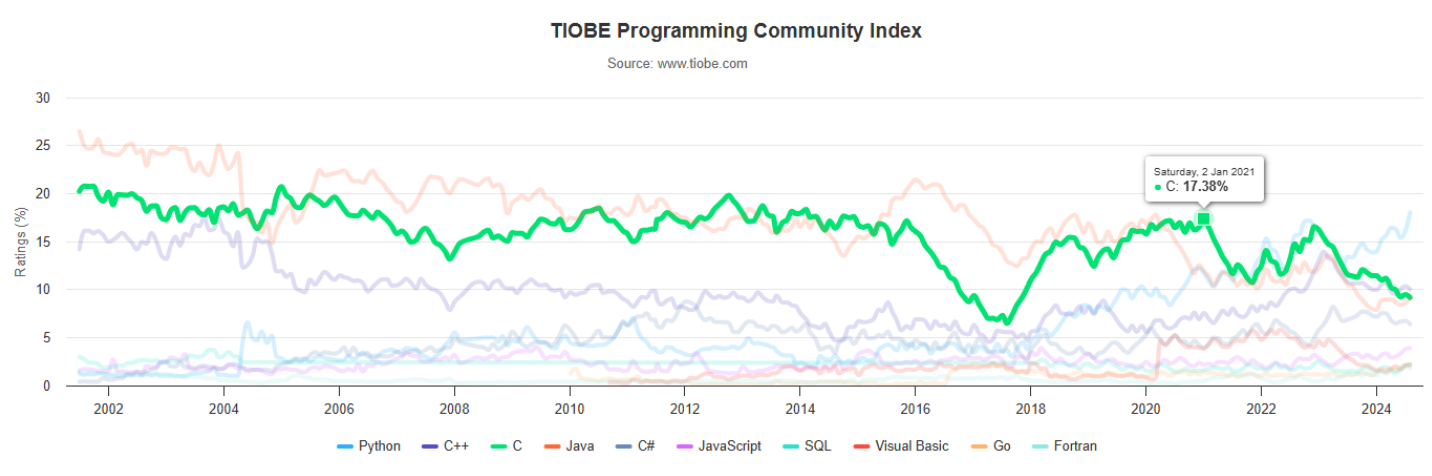
To be a programmer, not just a coder

1. 课程概述

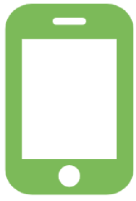
1.1 聊一聊C语言



- C语言是编程语言的常青树：长年霸榜编程语言TOP2（TIOBE：<https://www.tiobe.com/tiobe-index/>）。



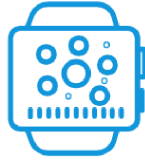
- C语言是最接近硬件的高级语言：在智能硬件中广泛应用



手机



无人机



IOT设备

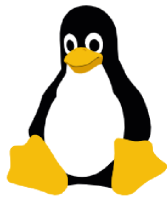


扫地机器人



智能驾驶舱

- C语言是「嵌入式软件开发」的必备工具和技能：



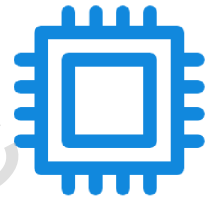
linux



RTOS



AutoSar



单片机裸机

1.2 课程定位

C语言在「嵌入式软件开发」中的应用（单片机裸机、RTOS、linux驱动、内核等）

1.3 课程设计思想

从"三个"视角出发，掌握C语言设计的底层逻辑。

内存视角

最适合操作底层的语言

C语言的底层逻辑

架构视角

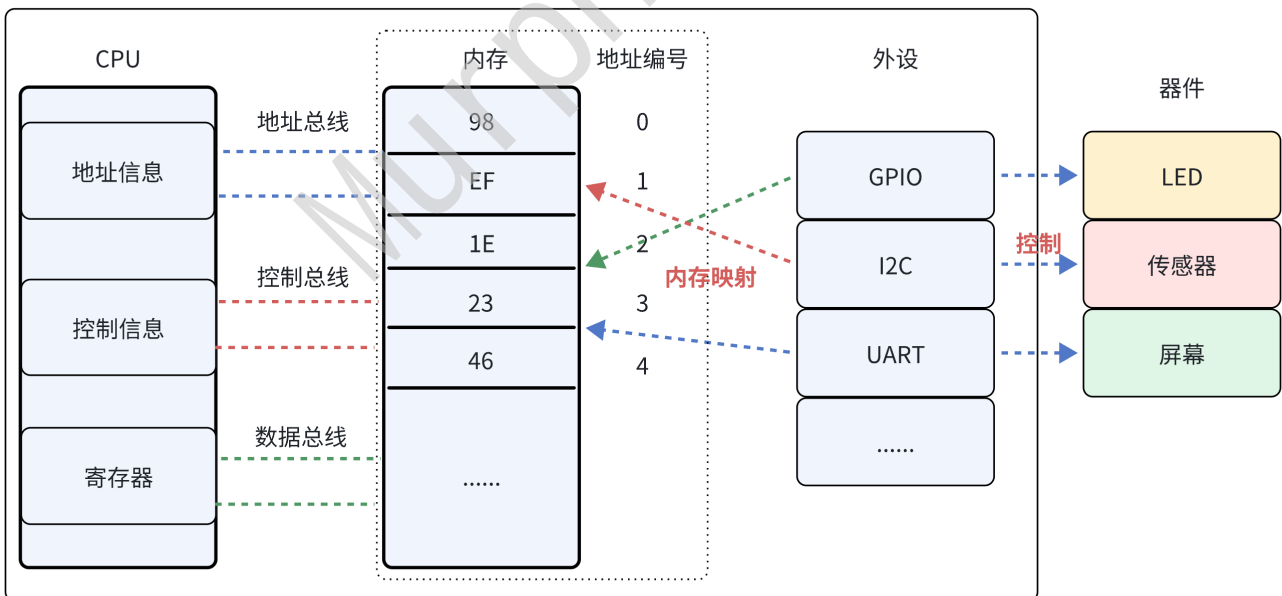
"solid"的C软件架构设计

编译器视角

是编译，也是优化

1. 「内存（硬件）视角」：操作底层最适合的语言

C语言最核心的、最powerful的能力：操作内存资源。实际上CPU外设硬件（GPIO、I2C、UART等等）本质上是内存映射。所以，C语言控制硬件本质上是操作内存，学会用内存的角度看待C语言，实际工程中面对问题才会得心应手。



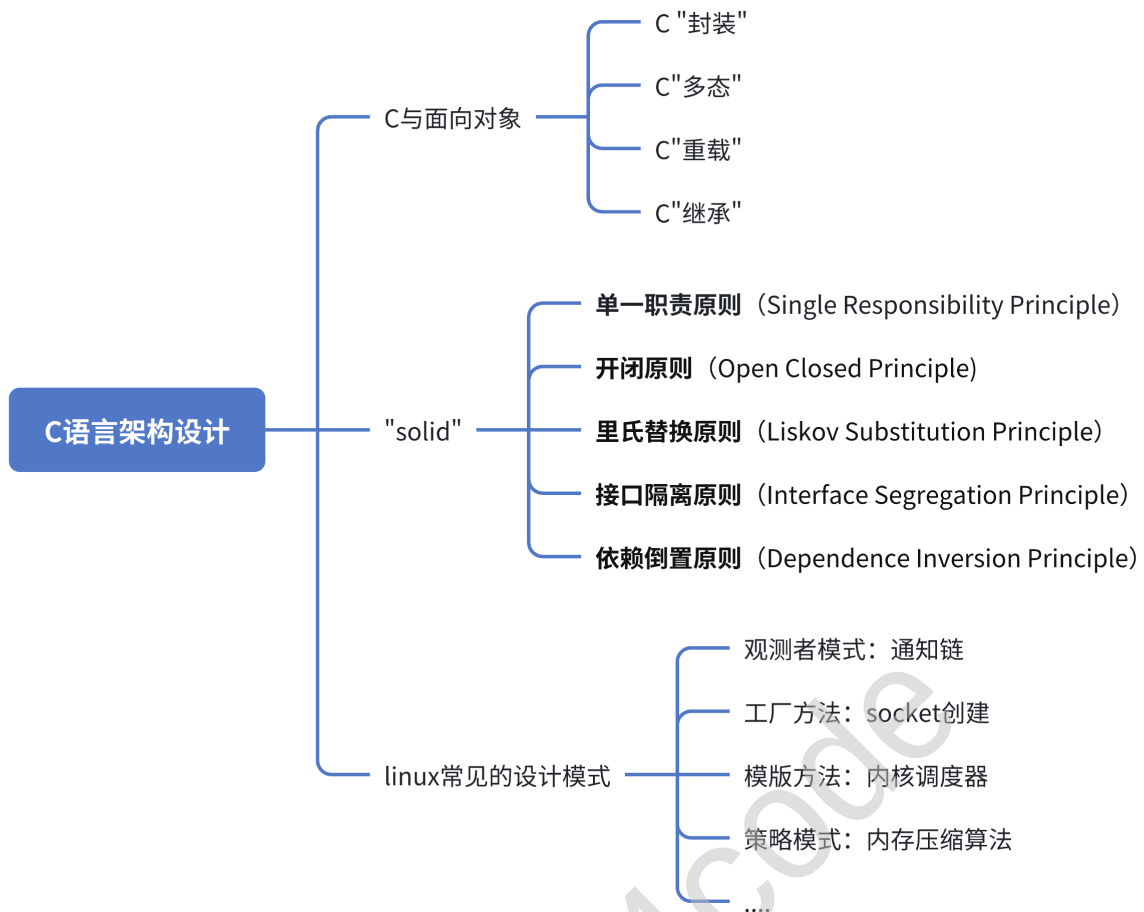
```
572 #define FLASH_BASE          0x08000000UL /*!< FLASH base address in the alias region */
573 #define FLASH_BANK1_END     0x0801FFFFUL /*!< FLASH END address of bank1 */
574 #define SRAM_BASE           0x20000000UL /*!< SRAM base address in the alias region */
575 #define PERIPH_BASE         0x40000000UL /*!< Peripheral base address in the alias region */
576
577 #define SRAM_BB_BASE        0x22000000UL /*!< SRAM base address in the bit-band region */
578 #define PERIPH_BB_BASE     0x42000000UL /*!< Peripheral base address in the bit-band region */
579
580
581 /*!< Peripheral memory map */
582 #define APB1PERIPH_BASE     PERIPH_BASE
583 #define APB2PERIPH_BASE     (PERIPH_BASE + 0x00010000UL)
584 #define AHBPERIPH_BASE      (PERIPH_BASE + 0x00020000UL)
585
586 #define TIM2_BASE           (APB1PERIPH_BASE + 0x00000000UL)
587 #define TIM3_BASE           (APB1PERIPH_BASE + 0x00000400UL)
588 #define TIM4_BASE           (APB1PERIPH_BASE + 0x00000800UL)
589 #define RTC_BASE            (APB1PERIPH_BASE + 0x00002800UL)
590 #define WWDG_BASE           (APB1PERIPH_BASE + 0x00002C00UL)
591 #define IWDG_BASE           (APB1PERIPH_BASE + 0x00003000UL)
592 #define SPI2_BASE           (APB1PERIPH_BASE + 0x00003800UL)
593 #define USART2_BASE         (APB1PERIPH_BASE + 0x00004400UL)
594 #define USART3_BASE         (APB1PERIPH_BASE + 0x00004800UL)
595 #define I2C1_BASE           (APB1PERIPH_BASE + 0x00005400UL)
596 #define I2C2_BASE           (APB1PERIPH_BASE + 0x00005800UL)
597 #define CAN1_BASE           (APB1PERIPH_BASE + 0x00006400UL)
598 #define BKP_BASE            (APB1PERIPH_BASE + 0x00006C00UL)
599 #define PWR_BASE            (APB1PERIPH_BASE + 0x00007000UL)
600 #define AFIO_BASE           (APB2PERIPH_BASE + 0x00000000UL)
601 #define EXTI_BASE           (APB2PERIPH_BASE + 0x00000400UL)
602 #define GPIOA_BASE          (APB2PERIPH_BASE + 0x00000800UL)
603 #define GPIOB_BASE          (APB2PERIPH_BASE + 0x00000C00UL)
604 #define GPIOC_BASE          (APB2PERIPH_BASE + 0x00001000UL)
605 #define GPIOD_BASE          (APB2PERIPH_BASE + 0x00001400UL)
606 #define GPIOE_BASE          (APB2PERIPH_BASE + 0x00001800UL)
607 #define ADC1_BASE           (AHBPERIPH_BASE + 0x00002A00UL)
```

stm32外设内存映射

2. 「函数（架构）视角」：程序逻辑、架构设计

面向过程的语言怎么做架构设计？能否借助C++/JAVA面向对象思想，写出优秀的代码？

- a. C语言中的”重载“、”继承“、”多态“、”封装“思想
- b. C语言程序设计怎么做到满足"solid"原则
- c. 内核中常见的设计模式有哪些？



3. 「编译器视角」

- 在工程中经常会讨论到，这个xx问题可能是编译器带来的。如，volatile防止O3编译优化导致的内存miss问题、宏函数展开没有语法检查，怎么用才安全？

```

1 #stm32f103xb.h
2 #define __IO volatile /*!< Defines 'write only' permissions
   */
3 #define __IIO volatile /*!< Defines 'read / write'
   permissions */
4
5 typedef struct
6 {
7     __IO uint32_t CRL;
8     __IO uint32_t CRH;
9     __IO uint32_t IDR;
10    __IO uint32_t ODR;
11    __IO uint32_t BSRR;
12    __IO uint32_t BRR;
13    __IO uint32_t LCKR;
14 } GPIO_TypeDef;
  
```

- 利用编译器特性做代码设计：__attribute__((packed))、__attribute__((section)) 等等。官网(<https://gcc.gnu.org/onlinedocs/gcc/Common-Function-Attributes.html>)

```
1 #cmsis_gcc.h
2 #define __NO_RETURN          __attribute__((__noreturn__))
3 #define __USED               __attribute__((used))
4 #define __WEAK               __attribute__((weak))
5 #define __PACKED             __attribute__((packed, aligned(1)))
6 #define __PACKED_STRUCT     struct __attribute__((packed, aligned(1)))
7 #define __PACKED_UNION       union __attribute__((packed, aligned(1)))
```

```
1 #include/linux/init.h
2 #ifndef MODULE
3 // 省略
4 #define module_init(x) __initcall(x);
5 // 省略
6 #else
7
8 #define module_init(x) __initcall(x);
9 |
10 --> #define __initcall(fn) device_initcall(fn)
11 |
12 --> #define device_initcall(fn) __define_initcall(fn, 6)
13 |
14 --> #define __define_initcall(fn, id) \
15     static initcall_t __initcall_##fn##id __used \
16     __attribute__((__section__(".initcall" #id ".init"))) = fn
```

1.4 课程适合人群

- 有一定编程基础，有意向转行嵌入式软件开发者
- 大学生，希望更系统、有深度的学习嵌入式软件编程
- 嵌入式软件从业者，想要进阶提升，往架构师、SE职业方向走的同学

1.5 学习方法

- 沉浸在linux环境中编程，体验贴近实际工程的编程过程。
- 学习行业内优秀案例。体会实际工程中优秀的代码设计（linux、RTOS、STM32官方软件设计）
- 在学习的过程中，希望同学们能站在C语言设计者的角度来思考，为什么要这样设计？如果是你，你怎么来设计？

2. 调试环境搭建

本课程是基于VMware搭建的ubuntu环境上，进行编码、编译、调试。

2.1 ubuntu环境搭建

<https://pan.baidu.com/s/1n1sigtYwfnX8EduoJAcqw?pwd=embe> 提取码:embe



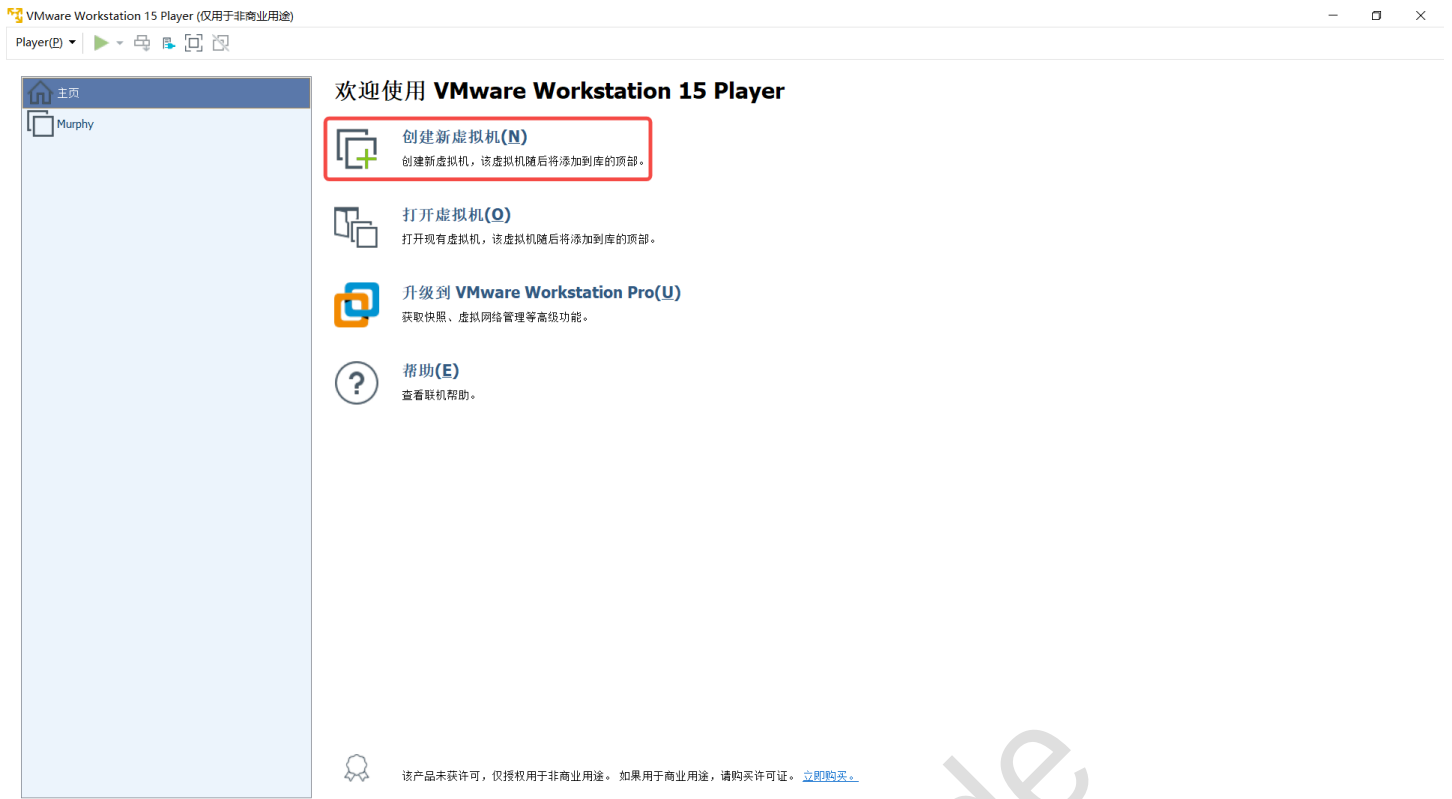
2.1.1 VMware安装

一路「默认」安装即可

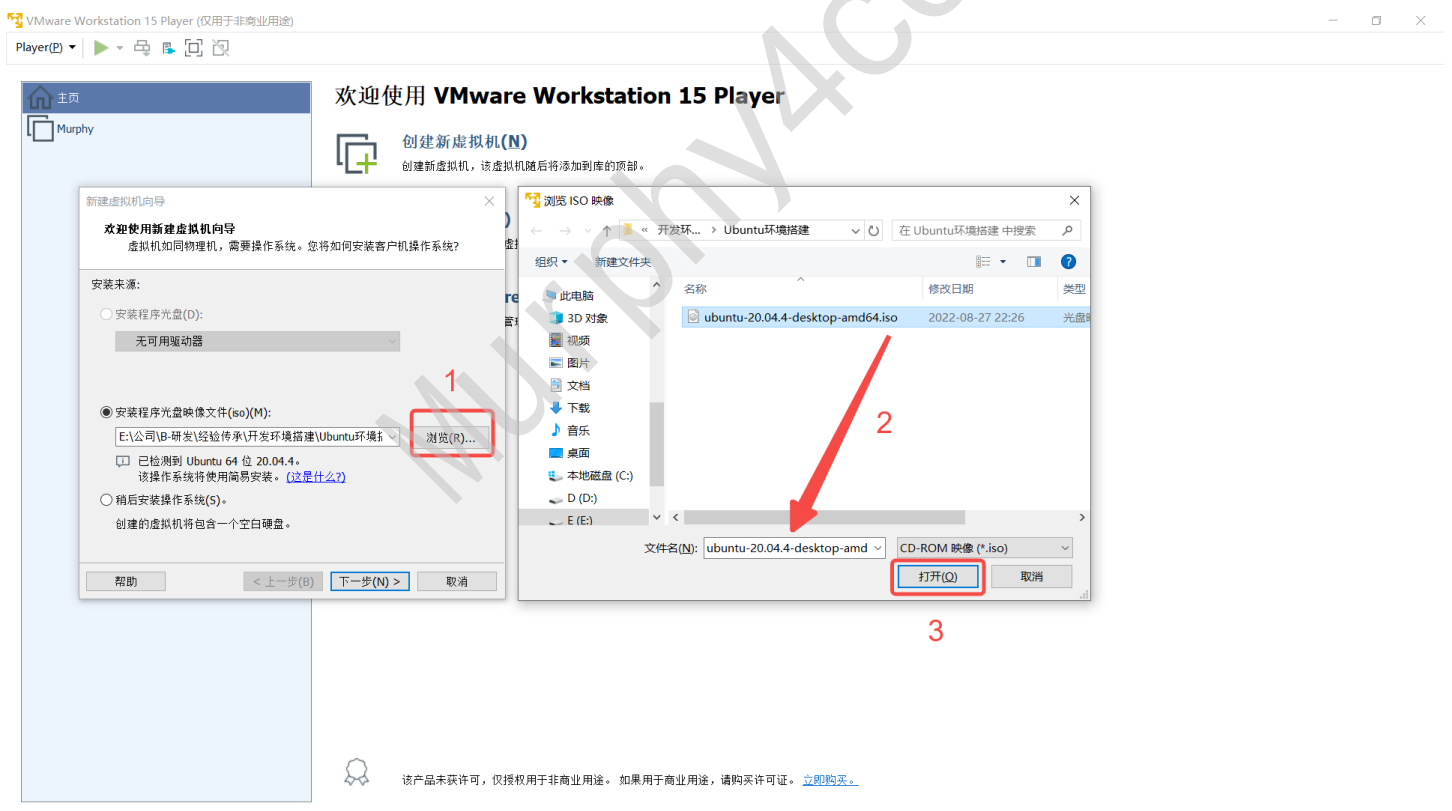


2.1.2 ubuntu虚拟机安装

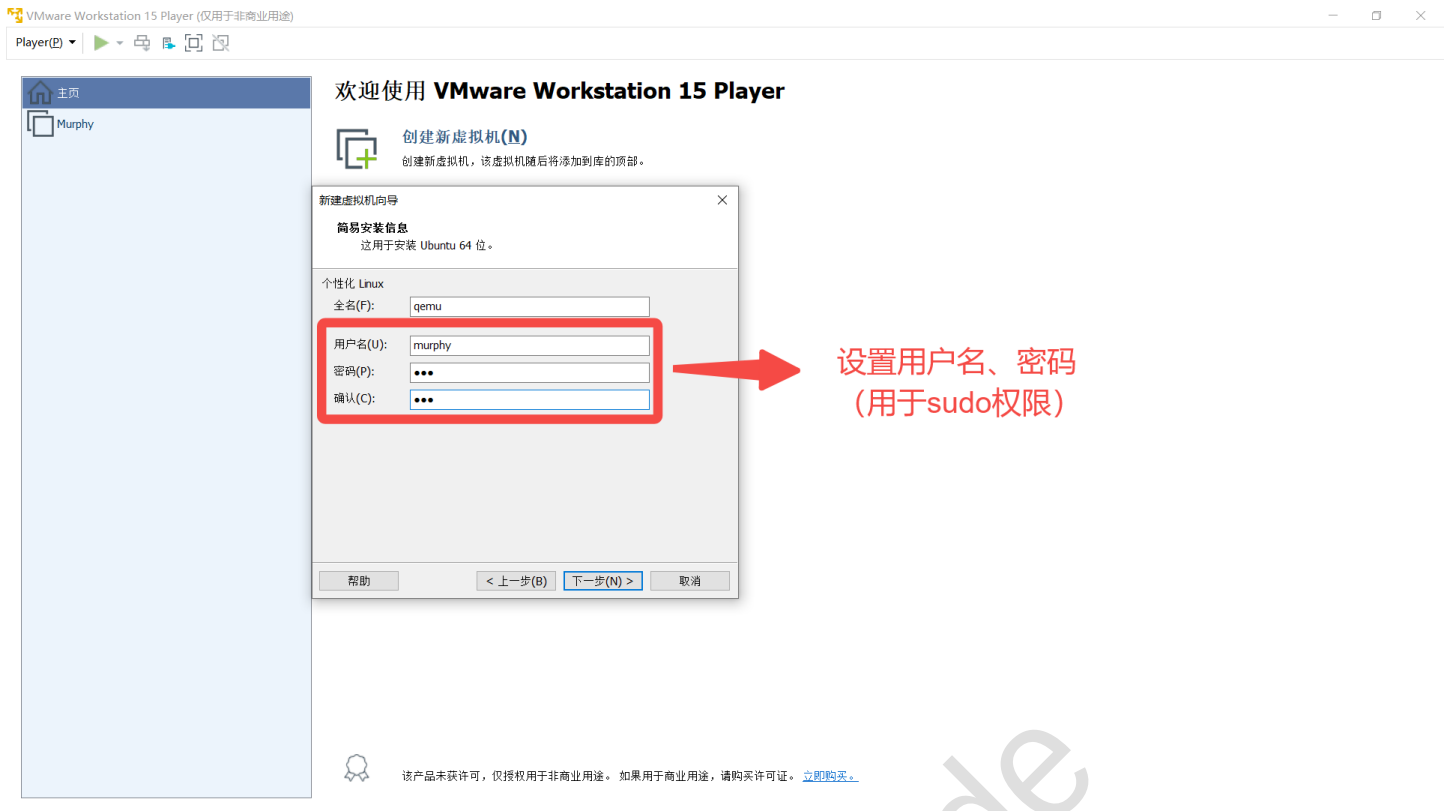
1. 打开VMware 选择 “创建新虚拟机”



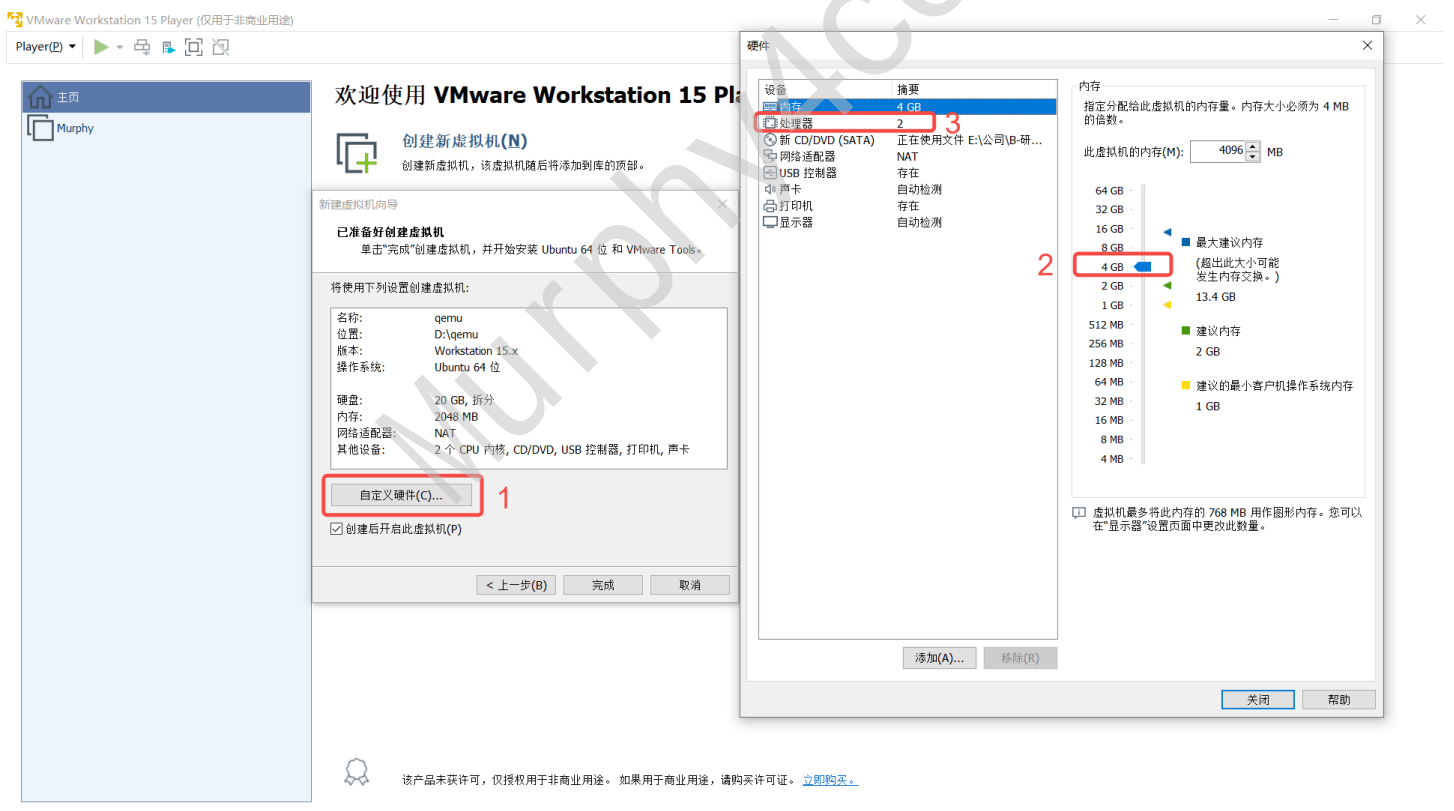
2. 选择下载好的“Ubuntu ISO 镜像”



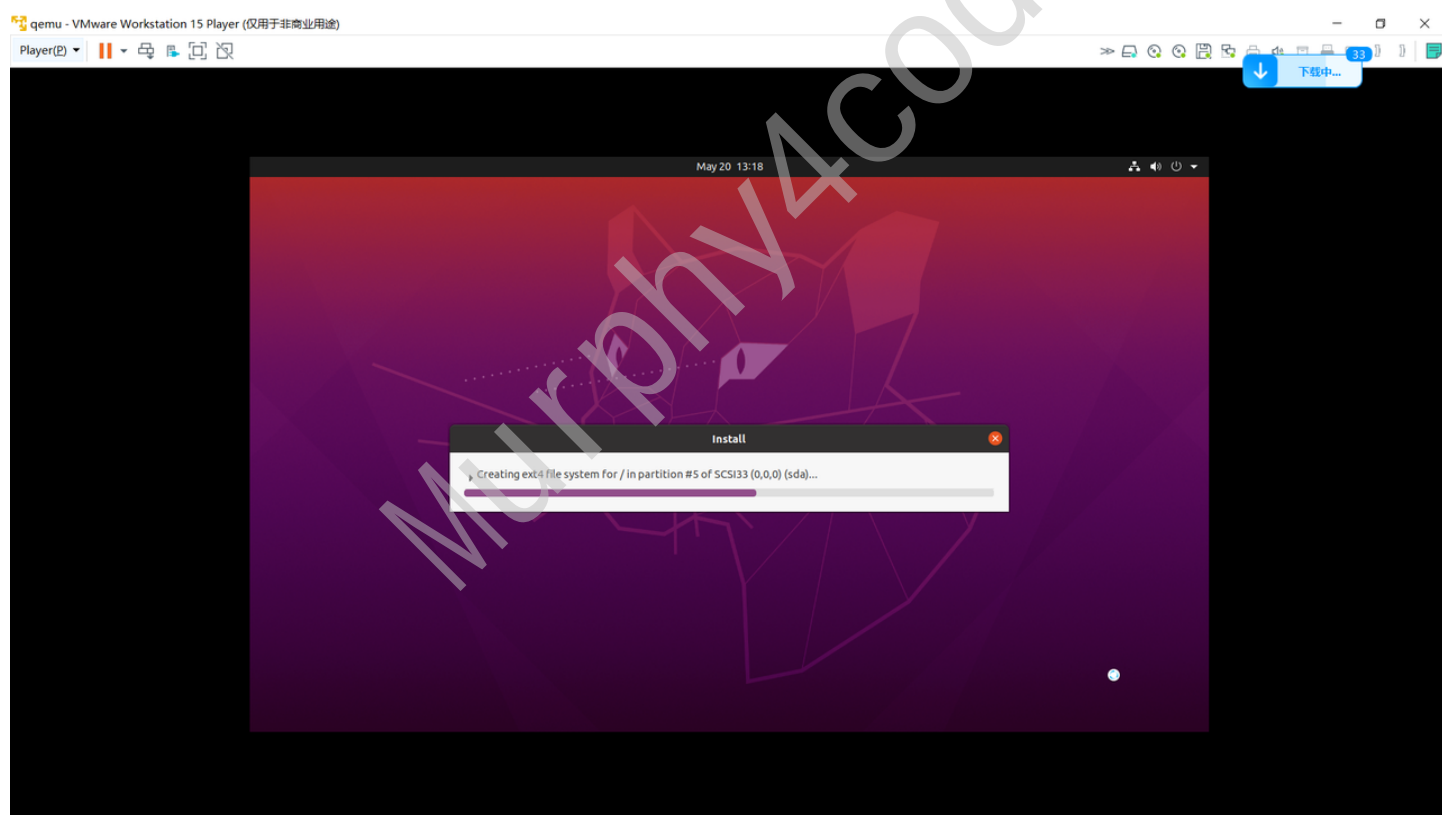
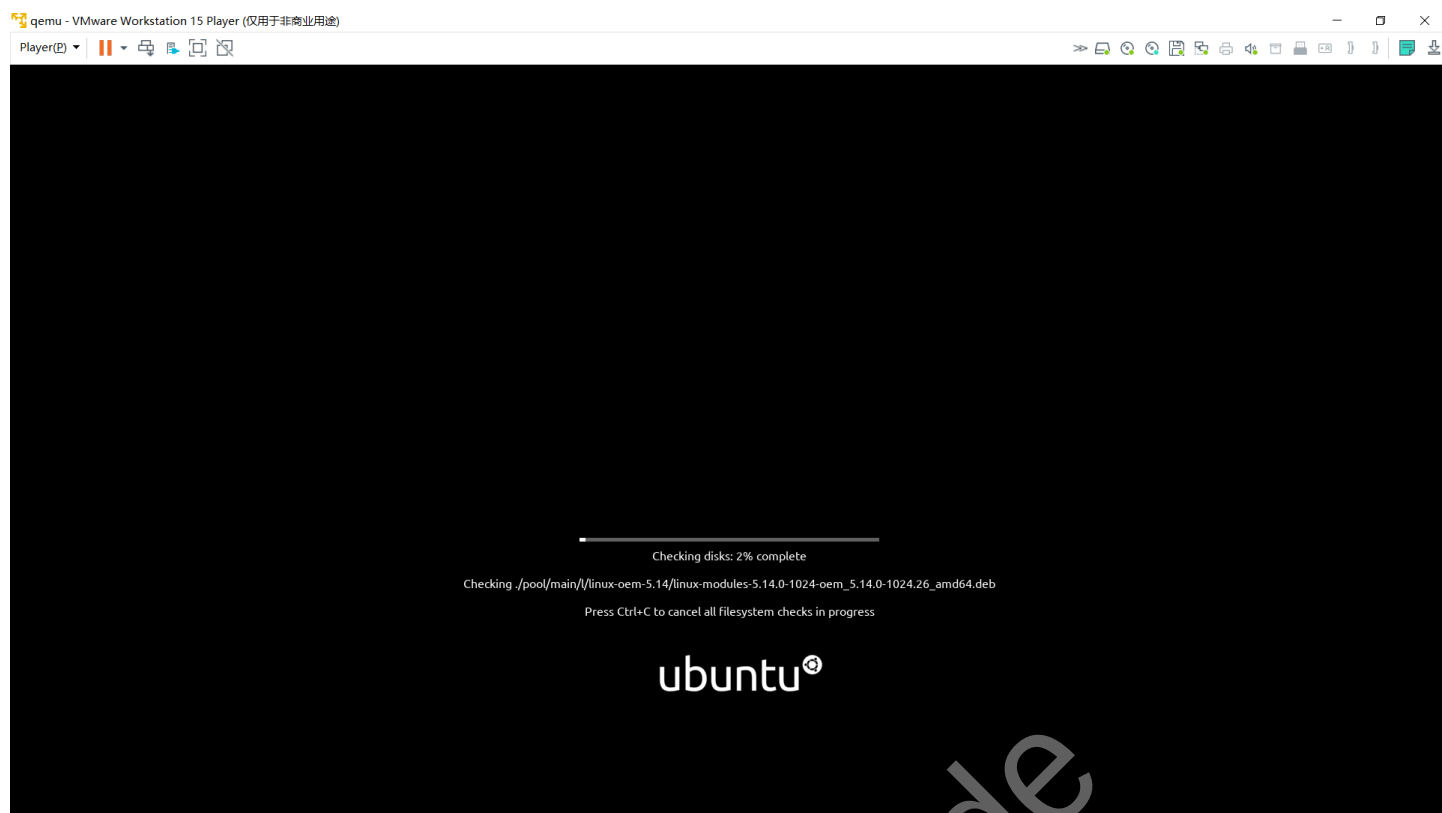
3. 设置用户名与密码



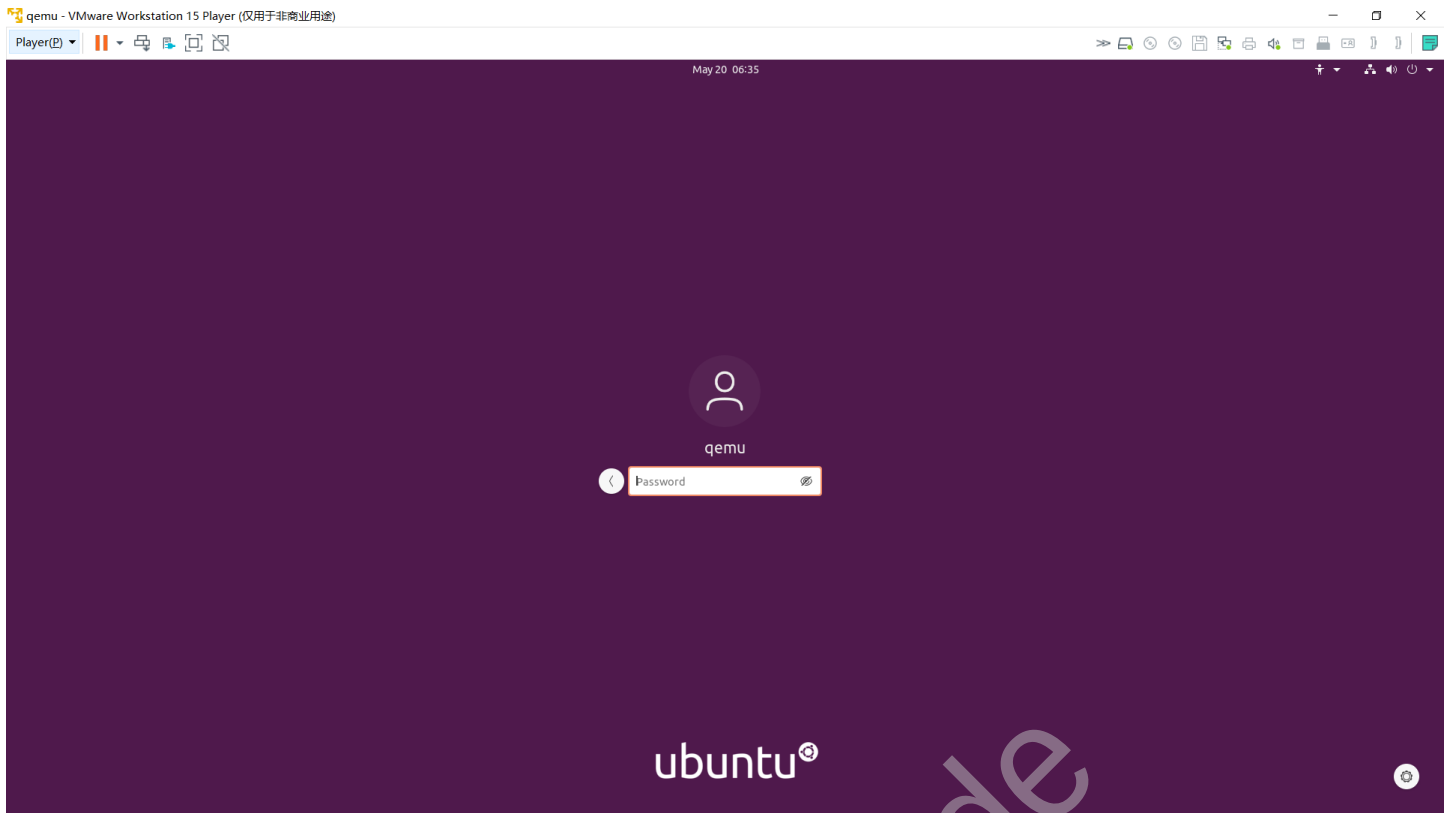
4. 一路默认选择后，到自定义界面：建议选择4G内存、处理器为2、硬盘大小为100GB



5. 设置好后，就是漫长的安装等待环节



6. 成功时的样子：



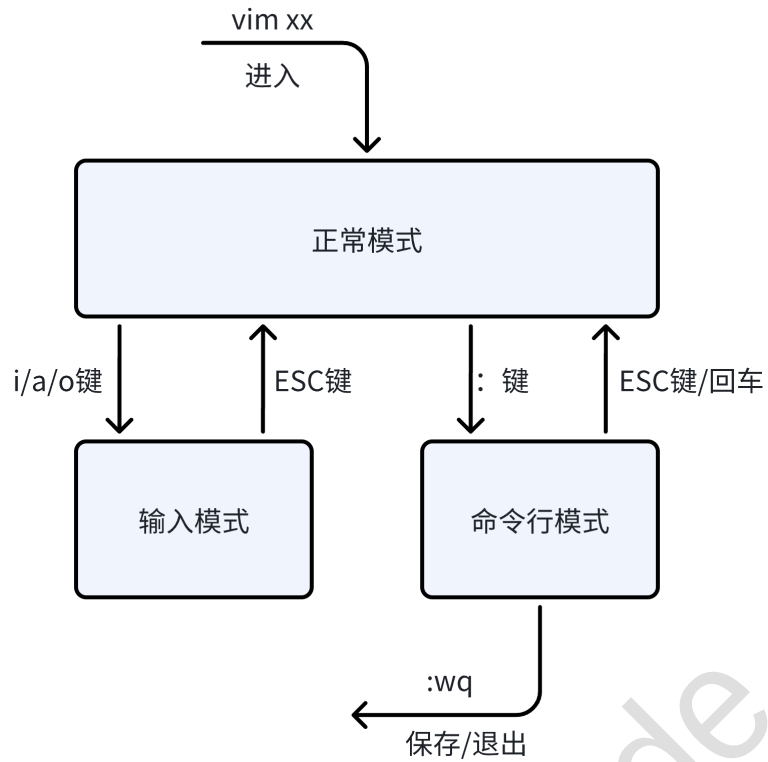
2.1.3 linux常用命令

- 1、**[ctrl+alt+t]**: 打开命令终端
- 2、**[cd]**: 进入目录
- 3、**[ls]**: 显示当前目录下的文件
- 4、**[mkdir]**: 创建一个文件夹
- 5、**[touch]**: 创建一个文件（也可以用vim工具来创建）
- 6、**[cp src dest]**: 复制
- 7、**[rm -rf]**: 删除
- 8、**[clear]**: 清除屏幕
- 9、**[提权操作]**: `sudo xx`

2.2 文本编辑器vim

2.2.1 vim简介

vim 是从 vi 发展出来的一个文本编辑器。具有代码补全、快速跳转、查找等功能，在linux中被广泛使用。vim 官方网站 (<https://www.vim.org/>)



「正常模式」：

这个模式下一般用于阅读、复制粘贴、文本替换、文本查找等功能。

A screenshot of a terminal window with a dark background. The terminal shows a C program being edited in Vim. The code is as follows:

```
#include <stdio.h>

#define ABC 123

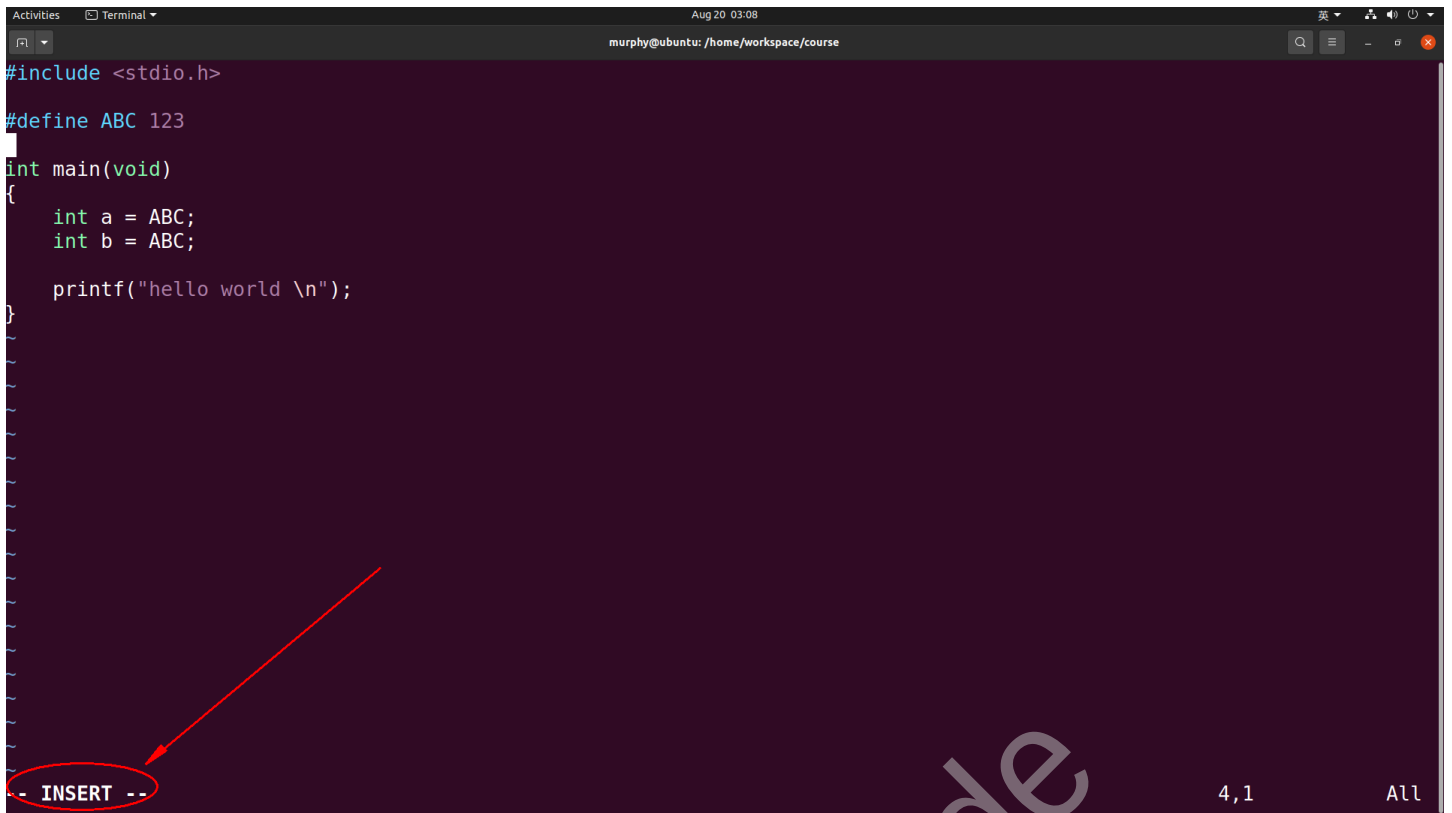
int main(void)
{
    int a = ABC;
    int b = ABC;

    printf("hello world \n");
}
```

The terminal window has a title bar that says 'Activities Terminal' and a status bar at the bottom right showing '4,0-1' and 'All'. A large, semi-transparent watermark 'Murphy's Code' is overlaid diagonally across the terminal content.

「输入模式」：

这个模式用于文本编辑输入。



```
#include <stdio.h>

#define ABC 123

int main(void)
{
    int a = ABC;
    int b = ABC;

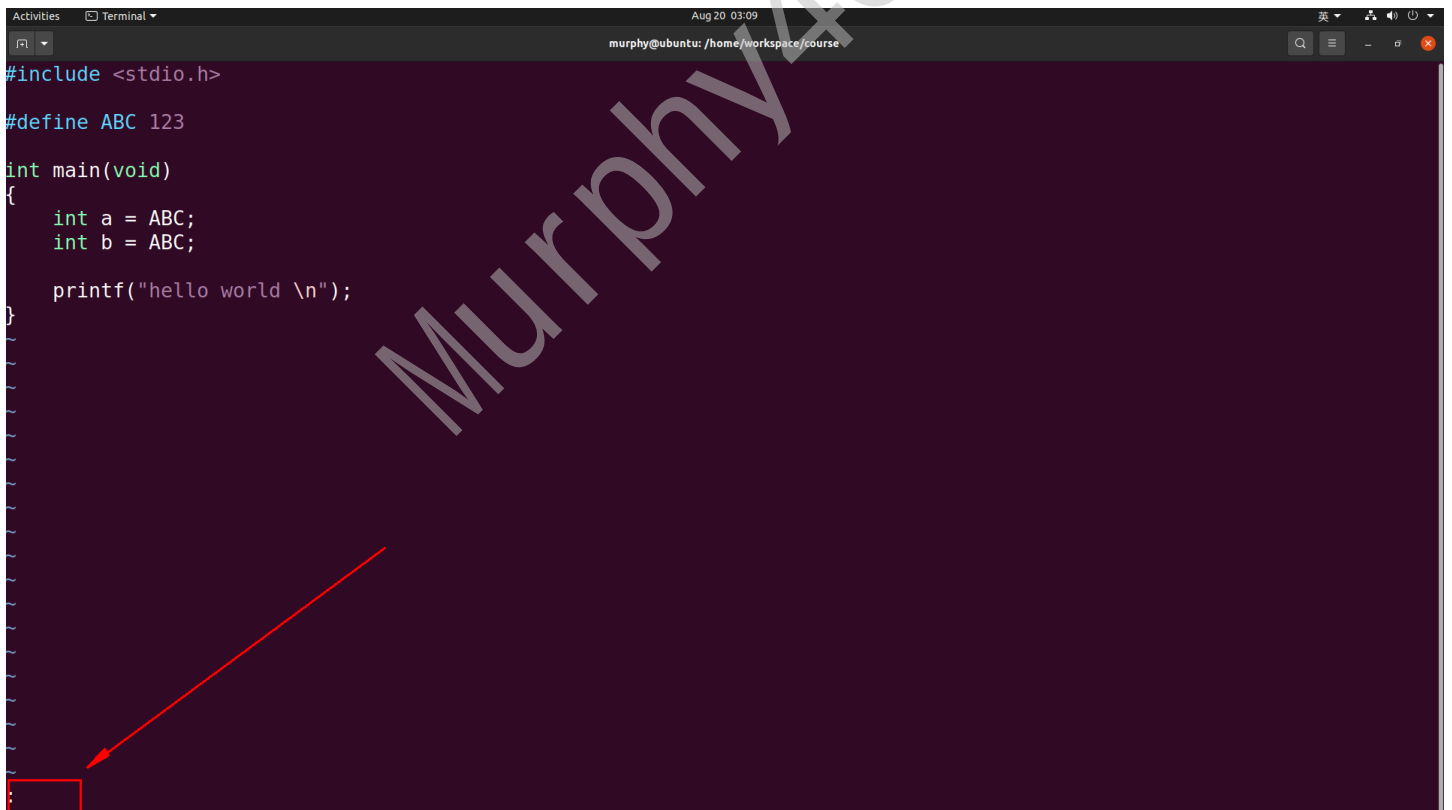
    printf("hello world \n");
}
```

- INSERT -

4,1 All

「命令行模式」：

这个模式用于文件的保存、退出等功能。



```
#include <stdio.h>

#define ABC 123

int main(void)
{
    int a = ABC;
    int b = ABC;

    printf("hello world \n");
}
```

:

2.2.2 安装vim

```
1 sudo apt-get install vim
```

```
Activities Terminal Aug 20 02:24
murphy@ubuntu: /home/workspace/course
murphy@ubuntu:/home/workspace/course$ sudo apt-get install vim
[sudo] password for murphy:
Reading package lists... Done
Building dependency tree
Reading state information... Done
vim is already the newest version (2:8.1.2269-1ubuntu5.23).
0 upgraded, 0 newly installed, 0 to remove and 94 not upgraded.
murphy@ubuntu:/home/workspace/course$
```

2.2.3 vim增删改查

通过vim编写一个hello world代码。

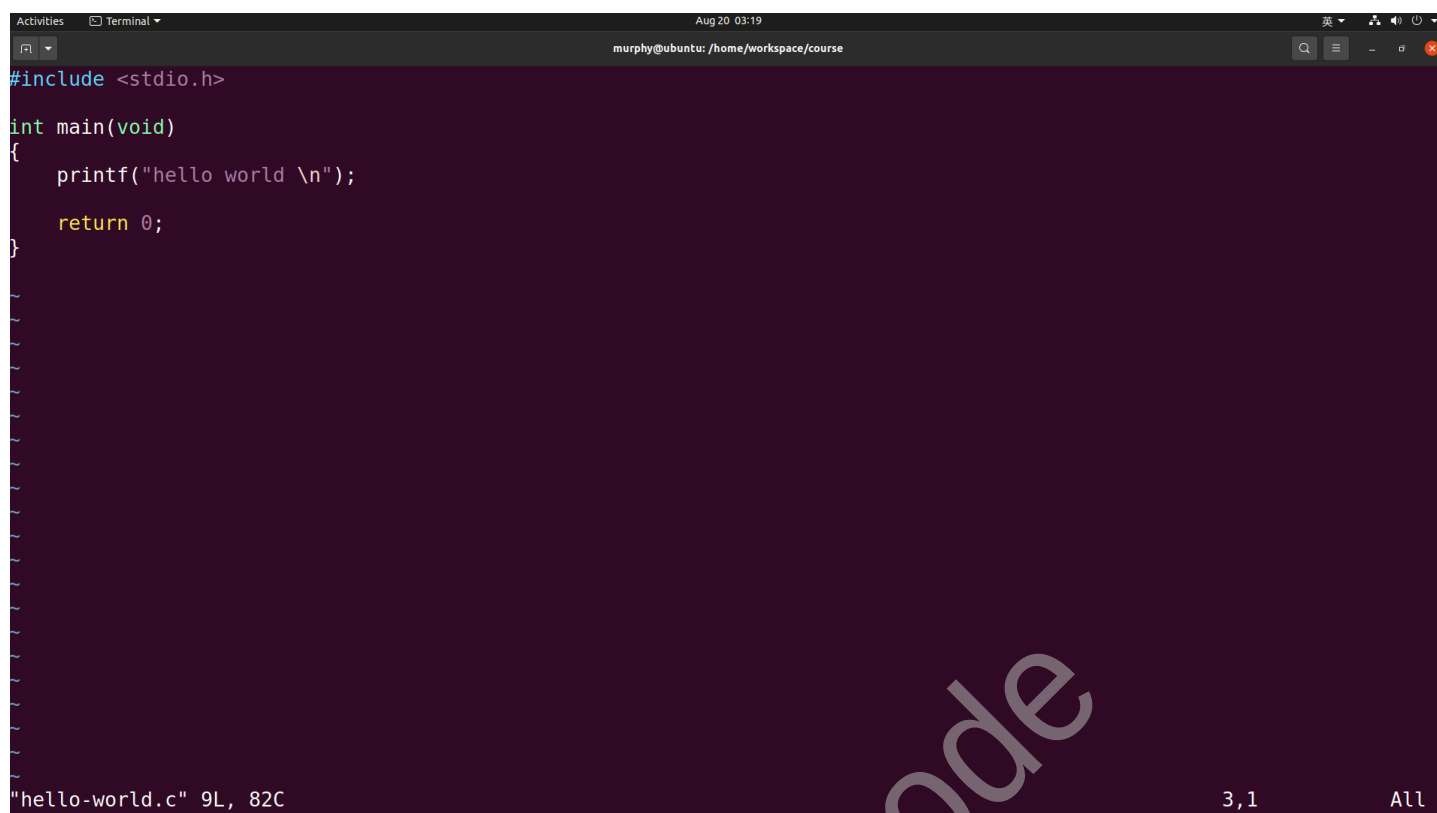
- 1、输入以下命令打开一个空的文件，此时默认进入了正常模式

```
1 vim hello-world.c
```

```
Activities Terminal Aug 20 03:15
murphy@ubuntu: /home/workspace/course

"hello-world.c" [New File] 0,0-1 All
```

2、按键盘中的「i」进入编辑模式，并输入hello world的代码

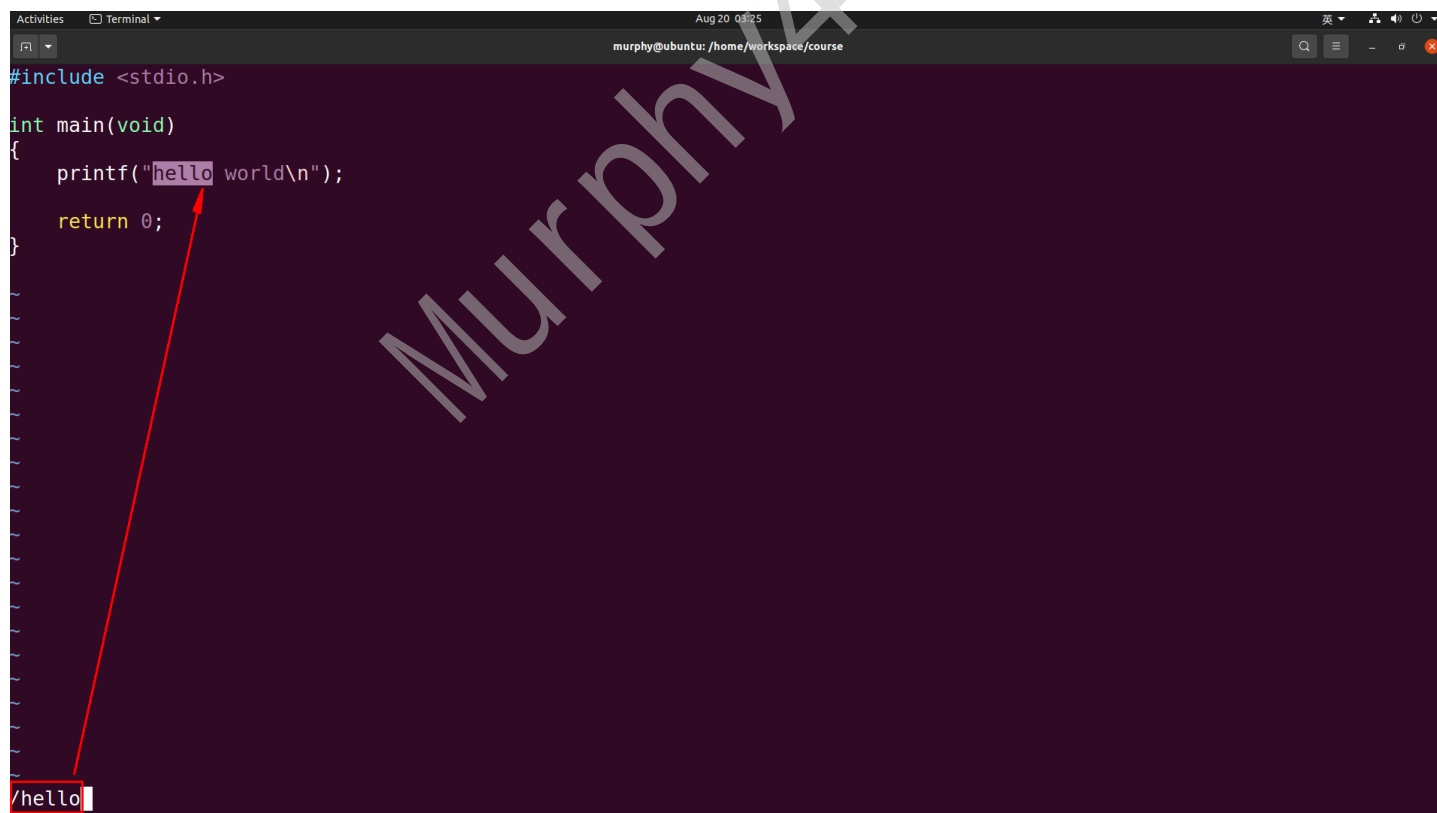


```
#include <stdio.h>

int main(void)
{
    printf("hello world \n");
    return 0;
}
```

"hello-world.c" 9L, 82C 3,1 All

3、在输入模式下，按键盘中的「Esc」进入正常模式，此时输入「/hello」可进行查找hello字符

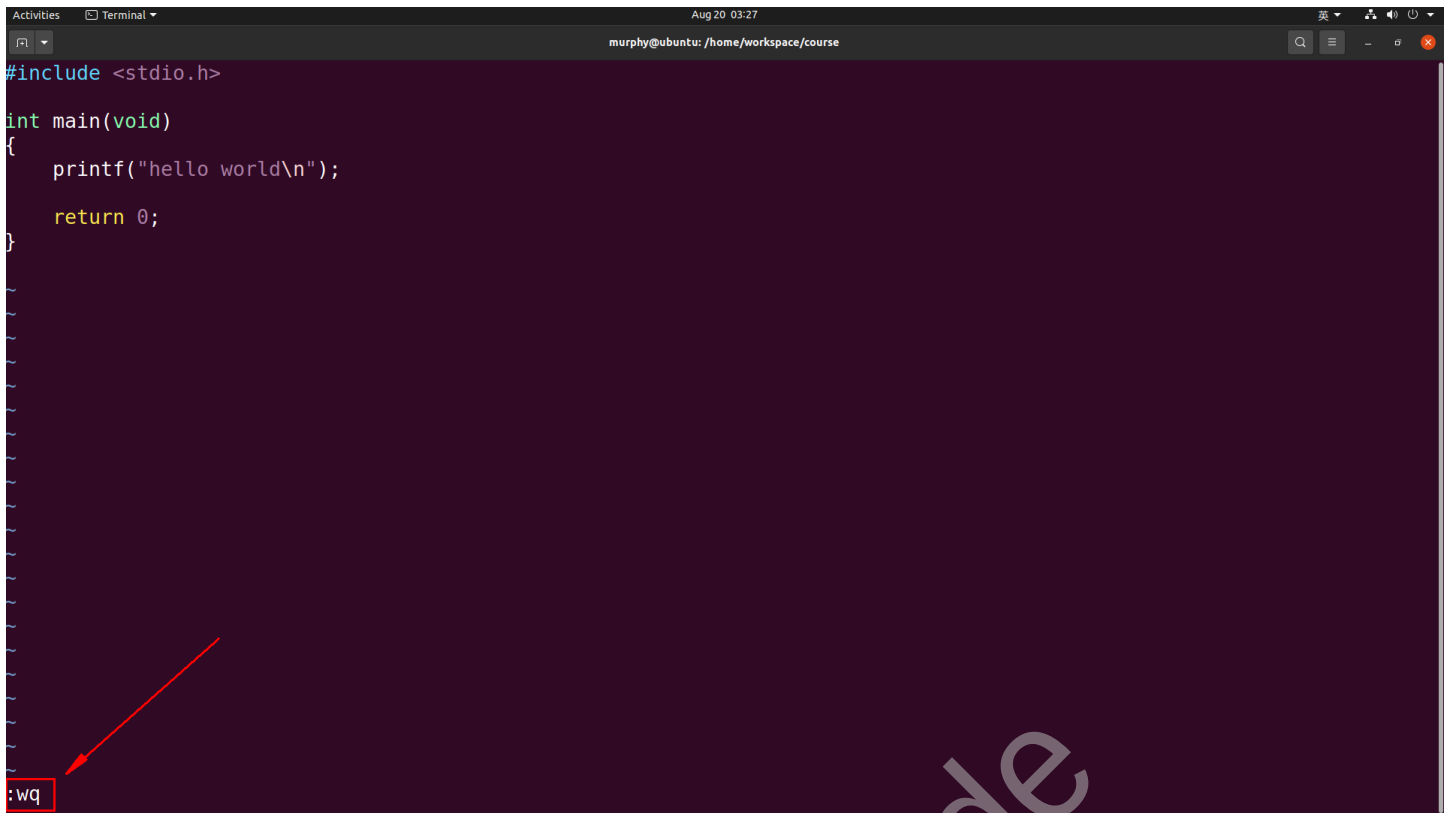


```
#include <stdio.h>

int main(void)
{
    printf("hello world\n");
    return 0;
}
```

/hello

4、在正常模式下，按键盘中的「:」进入命令行模式，输入「wq」，保存并退出。如果不想保存可以输入「q!」强制退出



```
#include <stdio.h>

int main(void)
{
    printf("hello world\n");
    return 0;
}
```

:wq

以上就相当于vim入门，可以完成文本的编辑、查找、保存基础操作。vim是一个工具，大家在学习)的时候不需要强行记忆命令，要学会在使用中熟练和记忆。下面介绍一个vim学习的神器：vimtutor。

2.2.4 vimtutor

vimtutor是vim自带的一个交互式教程程序，通过它你可以轻松掌握 vim 的基本使用方法和技巧，在终端中输入：

```
1 vimtutor zh
```

```
Activities Terminal Aug 20 02:35
murphy@ubuntu: /home/workspace/course

=====
欢 迎 阅 读 《 V I M 教 程 》 — 版 本 1.7
=====

Vim 是一个具有很多命令的功能非常强大的编辑器。限于篇幅，在本教程当中
就不详细介绍了。本教程的设计目标是讲述一些必要的基本命令，而掌握好这
些命令，您就能够很容易地将 Vim 当作一个通用编辑器来使用了。

完成本教程的内容大约需要25-30分钟，取决于您训练的时间。

注意：
每一节的命令操作将会更改本文。推荐您复制本文的一个副本，然后在副本上
进行训练(如果您是通过"vimtutor"来启动教程的，那么本文就已经是副本了)。

切记一点：本教程的设计思路是在使用中进行学习的。也就是说，您需要通过
执行命令来学习它们本身的正确用法。如果您只是阅读而不操作，那么您可能
会很快遗忘这些命令的！

好了，现在请确定您的Shift-Lock(大小写锁定键)还没有按下，然后按键盘上
的字母键 j 足够多次来移动光标，直到第一节的内容能够完全充满屏幕。

=====
第一讲第一节：移动光标

** 要移动光标，请依照说明分别按下 h、j、k、l 键。 **

      ^
      k      提示： h 的键位于左边，每次按下就会向左移动。
< h      l >      l 的键位于右边，每次按下就会向右移动。
"/tmp/tutorP0iVav" 996 lines, 38811 characters
```

2.3 代码编译器gcc

2.3.1 gcc简介

gcc是GNU Compiler Collection的简称，是GNU(GNU's Not Unix)开源项目的编译器套件。gcc 的初衷是为GNU操作系统专门编写的一款编译器，用于编译C代码。现如今已扩展为可以编译C++、Java、Objective-C等多种编程语言的集合。gcc本身也遵循GPL发行许可证，大名鼎鼎的linux就是基于gcc搭建的编译系统。gcc 官方网站 (<https://gcc.gnu.org/>)

GCC, the GNU Compiler Collection

The GNU Compiler Collection includes front ends for C, C++, Objective-C, Fortran, Ada, Go, D and Modula-2 as well as libraries for these languages (libstdc++...). GCC was originally written as the compiler for the GNU operating system. The GNU system was developed to be 100% free software, free in the sense that it respects the user's freedom.

We strive to provide regular, high quality releases, which we want to work well on a variety of native and cross targets (including GNU/Linux), and encourage everyone to contribute changes or help testing GCC. Our sources are readily and freely available via [Git](#) and weekly [snapshots](#).

Major decisions about GCC are made by the [steering committee](#), guided by the [mission statement](#).

News

- GCC 14.2 released** [2024-08-01]
- GCC 11.5 released** [2024-07-19]
- GCC 12.4 released** [2024-06-20]
- GCC 13.3 released** [2024-05-21]
- GNU Tools Cauldron 2024** [2024-05-07]
Prague, Czech Republic, September 14-16 2024
- GCC 14.1 released** [2024-05-07]
- GCC developer room at FOSDEM 2024: Call for Participation open** [2023-11-20]
FOSDEM 2024: Brussels, Belgium, February 3-4 2024
- GNU Tools Cauldron 2023** [2023-09-05]
Cambridge, United Kingdom, September 22-24 2023
- GCC 13.2 released** [2023-07-27]
- GCC 10.5 released** [2023-07-07]
- GCC Code of Conduct adopted** [2023-06-16]
- GCC 11.4 released** [2023-05-29]
- GCC 12.3 released** [2023-05-08]
- GCC 13.1 released** [2023-04-26]
- GCC BPF in Compiler Explorer** [2022-12-23]
Support for a nightly build of the bpf-unknown-none-gcc compiler has been contributed to Compiler Explorer (aka godbolt.org) by Marc Poulhiès
- Modula-2 front end added** [2022-12-14]
The Modula-2 programming language front end has been added to GCC. This front end was contributed by Gaius Mulley.
- GNU Tools Cauldron 2022** [2022-09-02]
Prague, Czech Republic and online, September 16-18 2022
- GCC 12.2 released** [2022-08-19]
- GCC 10.4 released** [2022-06-28]

Supported Releases

- GCC 14.2 (changes)**
Status: 2024-07-11 (regression fixes & docs only).
Serious regressions. All regressions.
- GCC 13.3 (changes)**
Status: 2024-04-30 (regression fixes & docs only).
Serious regressions. All regressions.
- GCC 12.4 (changes)**
Status: 2024-06-20 (regression fixes & docs only).
Serious regressions. All regressions.
- Development: GCC 15.0 (release criteria, changes)**
Status: 2024-04-26 (general development).
Serious regressions. All regressions.

Search our site

Match: All words Sort by: Newest

There is also a [detailed search form](#).

Get our announcements

About GCC

- Mission
- Releases
- Snapshots
- Mailing lists
- Contributors

[@gnutools](#)

[Mastodon](#)

[Make A Donation](#)

Documentation

- Installation
- Platforms
- Manual
- FAQ
- Wiki
- Pointers

Download

- Mirrors
- Binaries

Sources

- Git
- write access
- Rsync

Development

- Plan & Timeline
- Contributing
- Why contribute?
- Open projects
- Front ends
- Back ends
- Extensions
- Testing
- Benchmarks
- Translations

Bugs

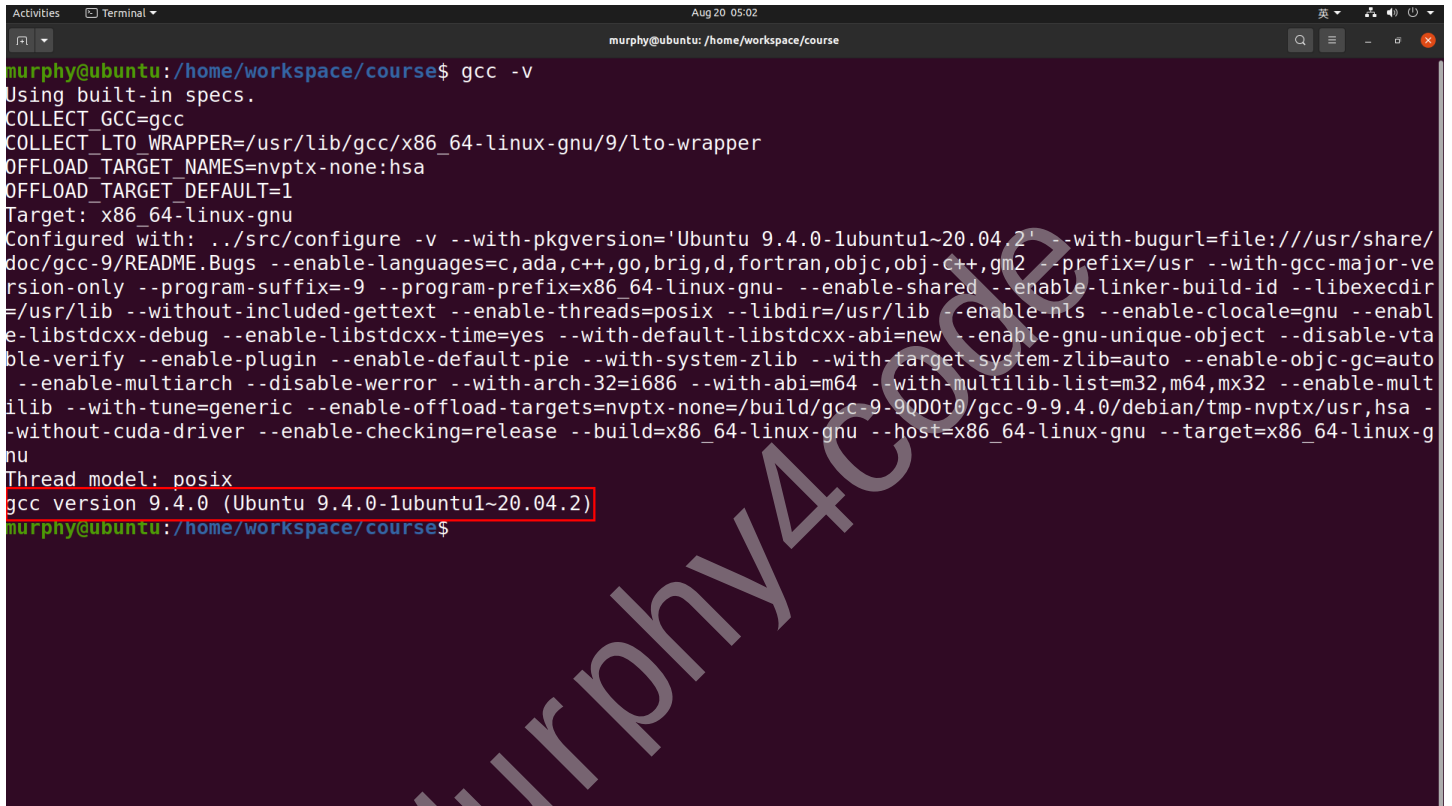
- Known bugs
- How to report
- Bug tracker
- Management

2.3.2 安装gcc

- 1 sudo apt-get update
- 2 sudo apt-get install build-essential

输入以下命令确认版本号：

- 1 gcc -v

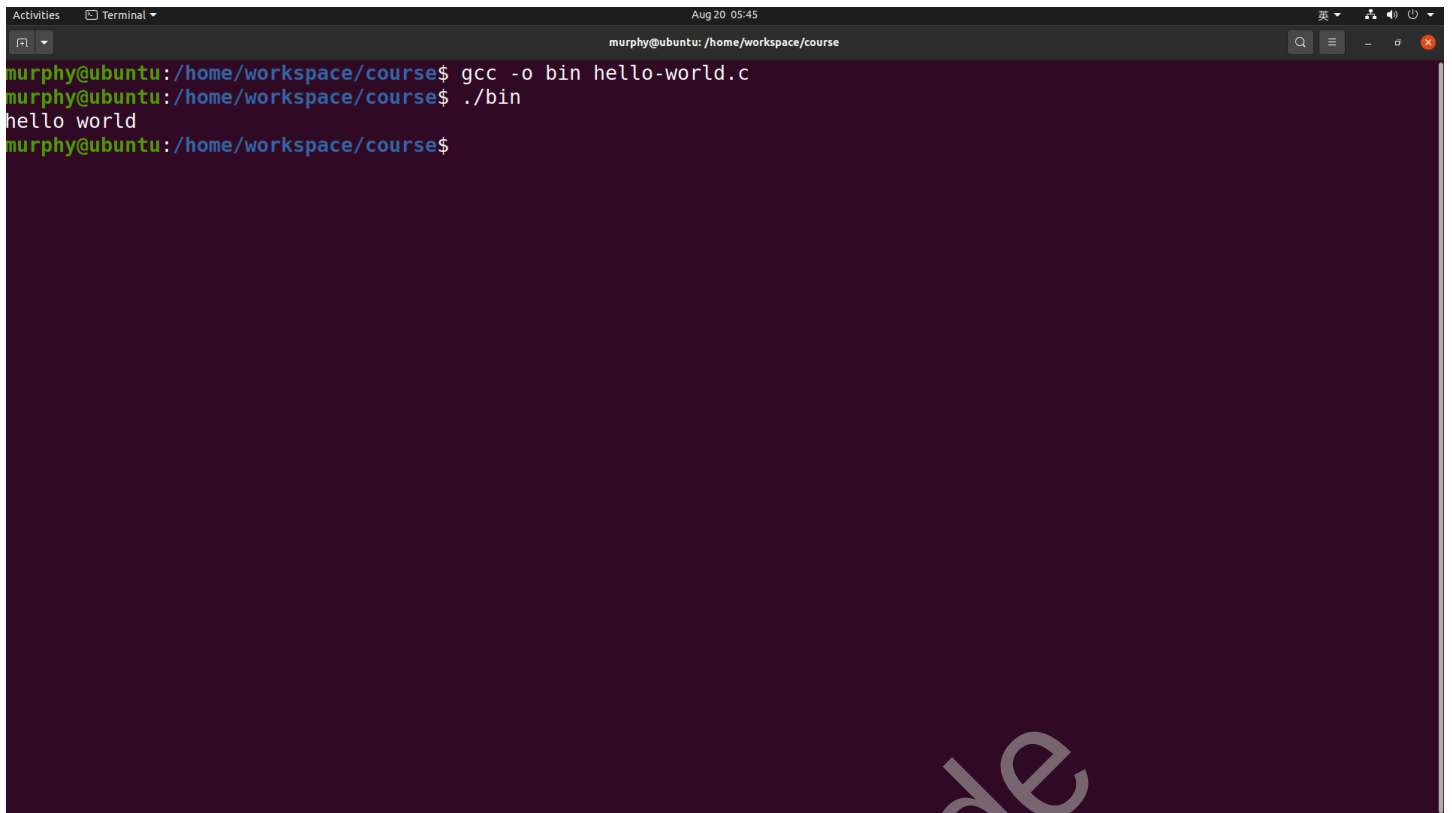
A terminal window screenshot showing the output of the 'gcc -v' command. The output includes configuration details for gcc 9.4.0 on Ubuntu 9.4.0-1ubuntu1~20.04.2. The line 'gcc version 9.4.0 (Ubuntu 9.4.0-1ubuntu1~20.04.2)' is highlighted with a red box. A large diagonal watermark 'Murphy408' is visible across the terminal output.

```
murphy@ubuntu:/home/workspace/course$ gcc -v
Using built-in specs.
COLLECT_GCC=gcc
COLLECT_LTO_WRAPPER=/usr/lib/gcc/x86_64-linux-gnu/9/lto-wrapper
OFFLOAD_TARGET_NAMES=nvptx-none:hsa
OFFLOAD_TARGET_DEFAULT=1
Target: x86_64-linux-gnu
Configured with: ../src/configure -v --with-pkgversion='Ubuntu 9.4.0-1ubuntu1~20.04.2' --with-bugurl=file:///usr/share/doc/gcc-9/README.Bugs --enable-languages=c,ada,c++,go,brig,d,fortran,objc,obj-c++,gm2 --prefix=/usr --with-gcc-major-version-only --program-suffix=-9 --program-prefix=x86_64-linux-gnu- --enable-shared --enable-linker-build-id --libexecdir=/usr/lib --without-included-gettext --enable-threads=posix --libdir=/usr/lib --enable-nls --enable-clocale=gnu --enable-libstdcxx-debug --enable-libstdcxx-time=yes --with-default-libstdcxx-abi=new --enable-gnu-unique-object --disable-vtable-verify --enable-plugin --enable-default-pie --with-system-zlib --with-target-system-zlib=auto --enable-objc-gc=auto --enable-multiarch --disable-werror --with-arch=32=i686 --with-abi=m64 --with-multilib-list=m32,m64,mx32 --enable-multilib --with-tune=generic --enable-offload-targets=nvptx-none=/build/gcc-9-9QD0t0/gcc-9-9.4.0/debian/tmp-nvptx/usr,hsa --without-cuda-driver --enable-checking=release --build=x86_64-linux-gnu --host=x86_64-linux-gnu --target=x86_64-linux-gnu
Thread model: posix
gcc version 9.4.0 (Ubuntu 9.4.0-1ubuntu1~20.04.2)
murphy@ubuntu:/home/workspace/course$
```

2.3.3 gcc编译

gcc命令的建议格式如下：

- 1 gcc -o output input -option

A terminal window titled 'Terminal' with a dark background. The prompt is 'murphy@ubuntu: /home/workspace/course'. The user enters 'gcc -o bin hello-world.c', followed by './bin', which outputs 'hello world'. The prompt returns to 'murphy@ubuntu: /home/workspace/course\$'.

```
murphy@ubuntu: /home/workspace/course$ gcc -o bin hello-world.c
murphy@ubuntu: /home/workspace/course$ ./bin
hello world
murphy@ubuntu: /home/workspace/course$
```

gcc常见命令：

1. `[gcc -E]`: 预编译
2. `[gcc -S]`: 生成汇编
3. `[gcc -c]`: 生成目标文件
4. `[gcc -o]`: 生成可执行文件
5. `[gcc -I]`: 添加头文件目录
6. `[gcc -D]`: 定义宏
7. `[gcc -g]`: 添加调试信息
8. `[gcc -v]`: 打印gcc版本信息

2.4 程序调试器gdb

2.4.1 gdb简介

gdb是GNU Project Debugger的简称，也是GNU(GNU's Not Unix)开源项目中遵循GPL发行许可证的free software。掌握gdb调试技巧，在项目上遇到问题才会得心应手。gdb 官方网站 (<https://sourceware.org/gdb/>)

一般来说，GDB 主要能够提供以下四个方面的帮助：

GDB can do four main kinds of things (plus other things in support of these) to help you catch bugs in the act:

- Start your program, specifying anything that might affect its behavior. (指定一些参数)

- Make your program stop on specified conditions. (断点)
- Examine what has happened, when your program has stopped. (分析crash现场)
- Change things in your program, so you can experiment with correcting the effects of one bug and go on to learn about another. (直接修改程序, 看结果)

GDB: The GNU Project Debugger

[\[bugs\]](#) [\[maintainers\]](#) [\[contributing\]](#) [\[current git\]](#) [\[documentation\]](#) [\[download\]](#) [\[home\]](#) [\[irc\]](#) [\[links\]](#) [\[mailing lists\]](#) [\[news\]](#) [\[schedule\]](#) [\[song\]](#) [\[wiki\]](#)

GDB: The GNU Project Debugger



What is GDB?

GDB, the GNU Project debugger, allows you to see what is going on 'inside' another program while it executes -- or what another program was doing at the moment it crashed.

GDB can do four main kinds of things (plus other things in support of these) to help you catch bugs in the act:

- Start your program, specifying anything that might affect its behavior.
- Make your program stop on specified conditions.
- Examine what has happened, when your program has stopped.
- Change things in your program, so you can experiment with correcting the effects of one bug and go on to learn about another.

Those programs might be executing on the same machine as GDB (native), on another machine (remote), or on a simulator. GDB can run on most popular UNIX and Microsoft Windows variants, as well as on macOS.

What Languages does GDB Support?

GDB supports the following languages (in alphabetical order):

- Ada
- Assembly
- C
- C++
- D
- Fortran
- Go
- Objective-C
- OpenCL
- Modula-2
- Pascal
- Rust

GDB version 15.1

Version [15.1](#) of GDB, the GNU Debugger, is now available for [download](#). See the [ANNOUNCEMENT](#) for details including changes in this release.

An errata list ([PROBLEMS](#)) and [documentation](#) are also available.

News

July 7th, 2024: **GDB 15.1 Released!**

2.4.2 安装gdb

```
1 sudo apt-get update
2 sudo apt-get install gdb
3
4 #查看gdb 版本信息
5 gdb --version
6 #启动gdb
7 gdb xx -q
8 gdb -p 进程号
9 gdb xx core
```

```
Activities Terminal Aug 20 06:54
murphy@ubuntu: /home/workspace/course
murphy@ubuntu:/home/workspace/course$ sudo apt-get install gdb
Reading package lists... Done
Building dependency tree
Reading state information... Done
gdb is already the newest version (9.2-0ubuntu1~20.04.2).
0 upgraded, 0 newly installed, 0 to remove and 97 not upgraded.
murphy@ubuntu:/home/workspace/course$ gdb ./bin -q
Reading symbols from ./bin...
(gdb) run
Starting program: /home/workspace/course/bin
hello world
[Inferior 1 (process 16835) exited normally]
(gdb) █
```

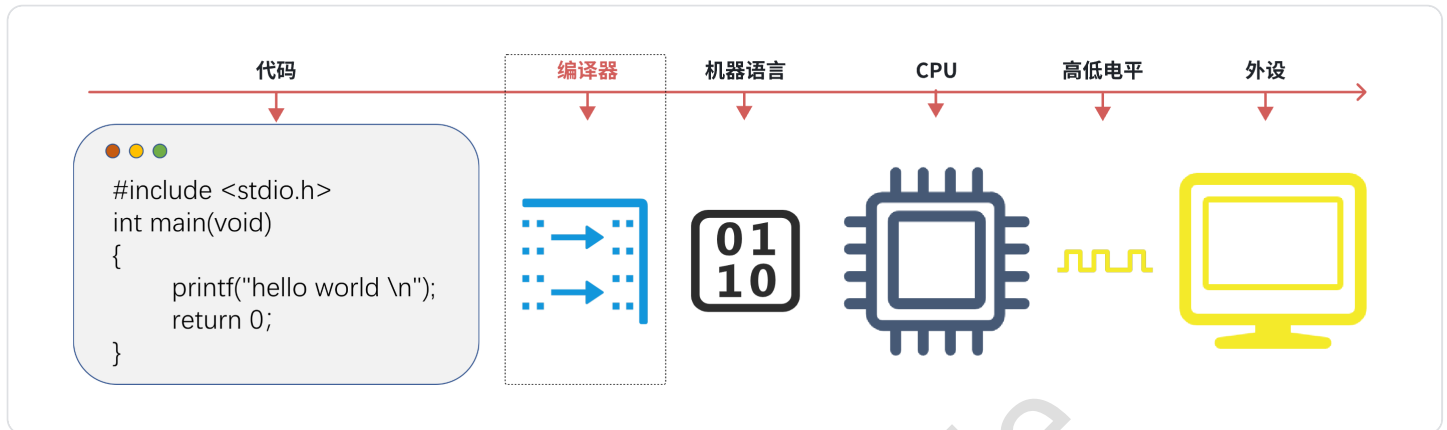
2.4.3 gdb常见命令

1. **[gdb xx]**: 启动xx可执行程序调试
2. **[gdb xx -q]**: 启动xx可执行程序调试，并忽略gdb版本信息打印
3. **[l 行号]**: 展示行号附近的代码
4. **[r]**: 运行程序
5. **[b 行号/函数名]**: 设置断点
6. **[info b]**: 查看断点
7. **[d xx]**: 删除断点
8. **[c]**: 继续执行
9. **[n]**: 单步执行
10. **[p xx]**: 打印变量的值
11. **[set var xx = xx]**: 设置变量的值
12. **[x 内存地址]**: 查看对应内存地址的值
13. **[info r]**: 查看寄存器的值
14. **[info thread]**: 查看信息信息
15. **[thread 线程号]**: 切换运行上下文到对应线程
16. **[disass]**: 展示当前汇编

3. 编译器视角

3.1 编译过程

编译器：程序员和计算机之间的翻译官。



输入命令：gcc -o bin 1.c -v 可得到编译全过程的日志（重点看紫色部分）：

```
1 /usr/lib/gcc/x86_64-linux-gnu/9/cc1 -quiet -v -imultiarch x86_64-linux-gnu 1.c
  -quiet -dumpbase 1.c -mtune=generic -march=x86-64 -auxbase 1 -version -
  fasynchronous-unwind-tables -fstack-protector-strong -Wformat -Wformat-
  security -fstack-clash-protection -fcf-protection -o /tmp/ccPNBd1N.s
2 GNU C17 (Ubuntu 9.4.0-1ubuntu1~20.04.2) version 9.4.0 (x86_64-linux-gnu)
3     compiled by GNU C version 9.4.0, GMP version 6.2.0, MPFR version
  4.0.2, MPC version 1.1.0, isl version isl-0.22.1-GMP
4
5 GGC heuristics: --param ggc-min-expand=100 --param ggc-min-heapsize=131072
6 ignoring nonexistent directory "/usr/local/include/x86_64-linux-gnu"
7 ignoring nonexistent directory "/usr/lib/gcc/x86_64-linux-gnu/9/include-fixed"
8 ignoring nonexistent directory "/usr/lib/gcc/x86_64-linux-
  gnu/9/../../../../x86_64-linux-gnu/include"
9 #include "..." search starts here:
10 #include <...> search starts here:
11 /usr/lib/gcc/x86_64-linux-gnu/9/include
12 /usr/local/include
13 /usr/include/x86_64-linux-gnu
14 /usr/include
15 End of search list.
16 GNU C17 (Ubuntu 9.4.0-1ubuntu1~20.04.2) version 9.4.0 (x86_64-linux-gnu)
17     compiled by GNU C version 9.4.0, GMP version 6.2.0, MPFR version
  4.0.2, MPC version 1.1.0, isl version isl-0.22.1-GMP
18
19 GGC heuristics: --param ggc-min-expand=100 --param ggc-min-heapsize=131072
20 Compiler executable checksum: 01da938ff5dc2163489aa33cb3b747a7
```

```

21 COLLECT_GCC_OPTIONS='-o' 'bin' '-v' '-mtune=generic' '-march=x86-64'
22 as -v --64 -o /tmp/cc6pbgpK.o /tmp/ccPNBd1N.s
23 GNU assembler version 2.34 (x86_64-linux-gnu) using BFD version (GNU Binutils
    for Ubuntu) 2.34
24 COMPILER_PATH=/usr/lib/gcc/x86_64-linux-gnu/9:/usr/lib/gcc/x86_64-linux-
    gnu/9:/usr/lib/gcc/x86_64-linux-gnu:/usr/lib/gcc/x86_64-linux-
    gnu/9:/usr/lib/gcc/x86_64-linux-gnu/
25 LIBRARY_PATH=/usr/lib/gcc/x86_64-linux-gnu/9:/usr/lib/gcc/x86_64-linux-
    gnu/9/../../../../x86_64-linux-gnu:/usr/lib/gcc/x86_64-linux-
    gnu/9/../../../../lib:/lib/x86_64-linux-gnu:/lib/./lib:/usr/lib/x86_64-
    linux-gnu:/usr/lib/./lib:/usr/lib/gcc/x86_64-linux-
    gnu/9/../../../../lib:/usr/lib/
26 COLLECT_GCC_OPTIONS='-o' 'bin' '-v' '-mtune=generic' '-march=x86-64'
27 /usr/lib/gcc/x86_64-linux-gnu/9/collect2 -plugin /usr/lib/gcc/x86_64-linux-
    gnu/9/liblto_plugin.so -plugin-opt=/usr/lib/gcc/x86_64-linux-gnu/9/lto-wrapper
    -plugin-opt=-fresolution=/tmp/ccE7eW0L.res -plugin-opt=-pass-through=-lgcc -
    plugin-opt=-pass-through=-lgcc_s -plugin-opt=-pass-through=-lc -plugin-opt=-
    pass-through=-lgcc -plugin-opt=-pass-through=-lgcc_s --build-id --eh-frame-hdr
    -m elf_x86_64 --hash-style=gnu --as-needed -dynamic-linker /lib64/ld-linux-x86-
    64.so.2 -pie -z now -z relro -o bin /usr/lib/gcc/x86_64-linux-
    gnu/9/../../../../x86_64-linux-gnu/Scrt1.o /usr/lib/gcc/x86_64-linux-
    gnu/9/../../../../x86_64-linux-gnu/crti.o /usr/lib/gcc/x86_64-linux-
    gnu/9/crtbeginS.o -L/usr/lib/gcc/x86_64-linux-gnu/9 -L/usr/lib/gcc/x86_64-
    linux-gnu/9/../../../../x86_64-linux-gnu -L/usr/lib/gcc/x86_64-linux-
    gnu/9/../../../../lib -L/lib/x86_64-linux-gnu -L/lib/./lib -L/usr/lib/x86_64-
    linux-gnu -L/usr/lib/./lib -L/usr/lib/gcc/x86_64-linux-gnu/9/../../../../
    /tmp/cc6pbgpK.o -lgcc --push-state --as-needed -lgcc_s --pop-state -lc -lgcc --
    push-state --as-needed -lgcc_s --pop-state /usr/lib/gcc/x86_64-linux-
    gnu/9/crtendS.o /usr/lib/gcc/x86_64-linux-gnu/9/../../../../x86_64-linux-
    gnu/crtn.o
28 COLLECT_GCC_OPTIONS='-o' 'bin' '-v' '-mtune=generic' '-march=x86-64'

```

3.1.1 预编译

以下面这段简单的代码为例：

```

1 #include <stdio.h>
2 #define ABC 123
3
4 int main(void)
5 {
6     int a = ABC;
7     printf("hello world \n");
8
9     return 0;

```

```
10 }
```

预编译做需要两个事情：

- 包含头文件：#include。把需要文件包含进来，不仅包含我们自己手动include的，也包含了必需的系统文件。
- 宏替换：#define。宏替换的过程是纯文本替换，预编译阶段不会检测语法的准确性。如ABC直接替换成123

输入命令：gcc -E 可得到以下预处理后的结果：

```
1 # 1 "1.c"
2 # 1 "<built-in>"
3 # 1 "<command-line>"
4 # 31 "<command-line>"
5 # 1 "/usr/include/stdc-predef.h" 1 3 4
6 .....
7 # 873 "/usr/include/stdio.h" 3 4
8 # 2 "1.c" 2
9 # 5 "1.c"
10 int main(void)
11 {
12     int a = 123;
13
14     printf("hello world \n");
15
16     return 0;
17 }
```

- 1 Q: #include "xx.c" 包含C文件可以吗？
- 2 A: 可以，但是不建议

3.1.2 翻译成汇编

编译器调用cc1(C compiler 1)工具把C的代码翻译成汇编。

- 1 #tips: cc1 默认会做预编译的过程。
- 2 /usr/lib/gcc/x86_64-linux-gnu/9/cc1 -quiet -v -imultiarch x86_64-linux-gnu 1.c
-quiet -dumpbase 1.c -mtune=generic -march=x86-64 -auxbase 1 -version -
fasynchronous-unwind-tables -fstack-protector-strong -Wformat -Wformat-
security -fstack-clash-protection -fcf-protection -o /tmp/ccPNBd1N.s

以上命令等价于：gcc -S

```
1 gcc -o 1.s 1.c -S -v
```

打开汇编代码（vim 1.s）结果如下（x86平台）。其实就是一些指令，这些指令跟具体的体系结构有关，不同的体系架构，相同的C语言，汇编也是不一样的。大家先建立这个认知即可

```
1 .....
2 main:
3 .LFB0:
4     .cfi_startproc
5     endbr64
6     pushq   %rbp
7     .cfi_def_cfa_offset 16
8     .cfi_offset 6, -16
9     movq    %rsp, %rbp
10    .cfi_def_cfa_register 6
11    subq     $16, %rsp
12    movl     $123, -4(%rbp)
13    leaq     .LC0(%rip), %rdi
14    call     puts@PLT
15    movl     $0, %eax
16    leave
17    .cfi_def_cfa 7, 8
18    ret
19    .cfi_endproc
20 .....
```

```
1 Q: 为什么要先翻译成汇编，而不直接编译成机器码？
2 A:
```

3.1.3 翻译成机器码

编译器调用as工具把汇编代码翻译成目标文件(.o)。

```
1 as -v --64 -o /tmp/cc6pbgpK.o /tmp/ccPNBd1N.s
2 GNU assembler version 2.34 (x86_64-linux-gnu) using BFD version (GNU Binutils
   for Ubuntu) 2.34
```

```
3 COMPILER_PATH=/usr/lib/gcc/x86_64-linux-gnu/9:/usr/lib/gcc/x86_64-linux-  
gnu/9:/usr/lib/gcc/x86_64-linux-gnu:/usr/lib/gcc/x86_64-linux-  
gnu/9:/usr/lib/gcc/x86_64-linux-gnu/
```

以上命令等价于: gcc -c

```
1 gcc -o 1.o 1.c -c -v
```

打开目标文件（vim 1.o）结果如下（x86平台）。给人第一感觉就是，啥都看不懂。看不懂就对了，这是给机器看的。到这里，我们自己写的代码已经可以让机器看懂了，但想要在机器上跑起来，还需要系统的一些资源（比如 共享库）。

1 ?

ELF^B^A^A^@^X^C^
@^@^@^@^@^@^@^@^@^@^@^@^@^@^@^N^@M^@ó^O^úUH<89>åH<83>ì^PÇEü{^@^@^@H<8d>=^
@^@^@^@è^@^@^@^@,^@^@^@^@ÉÃhello world ^@^@GCC: (Ubuntu 9.4.0-
1ubuntu1~20.04.2)
9.4.0^@^@D^@^@^@P^@^@^@E^@^@^@GNU^@^B^@^@Ä^D^@^@^@C^@^@^@^@^@^@T^@^@^@^@
^@^@^@AzR^@^Ax^PA^[^L^G^H<90>^A^@^@^\^@^@^@\^@^@^@^@^@^@&^@^@^@^@E^N^P<86>
^BC^M^F]^L

2

3.1.4 链接成可执行文件

有了目标文件，编译器会调用collect2工具把目标文件链接成可执行文件。可以看到，除了我们自己代码编出来的.o，编译器还包含系统很多的.o。所以一个简单的hello world，系统已经帮我们做了很多事情，才能真正跑起来。

```
1 /usr/lib/gcc/x86_64-linux-gnu/9/collect2 -plugin /usr/lib/gcc/x86_64-linux-  
gnu/9/liblto_plugin.so -o bin /tmp/cc6pbgpK.o -lgcc --push-state --as-needed -  
lgcc_s --pop-state -lc -lgcc --push-state --as-needed -lgcc_s --pop-state  
/usr/lib/gcc/x86_64-linux-gnu/9/crtendS.o /usr/lib/gcc/x86_64-linux-  
gnu/9/../../../../x86_64-linux-gnu/crtn.o  
2 .....
```

以上命令等价于: gcc -o

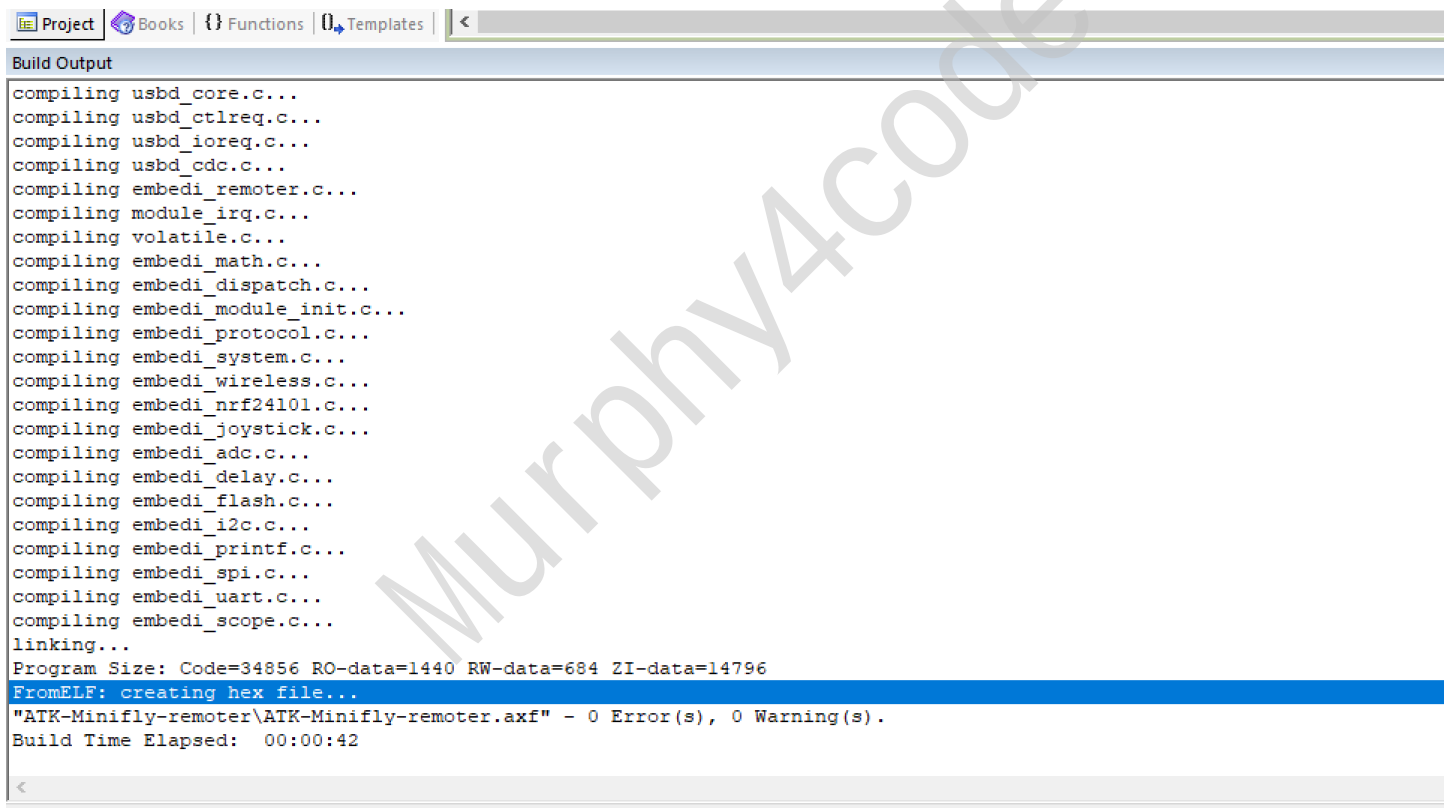
```
1 gcc -o bin 1.o -v
```

```
2
3 ./bin
4 hello world
```

- 1 Q: 链接的优势是什么?
- 2 A:

3.1.5 格式转换

编译过程一般到链接就结束了，但这里要提一下格式转化。单片机裸机和操作系统在这里有些不同，可执行文件（elf或者exe）真正被运行起来，还有一个二进制格式转化过程。对于单片机来说，这个由编译器来做，如stm32中有fromelf工具，编译时会把elf转化成hex，最后才下载到单片机的flash中；而在linux上，是由elf解析器在加载运行时完成。



```
Project | Books | Functions | Templates | <
Build Output
compiling usbd_core.c...
compiling usbd_ctlreq.c...
compiling usbd_ioreq.c...
compiling usbd_cdc.c...
compiling embedi_remoter.c...
compiling module_irq.c...
compiling volatile.c...
compiling embedi_math.c...
compiling embedi_dispatch.c...
compiling embedi_module_init.c...
compiling embedi_protocol.c...
compiling embedi_system.c...
compiling embedi_wireless.c...
compiling embedi_nrf24l01.c...
compiling embedi_joystick.c...
compiling embedi_adc.c...
compiling embedi_delay.c...
compiling embedi_flash.c...
compiling embedi_i2c.c...
compiling embedi_printf.c...
compiling embedi_spi.c...
compiling embedi_uart.c...
compiling embedi_scope.c...
linking...
Program Size: Code=34856 RO-data=1440 RW-data=684 ZI-data=14796
FromELF: creating hex file...
"ATK-Minifly-remoter\ATK-Minifly-remoter.axf" - 0 Error(s), 0 Warning(s).
Build Time Elapsed: 00:00:42
<
```

单片机

```
1 Symbol table '.dynsym' contains 7 entries:
2   Num:      Value              Size Type   Bind   Vis      Ndx Name
3     0: 0000000000000000          0 NOTYPE  LOCAL  DEFAULT  UND
4     1: 0000000000000000          0 NOTYPE  WEAK   DEFAULT  UND
      _ITM_deregisterTMCloneTab
5     2: 0000000000000000          0 FUNC    GLOBAL DEFAULT  UND puts@GLIBC_2.2.5 (2)
6     3: 0000000000000000          0 FUNC    GLOBAL DEFAULT  UND
      __libc_start_main@GLIBC_2.2.5 (2)
```

```

7      4: 0000000000000000      0 NOTYPE  WEAK  DEFAULT  UND __gmon_start__
8      5: 0000000000000000      0 NOTYPE  WEAK  DEFAULT  UND
   _ITM_registerTMCloneTable
9      6: 0000000000000000      0 FUNC    WEAK  DEFAULT  UND
   __cxa_finalize@GLIBC_2.2.5 (2)

```

linux

3.1.6 常见的编译错误

3.1.6.1 预编译错误

- 1、头文件找不到：缺失或者路径不对

```

1 #include <stdio.h>
2 #include "abc.h"
3
4 int main(void)
5 {
6     printf("hello world \n");
7
8     return 0;
9 }

```

```

1 1.c:2:10: fatal error: abc.h: No such file or directory
2     2 | #include "abc.h"
3       |             ^~~~~~
4 compilation terminated.

```

创建adb.h并添加头文件路径：gcc -I

```

1 #在inc目录下增加abc.h
2 gcc -o bin 1.c -I inc/

```

- 2、条件编译不成对：#ifdef #endif 不成对、#if #endif不成对

```

1 #include <stdio.h>
2 #include "abc.h"
3

```

```

4 int main(void)
5 {
6 #ifdef ABC
7     printf("hello world \n");
8 //#endif
9     return 0;
10 }

```

```

1 1.c:6: error: unterminated #ifdef
2     6 | #ifdef ABC
3       |

```

3.1.6.2 编译错误

各种语法错误：

- 1、缺少；

```

1 #include <stdio.h>
2
3 int a = 1
4
5 int main(void)
6 {
7     printf("hello world \n");
8
9     return 0;
10 }

```

```

1 1.c:5:1: error: expected ',' or ';' before 'int'
2     5 | int main(void)
3       |     ^~~

```

- 2、{}、[]等符号不成对

```

1 include <stdio.h>
2
3 int main(void)
4 {
5     printf("hello world \n");

```

```
6
7     return 0;
```

```
1 1.c: In function 'main':
2 1.c:7:2: error: expected declaration or statement at end of input
3     7 | return 0;
4       |
```

3.1.6.3 链接错误

- 1、变量或者函数缺少定义

```
1 #include <stdio.h>
2
3 void func(void);
4
5 int main(void)
6 {
7     func();
8     printf("hello world \n");
9
10    return 0;
11 }
```

```
1 /usr/bin/ld: 1.o: in function `main':
2 1.c:(.text+0x9): undefined reference to `func'
3 collect2: error: ld returned 1 exit status
4
```

- 2、变量或者函数重复定义

```
1 #file 1.c
2 #include <stdio.h>
3
4 void func(void);
5
6 int main(void)
7 {
8     func();
```

```

9     printf("hello world \n");
10
11     return 0;
12 }
13
14 #file 2.c
15 #include <stdio.h>
16
17 void func(void)
18 {
19     printf("func \n");
20 }

```

```

1 /usr/bin/ld: 2.o: in function `func':
2 2.c:(.text+0x0): multiple definition of `func'; 1.o:1.c:(.text+0x0): first
   defined here
3 collect2: error: ld returned 1 exit status

```

3.2 预处理

预处理在工程上有很多的用处，特别是在代码管理、调试上有很多编程技巧可以用。

3.2.1 条件编译

- 1、调试版本和release版本（调试信息的脱敏、性能的影响：code大小、运行效率）

```

1 #include <stdio.h>
2 int password = 0x37847110;
3 int main(void)
4 {
5     printf("debug log: password is %x\n", password);
6     return 0;
7 }

```

问：以上打印的debug log，怎么样在不修改代码的情况下去掉？

代码可以改造成这样：

```

1 #include <stdio.h>
2
3 int password = 0x37847110;
4

```

```

5 int main(void)
6 {
7 #ifdef DEBUG
8     printf("debug log: password is %x\n", password);
9 #else
10    printf("release version \n");
11 #endif
12    return 0;
13 }

```

编译时用gcc -D xx选项定义宏

```

1 gcc -o bin 1.c -D DEBUG
2 ./bin
3 #程序运行结果
4 log: password is 37847110

```

- 2、feature的配置（linux里面大量的feature宏设计）

```

1 #include <stdio.h>
2
3 #ifdef TOUCH_PANEL_DERIVER
4 void tp_driver(void)
5 {
6     printf("tp init \n");
7 }
8 #endif
9
10 int main(void)
11 {
12 #ifdef TOUCH_PANEL_DERIVER
13     tp_driver();
14 #endif
15     printf("main end\n");
16     return 0;
17 }

```

```

1 gcc -o bin 1.c -D TOUCH_PANEL_DERIVER
2 ./bin
3 #程序运行结果
4 tp init

```

```
5 main end
```

- 3、通过#warning、#error进行配置提示

如上面这个例子，如果想让程序员实现touch panel的驱动，可以在预编译的时候直接提示或者直接报错。

```
1 #include <stdio.h>
2
3 #ifndef TOUCH_PANEL_DERIVER
4 #warning "You should complement TP driver"
5 #endif
6
7 #ifdef TOUCH_PANEL_DERIVER
8 void tp_driver(void)
9 {
10     printf("tp init \n");
11 }
12 #endif
13
14 int main(void)
15 {
16 #ifdef TOUCH_PANEL_DERIVER
17     tp_driver();
18 #endif
19     printf("main end\n");
20     return 0;
21 }
```

```
1 gcc -o bin 1.c
2 ./bin
3 #程序运行结果
4 cc -o bin 1.c
5 1.c:4:2: warning: #warning "You should complement TP driver" [-Wcpp]
6     4 | #warning "You should complement TP driver"
7       | ^~~~~~
```

3.2.2 宏定义使用

宏展开过程时纯粹文本替换，不进行语法检查。

```
1 #define 宏名 宏体
```

```

2
3 #建议1:定义宏体用()包起来, 不要吝啬 ()
4 #建议2:宏名一般都是大写字母
5 #define ABC (123)
6 #define SECEND_OF_YEAR (365*24*60*60)UL
7
8 #多行可以用“\”进行换行
9 #define IIC_SDA(status) \
10     do { \
11         GPIO_InitTypeDef GPIO_InitStructure = { \
12             .Pin = _SDA, \
13             .Mode = GPIO_MODE_OUTPUT_PP, \
14             .Pull = GPIO_NOPULL, \
15             .Speed = GPIO_SPEED_FREQ_HIGH, \
16         }; \
17         HAL_GPIO_Init(_SDA_PORT, &GPIO_InitStructure); \
18         HAL_GPIO_WritePin(_SDA_PORT, _SDA, status); \
19     } while (0)
20
21 Q: 在Linux内核中有许多使用do{...}while(0)的宏定义。这种宏的用途是什么? 有什么好处?
22 A: do {...} while(0)在C中是唯一的构造程序, 让你定义的宏总是以你希望的方向工作, 这样不管
    怎么使用宏。感受一下这个例子:
23
24 1、
25 #define FUNC(a, b) a++; b++
26
27 if (xx)
28     FUNC(a, b);
29
30 //展开后语法错误
31 if (xx)
32     a++;
33     b++;
34
35 2、
36 #define FUNC(a, b) {a++; b++}
37
38 if (xx)
39     FUNC(a, b);
40 else
41     xx
42
43 //展开后语法错误
44 if (xx) {
45     a++;
46     b++;
47 };

```

```

48 else
49     xx
50
51 3、
52 #define FUNC(a, b) do {a++; b++;} while(0)
53 //展开后语法错误
54 if (xx)
55     do {
56         a++;
57         b++;
58     } while(0);
59 else
60     xx

```

1 Q: 下面代码运行是否符合预期?

```

2 #define ABC 10+2
3
4 void main(void)
5 {
6     int a = ABC * 10;
7     printf("a = %d \n", a);
8 }
9
10 A: 会有问题, 宏展开后是10+2*10, 不符合预期

```

1 Q: 下面代码编译完后, 在运行时是否有乘法计算?

```

2
3 #define SECEND_OF_YEAR (365*24*60*60)UL
4 void main(void)
5 {
6     int a = SECEND_OF_YEAR;
7
8     printf("a = %d \n", a);
9 }
10
11 A: 没有, 编译器已经完成的计算

```

3.2.3 宏函数

3.2.3.1 宏函数的定义

宏函数本质不是函数。展开规则依旧是纯粹文本替换，不会检测语法错误。正因如此，宏函数直接插入调用的地方，少了函数调用的消耗（出栈入栈等操作），所以是典型的用空间换性能的做法。同样的，为了宏展开不会出错建议：**整体和变量都加上()**

```
1 #define 宏名(x) (x)
2
3 #define ADD(a, b) ((a)+(b))
4 #define RAD2DEG (x) ((x)*57.295f)
5 #define F2C (x) (((x)-32)*5/9)
```

```
1 Q: 比较两个数大小的代码运行是否符合预期?
2 #include <stdio.h>
3 #define MAX(a, b) ((a)>(b)?(a):(b))
4
5 int main(void)
6 {
7     int a = 2;
8     int b = 3;
9
10    printf("a = %d, b = %d, the bigger value = %d \n",
11           a, b, MAX(a++, b++));
12    return 0;
13 }
14 期望: a = 3, b = 2, the bigger value = 3
15
16 A:
17
```

```
1 #参考linux下MAX的实现(3.18内核)
2 #define MAX(x, y) ({
3     typeof(x) _max1 = (x);
4     typeof(y) _max2 = (y);
5     (void) (&_max1 == &_max2);
6     _max1 > _max2 ? _max1 : _max2; })
```

3.2.3.2 '#' 字符串化

```
1 #define LOG_INFO(x) printf("%s is %d \n", #x, (x))
2
3 int main(void)
```

```

4 {
5     int a = 2;
6
7     LOG_INFO(a);
8     return 0;
9 }
10
11 #程序运行结果&
12 a is 2

```

3.2.3.3 '##' 连接符号

```

1 #include <stdio.h>
2
3 #define DAY_DECLARE(x) int day##x
4 #define DAY(x) day##x
5
6 DAY_DECLARE(1);
7 DAY_DECLARE(2);
8 int main(void)
9 {
10     DAY(1) = 10;
11     DAY(2) = 20;
12
13     printf("day1:%d day2:%d \n", DAY(1), DAY(2));
14     return 0;
15 }
16
17 #程序运行结果
18 day1:10 day2:20

```

3.2.4 系统预定义的宏

- `__FUNCTION__`: 函数名
- `__LINE__`: 调用所在行数
- `__FILE__`: 函数所在文件名

```

1 #include <stdio.h>
2
3 #define LOG_INFO(x) printf("%s %s:line:%d: %d \n", \
4                             __FILE__, \
5                             __FUNCTION__, \

```

```

6             __LINE__,\
7             (x))
8 int main(void)
9 {
10     int a = 10;
11
12     LOG_INFO(a);
13     return 0;
14 }
15
16 #程序运行结果
17 1.c main:line:12: 10

```

3.2.5 常用的log宏

```

1 #include <stdio.h>
2
3 #define TAG "ABC"
4 #define LOG_INFO(format,...) printf("[%s][%s:%d]: "format"", TAG, __func__ ,
   __LINE__, ##__VA_ARGS__)
5
6 int main(void)
7 {
8     int a = 10;
9
10    LOG_INFO("a = %d \n",a);
11    return 0;
12 }
13
14 #程序运行结果
15 [ABC][main:10]: a = 10

```

3.3 关键字（32个）

编译器翻译代码的过程是逐行字符解析，所以C语言标准里面预定义了32个有特定意义的"字符串"，当编译器读到这些字符串就知道程序员想表达的意思。关键字大体上可以分成3大类：数据类型关键字、修饰关键字、逻辑关键字。

3.3.1 sizeof关键字

sizeof (x) : 编译器给我们查看内存空间容量的一个工具，在**编译时**计算并确定出x在内存中所占字节数。

3.3.1.1 sizeof(常量/变量/数据类型)

sizeof传入参数可以是常量、变量、数据类型。

```
1 #include <stdio.h>
2
3 int main(void)
4 {
5     int a = 10;
6
7     printf("size of 10 is: %ld \n", sizeof(10));
8     printf("size of a is: %ld \n", sizeof(a));
9     printf("size of int is: %ld \n", sizeof(int));
10
11     return 0;
12 }
13
14 #程序运行结果
15 size of 10 is: 4
16 size of a is: 4
17 size of int is: 4
```

3.3.1.2 sizeof(函数)

sizeof传入函数时，得到的结果是计算出函数「返回值」的内存空间大小，函数本身不会被调用（侧面也反应sizeof是在编译时就计算出空间大小，而不是在运行时）。

```
1 #include <stdio.h>
2
3 int func(void)
4 {
5     printf("func call \n");//这句log不会被打印
6     return 0;
7 }
8
9 int main(void)
10 {
11     int a = 10;
12
13     printf("size of func is: %ld \n", sizeof(func()));
14     return 0;
15 }
16
17 #程序运行结果
18 size of func is: 4
```

3.3.1.3 常被误解成函数

- 1、sizeof使用时不一定要有 () ，但函数调用一定要有 () 。

```
1 #include <stdio.h>
2
3 int a = 10;
4
5 int main(void)
6 {
7     printf("size of a is: %ld", sizeof a);
8     return 0;
9 };
10
11 #程序运行结果
12 size of a is: 4
```

- 2、翻译成汇编就很清晰看到，sizeof在编译的时候就直接算出内存空间大小，并没有函数的入栈出栈、call 操作。可以体会一下 my_sizeof 和 sizeof 在汇编层面上的区别：

```
1 #include <stdio.h>
2
3 int a = 10;
4 int b = 0;
5 int c = 0;
6
7 int my_sizeof(int a)
8 {
9     return sizeof(a);
10 }
11
12 int main(void)
13 {
14     b = sizeof(a);
15     c = my_sizeof(a);
16     return 0;
17 }
```

```
1 main:
2 .LFB1:
3     ...
4     //b = sizeof(a);
```

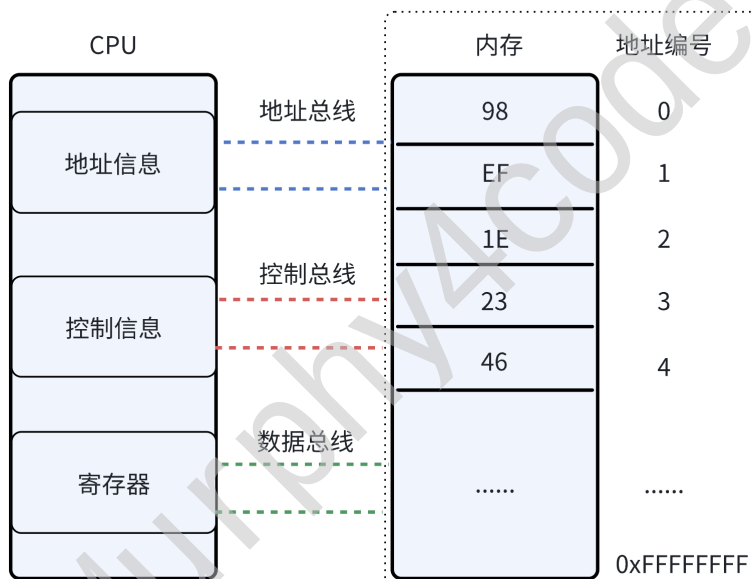
```

5      //编译时, 就计算出来 b = 4
6      movl    $4, b(%rip)
7
8      //c = my_sizeof(a);
9      movl    a(%rip), %eax
10     movl    %eax, %edi
11     call    my_sizeof
12     movl    %eax, c(%rip)
13     ...

```

3.3.2 数据类型关键字

我们知道C语言最终是为操作内存资源服务的。那操作内存必然要回答一个问题：如何来圈定一个要操作内存的大小？继续往下看：



3.3.2.1 标准类型

C语言标准给我们预定义了一系列的数据类型，我们逐一来思考这些数据类型关键字背后的逻辑是什么

3.3.2.1.1 char

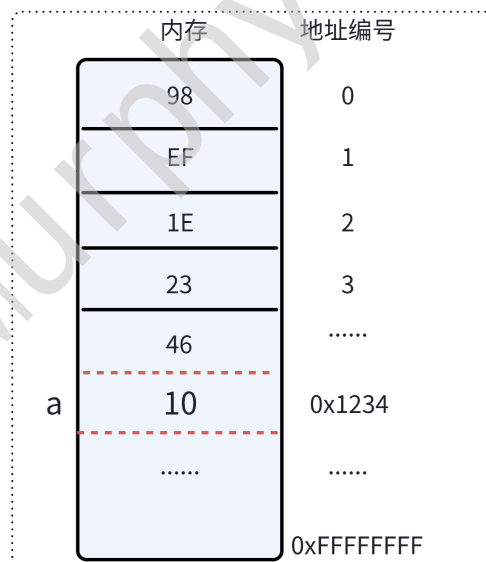
最小内存空间的数据类型，也是最适合操作硬件的数据类型。

我们知道硬件上最小的状态是高低电平，对于软件来讲就是0和1状态，那C语言中为什么没有bit这个数据类型关键字呢？因为0-1只有这两个状态，对于内存来讲当然是充分利用，但对于CPU来讲处理效率非常低。比如要把32 (0b100000) 这个数据从内存读到CPU内部寄存器，至少需要执行6个指令周期（1个指令周期：寻址->发读的控制信息->数据通过数据总线送到寄存器）才能读到0b100000。所以兼顾CPU性能和内存管理，计算机科学家把操作内存最小单位为8bit，也称1byte，也就是**char类型**，可以表示256种状态。

当编译器看到char关键字的时候，就知道圈定的内存大小是1byte：

```
1 #include <stdio.h>
2
3 int main(void)
4 {
5     char a = 9;
6     printf("a = %d, size of a is %ld byte \n", a, sizeof(a));
7     return 0;
8 }
9
10 #程序运行结果
11 a = 10, size of a is 1 byte
```

- 1 这里的 `char a = 10;` 相当于在内存中圈出一块1byte的内存，内存的标签叫a（也可以理解成这块内存属于a），内存里面的值是10
- 2 Q：想一想，声明和定义的区别？`extern char a` 和 `char a;`
- 3 A：声明没有分配内存，定义会分配内存。



所以内存的访问的方式分成：**标签访问和地址访问**。可以这么理解，假设你要去访问你朋友的家。你朋友带你去就是标签访问，你自己根据目的地导航过去就是地址访问。

- 字符空间

char类型除了表示一个具体数字之外，它还可以描述字符空间，即ASCII码。在计算机的世界里，只认识01不认识字符，所以ASCII是人为定义的一套映射标准(见附录 7.1)。

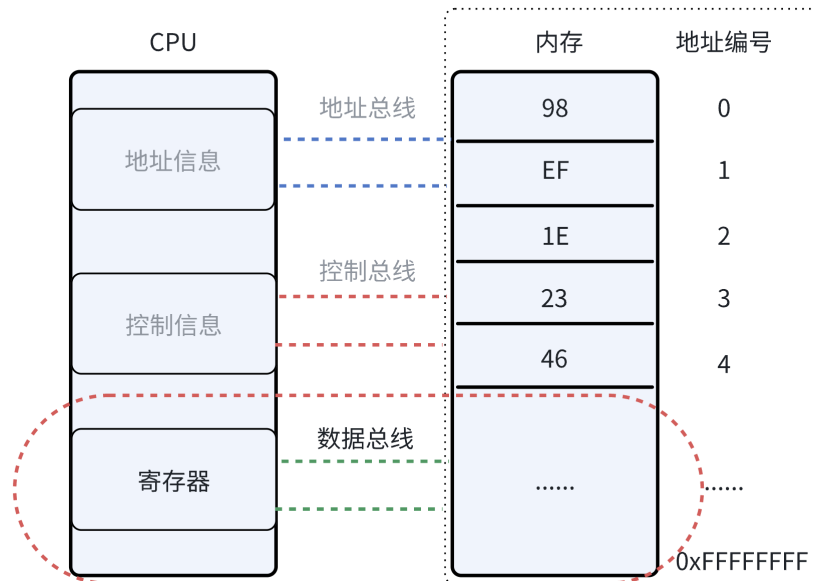
```
1 #include <stdio.h>
2
3 int main(void)
4 {
5     char a = 'a';
6     printf("a = %c, a ASCII = %d \n", a, a);
7     return 0;
8 }
9
10 #程序运行结果
11 a = a, a ASCII = 97
```

1 这里char a = 'a' 等价于char a = 97

```
1 #include <stdio.h>
2
3 int main(void)
4 {
5     char a = 97;
6     printf("a = %c, a ASCII = %d \n", a, a);
7     return 0;
8 }
9
10 #程序运行结果
11 a = a, a ASCII = 97
```

3.3.2.1.2 int

最适合CPU的数据类型，大小跟编译器有关。



为了充分发挥CPU的数据处理能力，数据总线尽量要充分利用，在系统一个周期内所能接受的最大处理单位就是int。所以32位的CPU，int就是32bit（4byte），16位的CPU，int就是16bit（2byte）。对于一个数字常量默认就是int的，体会以下代码：

```

1 #include <stdio.h>
2
3 int main(void)
4 {
5     int a = 10;
6
7     printf("size of 10 is: %ld \n", sizeof(10));
8     printf("size of a is: %ld \n", sizeof(a));
9     printf("size of int is: %ld \n", sizeof(int));
10
11     return 0;
12 }
13
14 #程序运行结果
15 size of 10 is: 4
16 size of a is: 4
17 size of int is: 4

```

3.3.2.1.3 short/long

short和long关键字本质是修饰int的，只是默认情况下我们用short和long类型的时候把int省略了。见名知意，short是减少int的空间，long是加大int的空间。具体的大小，由编译器决定

```

1 #include <stdio.h>

```

```

2
3 int main(void)
4 {
5     short int a = 1;
6     short b = 1;
7
8     long int c = 1;
9     long d = 1;
10    printf("size, a b: %ld, %ld byte; c d : %ld, %ld byte \n",
11           sizeof(a), sizeof(b), sizeof(c), sizeof(d));
12    return 0;
13 }
14
15 #程序运行结果
16 size, a b: 2, 2 byte; c d : 8, 8 byte

```

3.3.2.1.4 signed/unsigned

这两个关键字用于圈定变量的区间范围。signed代表有符号数，最高位代表正负（1：代表负，0：代表正），区间范围从0为支点一分为半。如，char是8bit，所以signed char范围[-128, 127]，unsigned char范围[0, 255]。一般情况可以省略signed不写，int、long、short默认就是signed，但是char在C的标准中，允许定义为signed和unsigned，所以由编译器决定。可以测试一下你的机器上char是否是有符号数：

```

1 #include <stdio.h>
2
3 int main(void)
4 {
5     char a = 127;
6
7     if (++a > 127) {
8         printf("char is unsigned \n");
9     } else {
10        printf("char is signed \n");
11    }
12
13    return 0;
14 }

```

3.3.2.1.5 float/double

float和double是浮点类型，即可以表示小数。float是4字节，double是8个字节。在一般的嵌入式中，浮点的运算相比于整形要慢，如果算法没有太多的精度要求，建议尽量用整形。

```

1 #include <stdio.h>
2
3 int main(void)
4 {
5     printf("float: %ld double %ld\n", sizeof(float), sizeof(double));
6     return 0;
7 }
8
9 #程序运行结果
10 float: 4 double 8

```

float和double是浮点类型在存储上存在精度，所以不能直接判断是否相等，而是人为的取一个精度，然后判断两个小数的模与这个精度的大小。

```

1 #include <stdio.h>
2 #include <math.h>
3
4 int main(void)
5 {
6     float a = 1.0;
7     float b = 0.1;
8
9     printf("    a = %.10f b = %.10f \n", a, b);
10    fabs((a - b) - 0.9) < 0.000001 ?
11        printf("    a - b equal 0.9\n") :
12        printf("    a - b not equal 0.9 \n");
13    return 0;
14 }

```

3.3.2.1.6 void

void类型不能用于定义变量，因为空大小的内存圈定没有意义。一般void用于函数占位符、void*、强制转换防止未用参数被编译器警告。

```

1 #include <stdio.h>
2
3 int main(void)
4 {
5     void a;
6     return 0;
7 }
8
9 #程序运行结果

```

```

10 1.c: In function 'main':
11 1.c:5:10: error: variable or field 'a' declared void
12     5 |     void a;
13       |
14

```

- 占位符：表示无或者空

```

1 #include <stdio.h>
2
3 //表示函数没有形参和返回值
4 void main(void)
5 {
6     printf("hello world \n");
7 }

```

- void *: 表示数据空间

```

1 #include <stdio.h>
2
3 void data_process(void *data, int len)
4 {
5     unsigned char* buff = (unsigned char *)data;
6     for (int i = 0; i < len; i++) {
7         ...
8         buff[i];
9     }
10 }

```

- 表示unused:

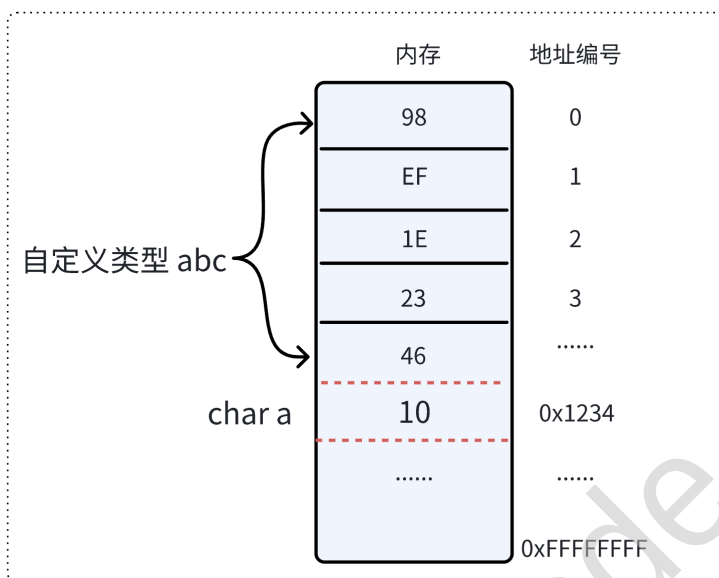
```

1 #include <stdio.h>
2
3 void func(int a)
4 {
5     int b;
6     //解决编译器警告：参数定义未使用
7     (void)a;
8 }

```

3.3.2.2 自定义类型

C语言默认定义的数据类型，不满足实际内存资源分配的。所以C语言支持在标准类型的基础上，通过组合来形成新的类型。本质上就是支持程序员根据实际需要来圈定内存的大小。



3.3.2.2.1 struct

C语言通过 struct 关键字表示新的组合类型，内存表现为累加且对齐，我们叫这种数据类型为**结构体**。声明的规则如下：

```
1 struct 名字 {
2     char b;
3     int a;
4     ....
5 };
6
7 Q: 下面这段代码有分配内存吗?
8 struct abc {
9     int a;
10    char b;
11 };
12 A: 这里只是声明，没有分配内存，a和b这里代表结构体的成员。
```

- 结构体的初始化和访问

```
1 #include <stdio.h>
2
3 struct abc {
```

```

4     int a;
5     char b;
6 };
7
8 int main(void)
9 {
10     struct abc c = {
11         .a = 1,
12         .b = 'c',
13     };
14
15     printf("a = %d b = %c \n",c.a, c.b);
16     return 0;
17 }
18
19 #程序运行结果
20 a = 1 b = c

```

- 字节对齐

什么是字节对齐，为什么要字节对齐？我们来体会一下下面这个例子：

```

1 #include <stdio.h>
2 struct abc {
3     char b;
4     int a;
5 };
6
7 int main(void)
8 {
9     struct abc c = {
10         .a = 1,
11         .b = 'c',
12     };
13
14     printf("int:%ld byte, char:%ld byte, abc:%ld byte\n",
15         sizeof(int), sizeof(char), sizeof(struct abc));
16     return 0;
17 }
18
19 #程序运行结果
20 int:4 byte, char:1 byte, abc:8 byte
21

```

22 可以看到，组合成新的结构体类型后，整体的内存变大了，不仅仅是简单的叠加。原因也很简单，就是想要充分利用CPU，提升性能。在32位的cpu中，每次读取的数据是4byte对齐访问，字节未对齐时，对

于成员变量a来说，需要读取两次才能拼凑成a的值；而对齐后，只需要一个机器周期就可以读取出来。这也是典型通过**空间换效率**的例子。

23

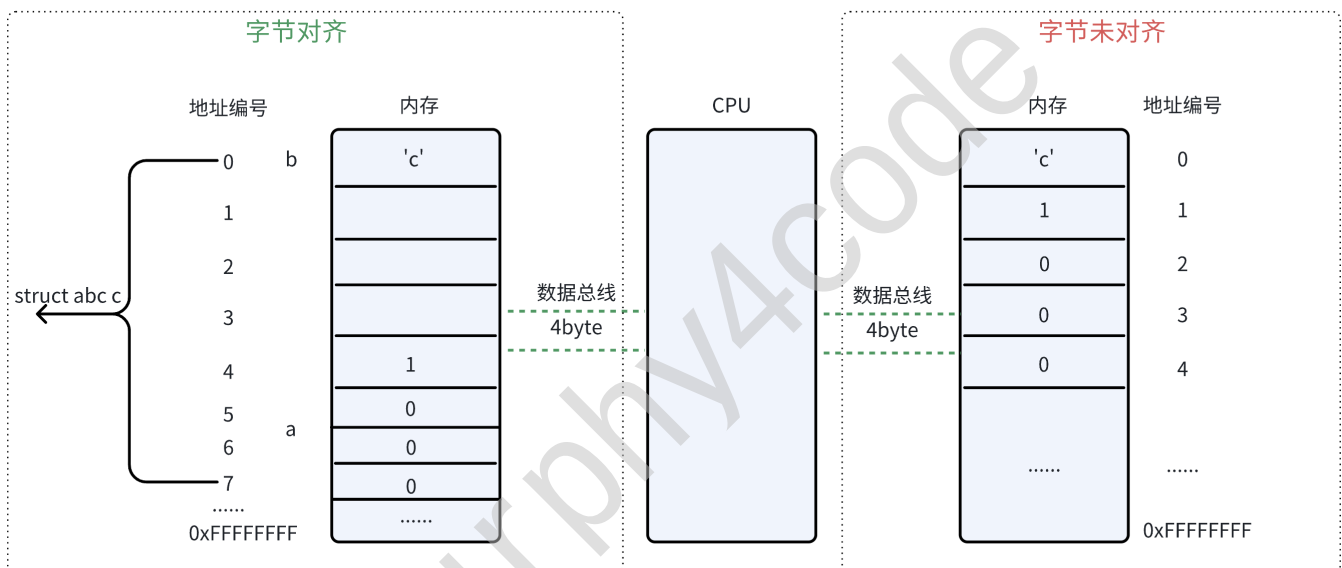
24 Tips1: 在定义结构体变量时，编译器默认会把首地址对齐到4字节（跟具体体系结构有关，也可能是2个字节对齐）的倍数：**首地址对齐**

25 Tips2: **边界对齐**

26 1、部分CPU硬件支持**非对齐访问**，典型的就X86，X86硬件会自动处理非对齐访问情况，对软件透明，代价是牺牲效率（纯硬件做的数据拼接）

27 2、部分CPU“部分支持”非对齐访问，典型的就ARM，其“单指令”操作支持非对齐，但“群指令”操作（SIMD）则不支持（必须对齐访问）

28 3、部分CPU硬件不支持对齐访问，但通过软件支持。典型的就部分mips架构，其通过内核中对alignment fault异常处理流程中进行处理，比如将非对齐的数据访问，通过多次访存操作和拼接操作来处理，也可以使用类似memcpy的方式来处理，当然代价是更严重的性能损失



1 Q: abc1 和 abc2 的内存空间一样吗?

```
2 #include <stdio.h>
```

```
3 struct abc1 {
```

```
4     char b;
```

```
5     int a;
```

```
6     short c;
```

```
7 };
```

```
8
```

```
9 struct abc2 {
```

```
10     char b;
```

```
11     short c;
```

```
12     int a;
```

```
13 };
```

```

14
15 int main(void)
16 {
17     printf("abc1:%ld byte, abc2:%ld byte\n", sizeof(struct abc1), sizeof(struct
    abc2));
18     return 0;
19 }
20
21 #程序运行结果
22 abc1:12 byte, abc2:8 byte
23
24 大家可以自己尝试分析分析为什么不一样

```

- **attribute((packed))**

在数据传输的时候，往往需要知道真实的数据长度，这时候可以通过packed属性告诉编译器，不需要做字节对齐。

```

1 #include <stdio.h>
2 struct abc {
3     char b;
4     int a;
5 } __attribute__((packed));
6
7 int main(void)
8 {
9     printf("int:%ld byte, char:%ld byte, abc:%ld byte\n",
10         sizeof(int), sizeof(char), sizeof(struct abc));
11     return 0;
12 }
13
14 #程序运行结果
15 int:4 byte, char:1 byte, abc:5 byte

```

3.3.2.2.2 union

union关键字表示新的类型叫联合体，内存表现为：**共享一份内存(以最大数据类型作为分配空间)和内存的首地址**。声明的规则如下：

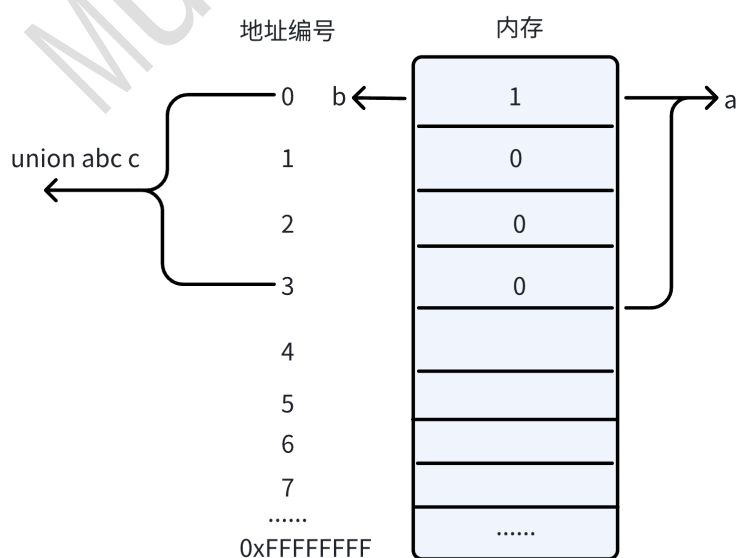
```

1 union 名字 {
2     char b;
3     int a;
4     ....
5 };

```

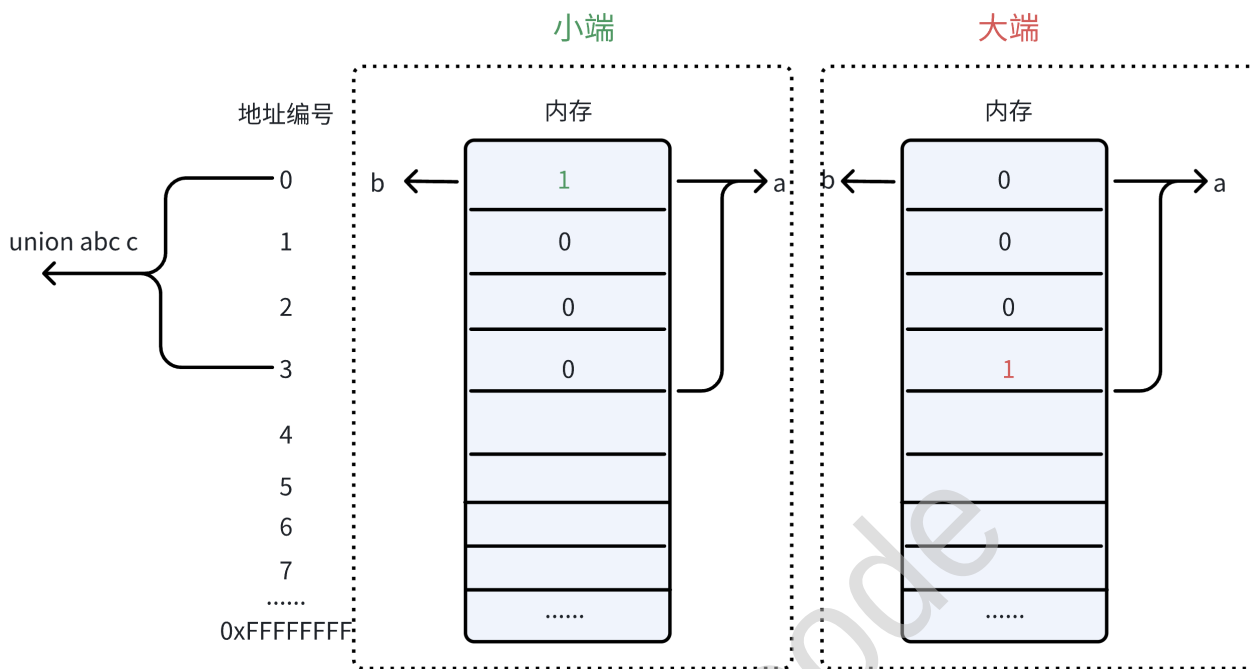
- 联合体的初始化和访问

```
1 #include <stdio.h>
2
3 union abc {
4     int a;
5     char b;
6 };
7
8 int main(void)
9 {
10     union abc c = {
11         .a = 1,
12     };
13     printf("a = %d b = %d \n",c.a, c.b);
14
15     c.b = 100;
16     printf("a = %d b = %d \n",c.a, c.b);
17     return 0;
18 }
19
20 #程序运行结果
21 a = 1 b = 1
22 a = 100 b = 100
```



- 识别计算机的大小端

大小端是计算机存储数据的一个规则，小端是低地址存放低位，高地址存放高位；大端则相反。一般像x86、arm都是小端，51单片机是大端。两种存储方式都是对的，可以通过联合体来判断大小端情况。



```
1 #include <stdio.h>
2
3 void check_endian(void)
4 {
5     union big_little {
6         int a;
7         char b;
8     } data;
9     data.a = 0x01;
10
11     if (data.b == 0x01) {
12         printf("little endian \n");
13     } else {
14         printf("big endian \n");
15     }
16 }
17 int main(void)
18 {
19     check_endian();
20     return 0;
21 }
```

- 联合体+结构体设计数据包

在uart、usb、2.4G等等常见的数据包传输，会用到联合体+结构体的设计，这样的设计拓展性很强，符合开放封闭原则：拓展是开放的，修改是封闭的。可以体会一下下面这个例子：

```
1  #include <stdio.h>
2
3  struct abc {
4      int a;
5      short b;
6      long c;
7  } __attribute__((packed));
8
9  //拓展的数据结构
10 struct efg {
11     char e;
12     float f;
13     int g;
14 } __attribute__((packed));
15
16 #pragma anon_unions
17 struct package {
18     unsigned char id;
19     union {
20         unsigned char payload[24];
21         struct abc a;
22         //拓展的数据结构
23         struct efg b;
24     };
25 };
26
27 void send_package(void* data, int len)
28 {
29     struct package *p = (struct package *)data;
30
31     uart_send(p->id);
32     for (int i = 0; i < len; i++) {
33         uart_send(p->payload[i]);
34     }
35 }
```

37 可以看到，拓展的数据结构efg，不影响数据传输的功能，即send_package不需要修改。理论上支持无限制的类型拓展。

3.3.2.2.3 enum

enum关键字表示的类型叫枚举类型，内存表现为：**内存根据定义值的大小默认选择整数常量大小(int)，如果超出int大小，编译器会选择更大的整型类型，比如long，所以内存大小不是固定的。**声明的规则如下：

```
1 enum 名字 {
2     A = 200,
3     B = 300,
4     C = 321
5 };
```

- enum的初始化和访问

```
1 #include <stdio.h>
2
3 enum abc {
4     A = 0,
5     B = 1,
6     C = 3,
7     D
8 };
9
10 enum efg {
11     E = 0x123456789,
12     F,
13     G
14 };
15
16 int main(void)
17 {
18     printf("a = %d, b = %d, c = %d, d = %d\n", A, B, C, D);
19     printf("size of a = %ld, abc = %ld\n", sizeof(A), sizeof(enum abc));
20     printf("size of E = %ld, abc = %ld\n", sizeof(E), sizeof(enum efg));
21     return 0;
22 }
23
24 #程序运行结果
25 a = 0, b = 1, c = 3, d = 4
26 size of a = 4, abc = 4 //int
27 size of E = 8, efg = 8 //long int
```

- enum在C语言中只是一个建议性关键字

```

1 #include <stdio.h>
2
3 enum abc {
4     A = 0,
5     B = 1,
6     C = 3,
7     D
8 };
9
10 enum abc func(void)
11 {
12     return 10;
13 }
14
15 int main(void)
16 {
17     enum abc a = func();
18
19     printf("a = %d \n", a);
20     return 0;
21 }
22
23 #程序运行结果
24 a = 10
25
26 Tips: 在C++ 11这会编译报错。

```

- enum和宏的差别

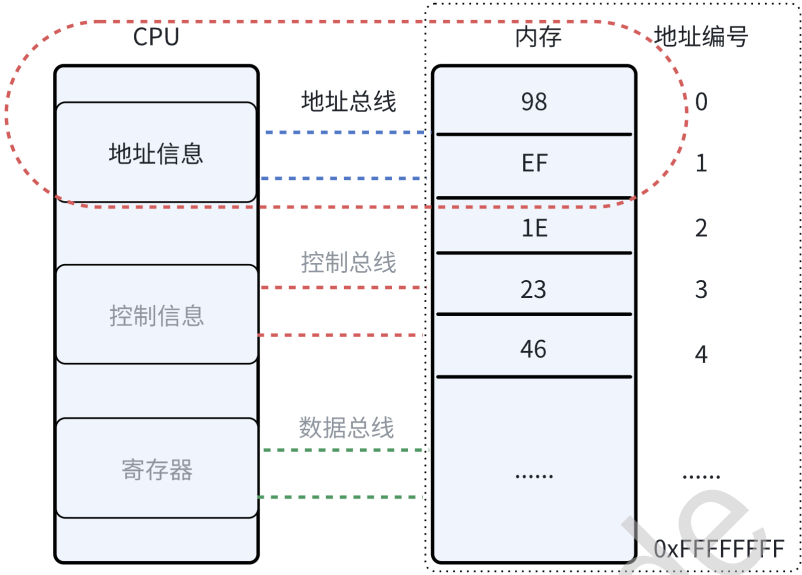
```

1 #define A 0
2 #define B 1
3 #define C 3
4 #define D 4
5
6 enum abc {
7     A = 0,
8     B = 1,
9     C = 3,
10    D
11 };
12
13 在C语言中，对于上面这段代码使用上几乎等价。

```

3.3.2.3 地址（指针）类型

C语言中，地址类型也称指针类型，圈定的内存用来存放地址编号的值。指针类型的内存表现跟编译器有关或者说跟CPU的地址总线有关。在32位的系统中，指针类型占用4byte，在64位的系统中，指针类型占用8byte。



声明的规则如下：

1 数据类型 *

• 指针变量初始化和访问

```
1 #include <stdio.h>
2
3 int main(void)
4 {
5     int a = 10;
6     int *p = &a;
7
8     printf("symbol visit: a = %d, addr visit: a = %d\n", a, *p);
9     return 0;
10 }
```

11 #程序运行结果

12 symbol visit: a = 10, addr visit: a = 10

13

14 1、首先，p也是一个变量（没啥神秘的），这本质上也是圈定一块内存，内存的大小跟编译器有关，p是这块内存的标签。到这里p和a的在变量的视角中一摸一样

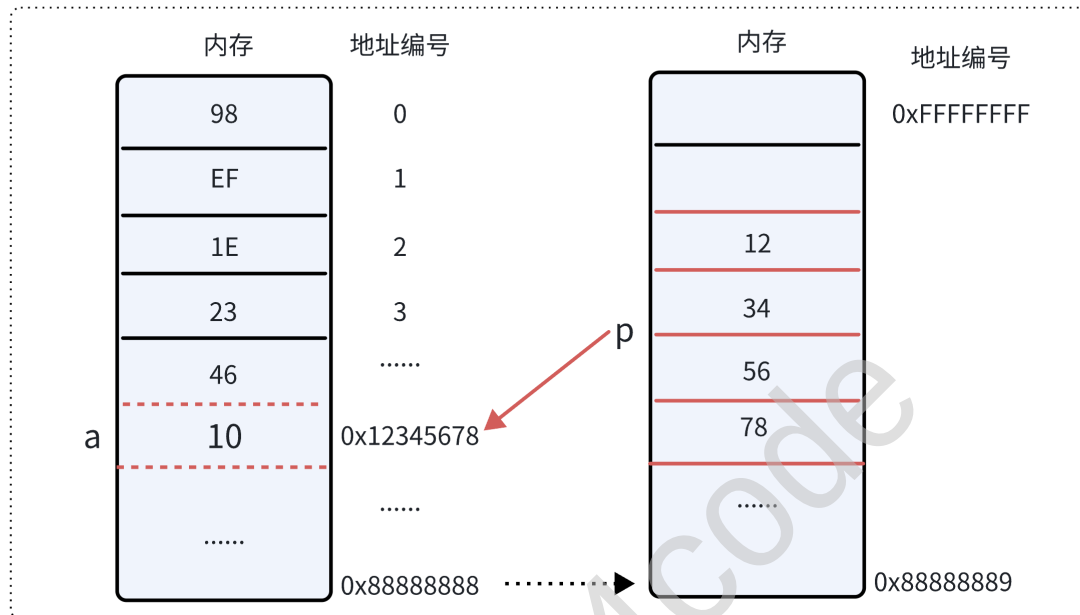
15 2、然后，p的特殊性在于，它圈定的那块内存，存放的不是一般的值而是一个地址编号。所以除了通过标签p读出这个地址编号外，C语言还提供一种直接访问这个地址编号的方法，那就是在p前面加一个*号。

16

17 举一个形象的例子：

18 你要去访问A朋友家，第一种是直接让A带你去，第二种是P知道A家的地址，通过P带你去。a 和 *p就是这种感觉

32位系统下指针访问



- 指针类型的内存大小跟数据类型没有关系

```
1 #include <stdio.h>
2
3 struct abc {
4     int a;
5     int b;
6     float c;
7 };
8
9 int main(void)
10 {
11     char *p1;
12     int *p2 ;
13     struct abc *p3;
14
15     printf("size of p1:%ld, p2:%ld, p3:%ld\n", sizeof(p1), sizeof(p2),
16         sizeof(p3));
17     return 0;
18 }
19 #程序运行结果
20 size of p1:8, p2:8, p3:8
```

后续会有专门的一part来介绍指针的使用，大家先了解指针类型的基本思想。

3.3.2.4 typedef

typedef关键字是给数据类型起一个别名，让程序的可读性更高，特别是复杂的类型（如函数指针类型），做到见字知意。使用的规则如下：

- 1 **typedef** 数据类型 别名

```
1 #include <stdio.h>
2
3 typedef struct abc {
4     int a;
5     int b;
6     float c;
7 } abc_t;
8
9 typedef int len_t;
10 typedef unsigned char uint8_t;
11 typedef void (func*)(void);
12 func p;
13
14 int main(void)
15 {
16     len_t a = 0;
17     uint8_t b;
18     abc_t c;
19
20     return 0;
21 }
```

- 体会typedef 和 #define 的区别

```
1 #include <stdio.h>
2
3 #define uint8 unsigned char
4 typedef unsigned char uint8_t;
5
6 int main(void)
7 {
```

```

8     uint8 a = 'c';
9     uint8_t b = 'c';
10
11     printf("a = %c, b = %c \n", a, b);
12     return 0;
13 }
14
15 #程序运行结果
16 a = c, b = c

```

```

1  #include <stdio.h>
2
3  #define uchar_p unsigned char *
4  typedef unsigned char * uint8_p;
5
6  int main(void)
7  {
8      uchar_p a, b;
9      uint8_p c, d;
10
11     a = "hello";
12     b = "hello"; //这里b是unsigned char, 不是 unsigned char *
13     c = "hello";
14     d = "hello";
15
16     return 0;
17 }
18
19 #程序运行结果
20 1.c: In function 'main':
21 1.c:12:7: warning: assignment to 'unsigned char' from 'char *' makes integer
    from pointer without a cast [-Wint-conversion]
22     12 |         b = "hello"; //
23         |
24
25 #宏展开后的结果:
26 typedef unsigned char * uint8_p;
27
28 int main(void)
29 {
30     unsigned char * a, b; //b 是 unsigned char
31     uint8_p c, d;
32
33
34     a = "hello";

```

```

35     b = "hello";
36     c = "hello";
37     d = "hello";
38
39     return 0;
40 }

```

3.3.3 修饰关键字

我们知道数据类型关键字的作用是告诉编译器圈定内存的大小。但实际工程应用中，只有大小的限制是不够的。想一想，我们的内存空间这么大（32位系统为例，有4G的内存寻址空间），该从哪个地方圈定内存呢？是全部区域都可以吗？分配给你的内存空间，要不要有些操作做限定呢？这些都是需要**修饰关键字**来告诉编译器该怎么来做。

3.3.3.1 auto

auto关键字是最无感的存在。编译器在默认缺省的情况下，所有变量都是auto的。你就当做它不存在，不曾来过。

3.3.3.2 register

register关键字是一个建议性关键字。设计的初衷是想要定义的变量放到cpu的寄存器中，但是往往编译器看到这个关键字的时候，发出一句呐喊：臣妾做不到啊。寄存器相对于内存，cpu在访问速度上更快，但也很稀缺，不是说想放就放的。所以编译器只能尽量的去做，它不保证一定能做得到，大家也就不要奢望了。但值得注意的是加上register修饰后的变量，无法通过&来取地址。即使大概率放进寄存器会失败，但编译器就不给你寻址，万一成功了呢？编译器还是有梦想的。

```

1  #include <stdio.h>
2
3  int main(void)
4  {
5      register char a = 1;
6
7      printf("a = %d , addr of a:%p", a, &a);
8      return 0;
9  }
10
11 #程序运行结果
12 1.c: In function 'main':
13 1.c:6:5: error: address of global register variable 'a' requested
14     6 |     printf("a = %d , addr of a:%p", a, &a);
15       |     ^~~~~~
16

```

3.3.3.3 static

在C语言中static关键字有两个作用：1、变量的内存从静态全局数据区分配；2、限定变量或函数的作用域

3.3.3.3.1 修饰变量

- 生命周期：不管是修饰局部变量还是全局变量，最后都在静态全局数据区分配。这意味着变量在整个程序的生命周期内都是有效合法的。

```
1 #include <stdio.h>
2
3 static int a = 20;
4 int func(void)
5 {
6     static int b = 10;
7     return ++b;
8 }
9
10 int main(void)
11 {
12     printf("a = %d , b = %d ,b = %d \n", a, func(), func());
13     return 0;
14 }
15 #程序运行结果
16 a = 20 , b = 12 ,b = 11
```

- 限定范围：限定全局变量只在当前定义的文件可见，其他文件即使加上extern修饰，也无法引用。会编译报错

```
1 //1.c
2 #include <stdio.h>
3
4 static int a = 20;
5
6 int main(void)
7 {
8     printf("a = %d\n", a);
9     return 0;
10 }
11
12 //2.c
13 extern int a;
14 int func(void)
```

```

15 {
16     return ++a;
17 }
18
19 #程序运行结果
20 /usr/bin/ld: 2.o: in function `func':
21 2.c:(.text+0xa): undefined reference to `a'
22 /usr/bin/ld: 2.c:(.text+0x13): undefined reference to `a'
23 /usr/bin/ld: 2.c:(.text+0x19): undefined reference to `a'
24 collect2: error: ld returned 1 exit status
25

```

3.3.3.3.2 修饰函数

- 限定范围：限定函数在当前定义的文件可见，其他文件不可引用，但可以实现相同名字的函数(也需要用static修饰)，互不影响。感受一下下面这个例子：

```

1 //1.c
2 #include <stdio.h>
3
4 static int a = 20;
5 static int func(void)
6 {
7     return ++a;
8 }
9
10 int main(void)
11 {
12     printf("a = %d\n", func());
13     return 0;
14 }
15
16 //2.c
17 static int a = 10;
18 static int func(void)
19 {
20     return ++a;
21 }
22
23 #程序运行结果
24 a = 21

```

3.3.3.4 extern

extern用来声明一个没有被static修饰的变量或者函数。

```

1 //1.c
2 #include <stdio.h>
3
4 extern int func(void);
5 int main(void)
6 {
7     func();
8     return 0;
9 }
10
11 //2.c
12 #include <stdio.h>
13 int func(void)
14 {
15     printf("func call \n");
16 }
17
18 #程序运行结果
19 func call

```

- 在架构设计中尽量不要用extern，会导致代码耦合度非常高而且不太可控。建议参考如下方案：

```

1 //1.c
2 #include <stdio.h>
3 #include "2.h"
4
5 int main(void)
6 {
7     set_a(20);
8     printf("a = %d\n", read_a());
9     return 0;
10 }
11
12 //2.h
13 void set_a(int value);
14 int read_a(void);
15
16 //2.c
17 static int a = 10;
18 void set_a(int value)
19 {
20     mutex_lock();
21     a = value;
22     mutex_unlock();

```

```

23 }
24 int read_a(void)
25 {
26     return a;
27 }

```

- extern "C", 用C++的编译器 (g++) 按照C的规则进行编译。C++和C语言在编译规则上有很多不同, 所以在C++中想完全复用C写的代码, 就可以用extern "C"来修饰。这种在C++使用C的lib库的时候比较常见。另外, 面试也很喜欢问这个。

```

1 #include <stdio.h>
2
3 void func(void)
4 {
5     printf("func call \n");
6 }
7 //FPIC 生成的so库跟绝对代码位置无关, 可以在任意位置被引用
8 //-shared 共享库
9 gcc -o libfunc.so 2.c -fPIC -shared

```

```

1 #include <stdio.h>
2
3 #if defined(__cplusplus)
4 extern "C" {
5 #endif
6 void func(void);
7 #if defined(__cplusplus)
8 }
9 #endif
10
11
12 int main(void)
13 {
14     func();
15     return 0;
16 }
17
18 export LD_LIBRARY_PATH=$LD_LIBRARY_PATH:./
19 g++ -o bin 1.c -lfunc -L.
20 ./bin
21 #程序运行结果
22 func call

```

3.3.3.5 const

const是constant的缩写，是恒定不变的意思，被修饰的变量经常被人误解成常数或者常量。这个理解不太准确，准确的理解应该是read only。本质上还是变量，编译器只能尽量不让你去修改，但是实际上总有一些方法可以做到修改const 修饰变量的值，往往都是一些异常的操作导致（比如数组越界、指针越界访问、栈溢出等等）。我们要建立一个正确的编程理念：const 修饰的变量技术上能改，但是不要去改。

```
1  const 数据类型 变量名 or 数据类型 const 变量名
2
3  #include <stdio.h>
4
5  int main(void)
6  {
7      const int a = 10;
8      int const b = 20;
9
10     //a = 20; //编译错误, 编译不给修改
11     printf("a = %d, b = %d\n", a, b);
12     return 0;
13 }
14
```

- 修改const修饰的变量

```
1  #include <stdio.h>
2
3  int main(void)
4  {
5      int a = 1234568;
6      const int b = 11111111;
7      int *p = &a;
8
9      p[1] = 22222222;
10     printf("b = %d \n", b);
11     return 0;
12 }
```

- const 修饰指针

很多同学对const修饰指针变量，有时候分不清楚是修饰**指针变量**还是限定**指针指向的内容**。今天教大家一个方法，叫**"近水楼台先得月"**：

```

1  #include <stdio.h>
2
3  const int * a;  //-> const * a ->靠近 *, 所以 地址指向的内容 (*a) 不能变
4  int const * b;  //-> const * b ->靠近 *, 所以 地址指向的内容 (*d) 不能变
5  int * const c;  //-> * const c ->靠近 c, 所以 c指针变量不能变
6  const int * const d;  //-> const * const d ->靠近 * 又靠近c, 所以d和*d都不能变
7
8  int main(void)
9  {
10     //下面5个语句编译器会报错拦截
11     *a = 1;
12     *b = 1;
13     c = 1;
14     *d = 1;
15     d = 1;
16
17     return 0;
18 }
19 //编译器报错
20 1.c: In function 'main':
21 1.c:11:8: error: assignment of read-only location '*a'
22    11 |     *a = 1;
23        |         ^
24 1.c:12:8: error: assignment of read-only location '*b'
25    12 |     *b = 1;
26        |         ^
27 1.c:13:7: error: assignment of read-only variable 'c'
28    13 |     c = 1;
29        |         ^
30 1.c:14:8: error: assignment of read-only location '*d'
31    14 |     *d = 1;
32        |         ^
33 1.c:15:7: error: assignment of read-only variable 'd'
34    15 |     d = 1;
35        |         ^
36    ^

```

- 使用const可以提高运行效率：如果用const修饰，编译器会直接把立即数200赋值给变量a，而没有const修饰的则每次都需要读取内存中ABC值。这样用const修饰时，执行效率就比较高（我们知道读取内存是相对慢的操作）。

```

1  #include <stdio.h>
2
3  const int ABC = 200;

```

```

4
5 int main(void)
6 {
7     int a = ABC;
8     return 0;
9 }
10
11
12 //int a = ABC;
13 movl    ABC(%rip), %eax //把内存中ABC的值读到寄存器中去
14 movl    %eax, -4(%rbp)
15
16 //const int a = ABC;
17 movl    $200, -4(%rbp)
18

```

- const和宏的区别

const设计的初衷就是为了代替宏，消除它的缺点，继承它的优点。

1. 编译器处理方式：define宏是在预处理阶段展开；const变量是编译运行阶段使用
2. 安全检查：define宏不做任何类型检查，仅仅是展开；const变量编译阶段会执行类型检查
3. 内存位置：define宏在代码段；const常量可以在静态全局数据段、栈中

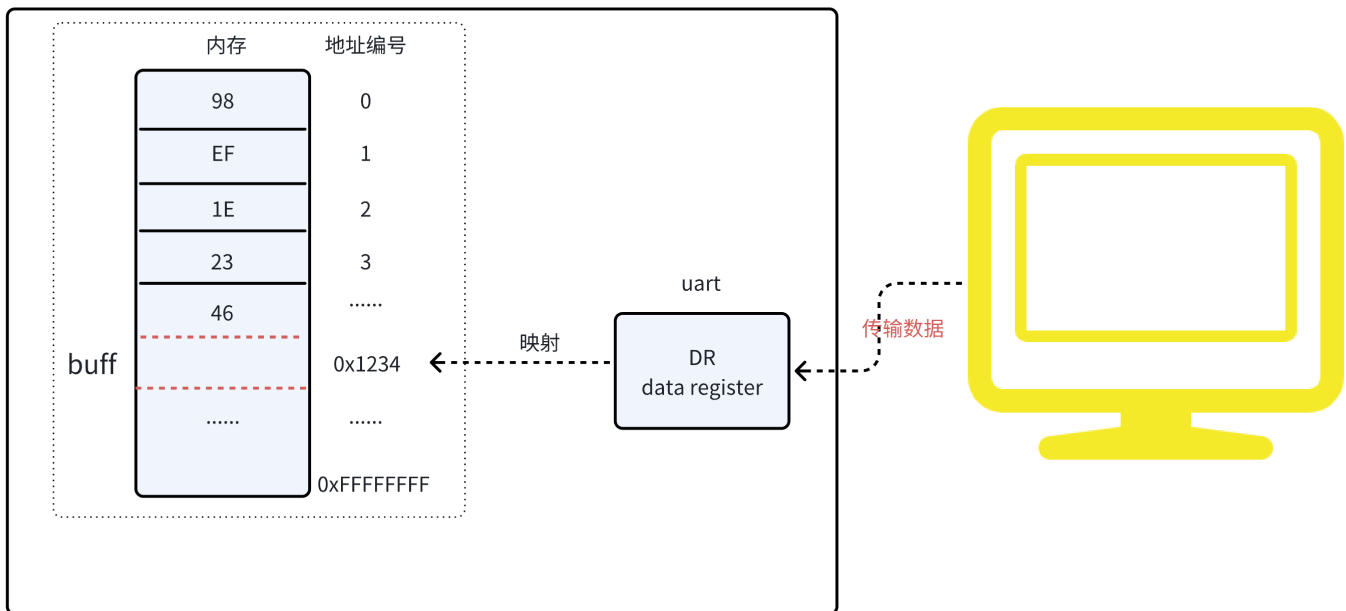
3.3.3.6 volatile

volatile关键字用于告诉编译器，被声明为 volatile 的变量的值可能在程序的控制之外发生变化，因此编译器不应该对其进行某些优化。主要用于处理硬件寄存器、中断服务程序和多线程等情况：下面如果buff变量是uart的DR寄存器映射，即使代码中没有地方更新buff的值，也应该告诉编译器不要优化，如果有地方读取buff的值的时候，都需要从内存中读取，因为外部uart会触发更新。

```

1 #include <stdio.h>
2
3 volatile unsigned char buff;
4
5 int main(void)
6 {
7     while (buff) { //等待buff不为0
8         //do something
9     }
10    return 0;
11 }

```



- 体会一下下面这个例子：

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <unistd.h>
4 #include <pthread.h>
5
6 static volatile int wait = 1;
7
8 void* function(void* arg)
9 {
10     while (1) {
11         sleep(1);
12         wait = 0;
13         printf("flag wait = %d in thread \n", wait);
14     }
15     return NULL;
16 }
17 int main(void)
18 {
19     pthread_t tid;
20     pthread_create(&tid, NULL, function, NULL);
21
22     while(wait);
23     printf("flag wait = %d in main \n", wait);
24     return 0;
25 }
26

```

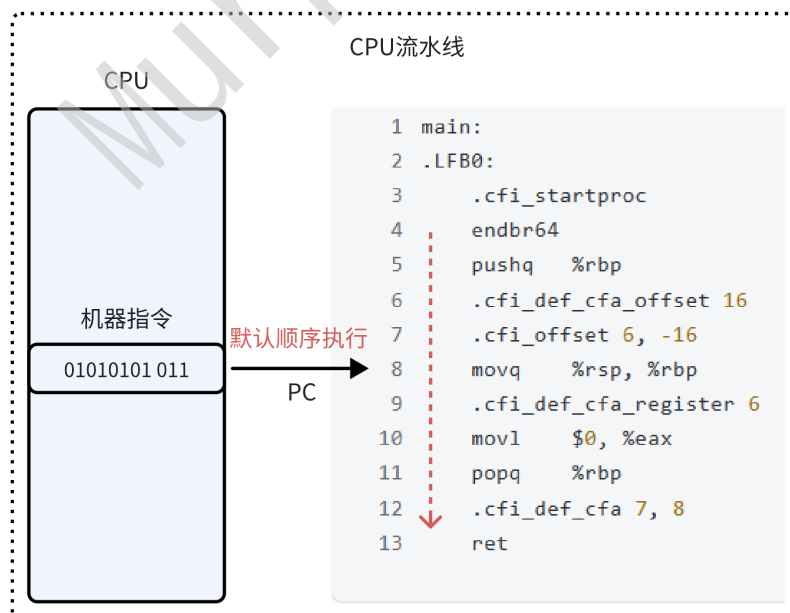
```

27
28 gcc -o bin 1.c -lpthread -O3
29 ./bin
30 #程序运行结果
31 当wait不加volatile时, 程序出现死循环
32
33 #体会一下两种情况的下汇编
34 //不加volatile时, while(wait)
35     movl    wait(%rip), %eax
36     testl   %eax, %eax
37     je      .L6
38 .L7:
39     jmp     .L7
40
41 //加volatile时, while(wait)
42 .L6:
43     movl    wait(%rip), %eax
44     testl   %eax, %eax
45     jne     .L6

```

3.3.4 逻辑关键字

在cpu的眼里很单纯, 程序指针 (PC) 指到哪里就执行哪里, 默认情况下顺序往下执行, 而**逻辑关键字**作用就是改变PC指针的指向, 这个最基本的思想就构成了程序里各种逻辑设计 (条件、选择、跳转、循环)。



3.3.4.1 条件: if、else

条件逻辑关键字, 判断条件表达式是否为真, 为真后停止判断并进入处理对应逻辑。

```

1  if (条件表达式)
2      xx; //表达式为真, 进来此分支
3  else if(条件表达式)
4      yy;
5      ....
6  else
7      zz;

```

```

1  #include <stdio.h>
2
3  int main(void)
4  {
5      int a = 2;
6
7      if (a == 1) {
8          printf("hello 1 \n");
9      } else if (2 == a) {
10         printf("hello 2 \n");
11     } else {
12         printf("hello a = %d \n", a);
13     }
14     return 0;
15 }
16
17 #程序运行结果
18 hello 2

```

3.3.4.2 选择：switch、case、default

选择逻辑关键字基于某个表达式的值选择不同的执行路径。它的基本语法如下：

```

1  switch (表达式  整形数字) {
2      case 值1:
3          // 当表达式等于值1时执行的代码
4          break;
5      case 值2:
6          // 当表达式等于值2时执行的代码
7          break;
8      //....
9      default:
10         // 如果表达式的值与所有case不匹配时执行的代码
11 }

```

```

1 #include <stdio.h>
2
3 int main(void)
4 {
5     int a = 2;
6
7     switch (a) {
8     case 1:
9         printf("hello 1 \n");
10        break;
11    case 2:
12        printf("hello 2 \n");
13        break;
14    default:
15        printf("hello a = %d \n", a);
16    }
17    return 0;
18 }
19
20 #程序运行结果
21 hello 2
22
23 Tips:
24 1、每个case块末尾都需要有break语句，否则将继续执行后面的case块，直到遇到break或者switch
    结束。
25 2、switch条件不能是浮点数：
26 #程序运行结果
27 1.c: In function 'main':
28 1.c:7:13: error: switch quantity not an integer
29     7 |     switch (1.1) {
30       |

```

3.3.4.3 循环：do、while、for

- do-while循环是C语言中的一种循环结构，它的基本语法如下：

```

1 while (条件) {
2     // 循环体
3 };
4
5 do {
6     // 循环体
7     // 这里的代码至少会执行一次
8 } while (条件);

```

9

10 **do-while**与**while**循环不同的是，**do-while**循环会先执行一次循环体，然后检查条件，如果条件为真，循环将继续执行，这确保循环体至少被执行一次。

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int number;
6     do {
7         printf("please enter a positive number: ");
8         scanf("%d", &number);
9
10        if (number <= 0) {
11            printf("ensure positive number entered \n");
12        }
13    } while (number <= 0);
14    printf("positive number: %d\n", number);
15    return 0;
16 }
17
18 #程序运行结果
19 please enter a positive number: 1
20 positive number: 1
```

- **for** 也是另一种循环结构，它的基本语法如下：

```
1 for (初始化; 条件表达式; 条件更新) {
2     //循环体 ->条件表达式为真时进来
3 }
4
5 Tips: 养成好的习惯，条件的更新不要在循环体里进行更新，容易出现死循环
```

```
1 #include <stdio.h>
2
3 int main()
4 {
5     for (int i = 0; i < 5; i++) {
6         printf("i = %d \n", i);
7     }
8     return 0;
```

```

9 }
10
11 #程序运行结果
12 i = 0
13 i = 1
14 i = 2
15 i = 3
16 i = 4

```

- do-while 和 for 怎么选

一般是：某个条件不满足时退出循环，用do-while或者while，而靠计数结束退出循环，用for。

```

1 #include <stdio.h>
2
3 void main()
4 {
5     while(1) {
6
7     }
8
9     for(;;) {
10
11     }
12 }
13
14 单片机中有两种死循环的写法？你一般用哪种？
15 （不用纠结，用哪种都可以，基本没有什么区别）

```

3.3.4.4 跳转：continue、break、return、goto

- continue、break一般结合循环来用：continue 跳过以下的循环体，进行下一轮循环，break 提前结束循环。

```

1 #include <stdio.h>
2 int main()
3 {
4     for (int i = 0; i < 5; i++) {
5         if (i == 1) {
6             continue;
7         } else if (i == 4) {
8             break;
9         }
10        printf("i = %d \n", i);

```

```

11     }
12     return 0;
13 }
14 #程序运行结果
15 i = 0
16 //i = 1 不打印是因为continue跳过了
17 i = 2
18 i = 3
19 //i = 4 不打印是因为break直接结束循环了

```

- return 结束本次的函数执行，至多可带一个数值返回。

```

1  #include <stdio.h>
2
3  int func(void)
4  {
5      for (int i = 0; i < 5; i++) {
6          if (i == 4) {
7              return i;
8          }
9          printf("i = %d \n", i);
10     }
11     printf("func end\n");
12 }
13
14 int main()
15 {
16     printf("value:%d of func return \n", func());
17
18     return 0;
19 }
20 #程序运行结果
21 i = 0
22 i = 1
23 i = 2
24 i = 3
25 //i = 4 和 func end 不打印是因为return直接结束了func
26
27 value:4 of func return

```

- goto：直接跳转到对于label位置执行。尽管在许多编程风格中，使用goto被视为不推荐的实践，因为它可能导致代码不易理解和维护，但有时候它可以用于一些特殊的控制流需求（linux的源码中有大量的使用）。建议是在同一个函数内用，不要在不同函数直接跳，程序会把你逼疯的。它的基本语法如下：

```
1 goto label;
2
3 // 在标签位置定义代码块
4 label:
5     // 代码块
```

```
1 //体会linux驱动中常用的goto案例
2 #include <stdio.h>
3 #include <stdlib.h>
4
5 int main()
6 {
7     int *p1;
8     int *p2;
9     int *p3;
10
11     p1 = (int*)malloc(sizeof(int));
12     if (!p1) {
13         return -1;
14     }
15     p2 = (int*)malloc(sizeof(int));
16     if (!p2) {
17         goto fail_1;
18     }
19     p3 = (int*)malloc(sizeof(int));
20     if (!p3) {
21         goto fail_2;
22     }
23
24     free(p3);
25 fail_2:
26     free(p2);
27 fail_1:
28     free(p1);
29     return 0;
30 }
31
```

3.4 运算符

程序的本质是逻辑和数据。狭义上运算是指数学运算，广义的运算是数据处理。在C语言中运算符分为：算数运算、逻辑运算、位运算、赋值更新、内存访问。这里提一个理念：对于运算符的优先级问题，不建议死记硬背，用()万能钥匙来人为定义优先级。不要吝啬用()，()会让你的程序变得清晰可读。

3.4.1 算数运算

3.4.1.1 加减乘除：+、-、*、/

加减乘除大家从小学就开始学了，所以非常容易理解。这里提一个观念就是：对于CPU来讲，加减法的运行速率比乘除要快。这是因为CPU进行运算主要依赖于算术逻辑单元（ALU）和浮点运算单元（FPU），而ALU的本质是累加器，所以加减法在硬件处理就很快，乘除法有时候编译器会转换成加法或者移位操作。随着计算机的发展，现在已经有了"乘法器"的硬件支持，速度上也已经很快。我们感受一下编译器对乘除法的优化：

- 直接调用"乘法器"进行运算

```
1  #include <stdio.h>
2
3  int main()
4  {
5      int a = 10;
6      int b = a * 34;
7
8      printf("b = %d \n", b);
9      return 0;
10 }
11 //      int a = 10;
12 //      int b = a * 33;
13 movl    $10, -8(%rbp)
14 movl    -8(%rbp), %eax
15 imull   $34, %eax, %eax
16 movl    %eax, -4(%rbp)
```

- 乘法转换成向左移位和加法运算

```
1  int main()
2  {
3      int a = 10;
```

```

4     int b = a * 33;
5
6     return 0;
7 }
8
9 //     int a = 10;
10 //    int b = a * 33;
11 movl    $10, -8(%rbp)
12 movl    -8(%rbp), %edx
13 movl    %edx, %eax
14 sall    $5, %eax
15 addl    %edx, %eax
16 movl    %eax, -4(%rbp)
17 movl    $0, %eax

```

- 除法转换成向右移位运算

```

1 int main()
2 {
3     int a = 100;
4     int b = a / 32;
5
6     return 0;
7 }
8
9 //     int a = 100;
10 //    int b = a / 32;
11 movl    $100, -8(%rbp)
12 movl    -8(%rbp), %eax
13 sarl    $5, %eax
14 movl    %eax, -4(%rbp)

```

3.4.1.2 mod操作：%

%运算我们也称为mod运算，表示取余数。

```

1 #include <stdio.h>
2
3 int main()
4 {
5     int a = 10;
6
7     printf("a mod 3 = %d \n", a % 3);
8     return 0;

```

```
9 }
10
11 #程序运行结果
12 a mod 3 = 1
```

- 循环队列的索引更新

```
1 #define LEN 10
2 int buff[LEN];
3 int index = 0;
4
5 void data_process(int data)
6 {
7     index = index % LEN;
8     buff[index] = data;
9     index++;
10 }
```

- 生成[L, R]区间内的随机数

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <malloc.h>
4 #include <time.h>
5
6 int* generate_array(int n, int L, int R)
7 {
8     int* array = (int*)malloc(sizeof(int) * n);
9     srand(time(NULL));
10    for (int i = 0; i < n; i++) {
11        array[i] = rand() % (R - L + 1) + L;
12    }
13    return array;
14 }
15
16 void print_array(int* data, int n)
17 {
18    for (int i = 0; i < n; i++) {
19        printf("%d ", data[i]);
20    }
21    printf("\n");
22 }
23
```

```

24 int main()
25 {
26     int *arr = generate_array(10, 0, 10);
27     print_array(arr, 10);
28     return 0;
29 }
30 //运行结果
31 0 4 7 0 8 6 1 10 3 5

```

- %还可以做占位符号，在格式转换时用

```

1 #include <stdio.h>
2
3 int main()
4 {
5     int a = 10;
6
7     printf("a = %d %x %o \n", a, a, a)
8     return 0;
9 }

```

3.4.2 位运算

3.4.2.1 移位：<<、>>

- << 左移：在没有溢出之前，每移动一位等价于乘以2，且移位后右边补0；

```

1 #include <stdio.h>
2
3 #define CALCULATE_LEN(a) (sizeof(a)*8)
4 // %d %x %o
5 void print_binary(long long a, int n)
6 {
7     printf(": ");
8     while (n--) {
9         printf("%lld ", (a >> n) & 0x01);
10    }
11    printf(": %lld \n", a);
12 }
13
14 void main(void)
15 {
16     char a = 1;

```

```

17
18     for (int i = 0; i < 8; i++) {
19         printf("%d left shift %d ",a, i);
20         print_binary(a << i, CALCULATE_LEN(a));
21     }
22 }
23
24 #程序运行结果
25 a = 1, left shift 0 : 0 0 0 0 0 0 0 1 : 1 // 1 << 0
26 a = 1, left shift 1 : 0 0 0 0 0 0 1 0 : 2 // 1 << 1
27 a = 1, left shift 2 : 0 0 0 0 0 1 0 0 : 4 // 1 << 2
28 a = 1, left shift 3 : 0 0 0 0 1 0 0 0 : 8 // 1 << 3
29 a = 1, left shift 4 : 0 0 0 1 0 0 0 0 : 16
30 a = 1, left shift 5 : 0 0 1 0 0 0 0 0 : 32
31 a = 1, left shift 6 : 0 1 0 0 0 0 0 0 : 64
32 a = 1, left shift 7 : 1 0 0 0 0 0 0 0 : 128
33 a = 1, left shift 8 : 0 0 0 0 0 0 0 0 : 0 //因为a为char类型，左移8位后溢出了

```

- >> 右移：每移动一位等价于除以2。signed类型和unsigned在右移的时候有差别：如果是负数，移位后补1，正数补0，**所以负数通过右移永远都不会等于0**：

```

1 #include <stdio.h>
2
3 #define CALCULATE_LEN(a) (sizeof(a)*8)
4
5 void print_binary(char a, int n)
6 {
7     printf(": ");
8     while (n--) {
9         printf("%d ", (a >> n) & 0x01);
10    }
11    printf(": %d \n", a);
12 }
13
14 void main(void)
15 {
16     char a = 0x80;
17
18     for (int i = 0; i < 9; i++) {
19         printf("a = 0x%x, right shift %d times",a & 0xFF, i);
20         print_binary((a >> i), CALCULATE_LEN(a));
21     }
22 }
23
24 #程序运行结果: a 为 char类型 a = -1

```

```

25 a = 0x80, right shift 0 times: 1 0 0 0 0 0 0 0 : -128
26 a = 0x80, right shift 1 times: 1 1 0 0 0 0 0 0 : -64
27 a = 0x80, right shift 2 times: 1 1 1 0 0 0 0 0 : -32
28 a = 0x80, right shift 3 times: 1 1 1 1 0 0 0 0 : -16
29 a = 0x80, right shift 4 times: 1 1 1 1 1 0 0 0 : -8
30 a = 0x80, right shift 5 times: 1 1 1 1 1 1 0 0 : -4
31 a = 0x80, right shift 6 times: 1 1 1 1 1 1 1 0 : -2
32 a = 0x80, right shift 7 times: 1 1 1 1 1 1 1 1 : -1
33 a = 0x80, right shift 8 times: 1 1 1 1 1 1 1 1 : -1
34
35 #程序运行结果: a 为 unsigned char类型 a = 128
36 a = 0x80, right shift 0 times: 1 0 0 0 0 0 0 0 : 128
37 a = 0x80, right shift 1 times: 0 1 0 0 0 0 0 0 : 64
38 a = 0x80, right shift 2 times: 0 0 1 0 0 0 0 0 : 32
39 a = 0x80, right shift 3 times: 0 0 0 1 0 0 0 0 : 16
40 a = 0x80, right shift 4 times: 0 0 0 0 1 0 0 0 : 8
41 a = 0x80, right shift 5 times: 0 0 0 0 0 1 0 0 : 4
42 a = 0x80, right shift 6 times: 0 0 0 0 0 0 1 0 : 2
43 a = 0x80, right shift 7 times: 0 0 0 0 0 0 0 1 : 1
44 a = 0x80, right shift 8 times: 0 0 0 0 0 0 0 0 : 0

```

3.4.2.2 位与或、取反：&、|、~

我们知道操作硬件的最小能力是bit位，所以按位与"&"、按位或"|"、按位取反"~" 在嵌入式的寄存器操作中运用非常广泛。下面是基本规则：

```

1 | : 1 | 1 = 1; 1 | 0 = 1; 0 | 1 = 1; 0 | 0 = 0 -> 俗称 置位器 (set)
2 & : 1 & 1 = 1; 1 & 0 = 0; 0 & 1 = 0; 0 & 0 = 0 -> 俗称 清零器 (clr)
3 ~ : ~(0x80) = 0x7F -> 按位取反 0b10000000 ~ 0b01111111

```

• 置位: |

```

1 #include <stdio.h>
2
3 #define CALCULATE_LEN(a) (sizeof(a)*8)
4 #define SET(reg, n) ((reg) |= 0x1<<(n))
5
6 void print_binary(unsigned char a, int n)
7 {
8     printf(": ");
9     while (n-- > 0) {
10         printf("%d ", (a >> n) & 0x01);
11     }

```

```

12     printf(": %d \n", a);
13 }
14
15 void main(void)
16 {
17     unsigned char a = 0x01;
18
19     printf("a");
20     print_binary(a, CALCULATE_LEN(a));
21
22     //a = a | 0x80;
23     //a = a | 0x1 << 7;
24     SET(a, 7);
25     printf("a");
26     print_binary(a, CALCULATE_LEN(a));
27 }
28
29 #程序运行结果:
30 a: 0 0 0 0 0 0 0 1 : 1
31 a: 1 0 0 0 0 0 0 1 : 129

```

- 清零: & 和 ~ 一起用

```

1 #include <stdio.h>
2
3 #define CALCULATE_LEN(a) (sizeof(a)*8)
4 #define CLR(reg, n) ((reg) &= ~(0x1<<(n)))
5
6 void print_binary(unsigned char a, int n)
7 {
8     printf(": ");
9     while (n-- > 0) {
10         printf("%d ", (a >> n) & 0x01);
11     }
12     printf(": %d \n", a);
13 }
14
15 void main(void)
16 {
17     unsigned char a = 0x81;
18
19     printf("a");
20     print_binary(a, CALCULATE_LEN(a));
21
22     //a = a & 0x7F;

```

```

23     //a = a & ~(0x1<<7);
24     CLR(a, 7);
25     printf("a");
26     print_binary(a, CALCULATE_LEN(a));
27 }
28
29 #程序运行结果:
30 a: 1 0 0 0 0 0 0 1 : 129
31 a: 0 0 0 0 0 0 0 1 : 1

```

3.4.2.3 位异或：^

按位异或"^"在嵌入中运用相对较少，一般用于算法中，如加密算法。下面是基本规则：

1 ^: 1 ^ 1 = 0; 1 ^ 0 = 1; 0 ^ 1 = 1; 0 ^ 0 = 0 -> 当相同的时候为0，不相同的时候为1

```

1 //不需要用第三个变量就可以实现两个变量的值交换
2
3 void swap(int* a, int* b)
4 {
5     if ((*a) == (*b)) {
6         return;
7     }
8
9     *a = (*a) ^ (*b);
10    *b = (*a) ^ (*b);
11    *a = (*a) ^ (*b);
12 }
13

```

14 Tips: 这个方法的优势在于不需要额外的空间，并且没有溢出的风险。需要注意的是，该方法在处理相同的数值时会导致结果为零。在实际应用中，要考虑这一点

3.4.3 逻辑运算

逻辑运算的结果只有真假，即1和0。

3.4.3.1 条件或与：||、&&

```

1 A && B （与运算，都为真才为真）
2 Tips: 只要A为假，则B不会被执行
3

```

4 A || B (或运算, 一个为真就为真)

5 Tips: 只要A为真, 则B不会被执行

```
1 #include <stdio.h>
2
3 int func1(void)
4 {
5     printf("func1 call \n");
6     return 0;
7 }
8 int func2(void)
9 {
10    printf("func2 call \n");
11    return 10;
12 }
13 void main(void)
14 {
15    printf("-----1-----\n");
16    if (func1() && func2()) {
17        printf("do something \n");
18    }
19
20    printf("-----2-----\n");
21    if (func2() && func1()) {
22        printf("do something \n");
23    }
24
25    printf("-----3-----\n");
26    if (func1() || func2()) {
27        printf("do something \n");
28    }
29
30    printf("-----4-----\n");
31    if (func2() || func1()) {
32        printf("do something \n");
33    }
34 }
35
36 #程序运行结果:
37 -----1-----
38 func1 call
39 -----2-----
40 func2 call
41 func1 call
42 -----3-----
```

```

43 func1 call
44 func2 call
45 do something
46 -----4-----
47 func2 call
48 do something

```

3.4.3.2 大小判断：==、>、<、>=、<=、?:

```

1 Tips1: == 因为 = 在使用上容易混淆。建议使用==时，把数字放在左边，变量放在右边，这样如果写成=编译器就会报错拦截出来。
2 if (10 == a) {
3     ...
4 }
5
6 Tips2: A ? B : C 等价于
7 if (A) {
8     B;
9 } else {
10    C;
11 }

```

3.4.3.3 条件取反：!

```

1 Tips1: 常用于指针空判断
2 int *p = (int*)malloc(sizeof(int));
3 if (!p) {
4     //do something
5 }
6
7 Tips2: 用于二值化处理
8 int a = xx;
9 unsigned char b = 0;
10
11 b = !!a; -->不管a的值是多少，b只有0和1

```

3.4.4 赋值更新：=、+=、-=、&=、|=，++、--

赋值更新的本质是：通过标签（变量名）改变内存里面的值。

```

1 int a;

```

```

2
3 a = 10;
4 a += 1; //-> a = a + 1
5 a -= 1; //-> a = a - 1
6 a &= 1; //-> a = a & 1
7 a |= 1; //-> a = a | 1
8 a++;    //-> a = a + 1
9 a--;    //-> a = a - 1
10 *(int*)0x12345678 = 1;
11
12 tips1:主要 a++ 和 ++a 的区别:
13
14 a = 0;
15 if (a++) { //先用a, 再++
16     //不执行
17 }
18
19 a = 0;
20 if (++a) { //先++, 再用a
21     //执行
22 }

```

3.4.5 内存操作

3.4.5.1 函数访问: ()

```

1 #include <stdio.h>
2
3 int func1(void)
4 {
5     int a = 0;
6
7     printf("func1 call \n");
8     return 0;
9 }
10
11 void main(void)
12 {
13     func1();
14 }
15
16 func1:
17 .LFB0:
18     endbr64
19     pushq   %rbp

```

```

20     movq    %rsp, %rbp
21     subq    $16, %rsp //开辟栈空间
22     movl    $0, -4(%rbp) //访问栈内存中的变量a
23     ...
24     ret
25
26 main:
27 .LFB1:
28     ...
29     call    func1 //调用函数
30     ...
31     ret
32

```

3.4.5.2 取值操作：[]、*、->、.

```

1  #include <stdio.h>
2  struct abc {
3      int a;
4      int b;
5      int c;
6  };
7
8  int main(void)
9  {
10     struct abc d = {
11         .a = 1,
12         .b = 2,
13         .c = 3,
14     };
15     struct abc *p = &d;
16     printf("%d %d %d \n", d.a, d.b, d.c);
17     printf("%d %d %d \n", p->a, p->b, p->c);
18
19     int a[3] = {1,2,3};
20     int *p1 = a;
21     printf("%d %d %d \n", a[0], a[1], a[2]);
22     printf("%d %d %d \n", (*p1), (*(p1+1)), (*(p1+2)));
23
24     return 0;
25 }
26
27 #程序运行结果:
28 1 2 3
29 1 2 3

```

```
30 1 2 3
31 1 2 3
```

3.4.5.3 取址操作：&

```
1 #include <stdio.h>
2
3 int func1(void)
4 {
5     int a = 0;
6     printf("addr of a is %p \n", &a);
7
8     return 0;
9 }
10
11 Tips: & 还有位与的功能。
12 1、&左边有值就是位与：0x1 & a
13 2、&左边没有值就是取变量地址，&a
```

3.4.5.4 内存打包：{}

结构体、联合体、枚举、函数就是典型的通过{}进行打包

```
1 #include <stdio.h>
2
3 struct abc {
4     int a;
5     int b;
6     int c;
7 }
8
9 void func(void)
10 {
11     return;
12 }
```

4. 函数视角

希望大家养成一个好的编程"洁癖": Don't Repeat Yourself, 不要写重复的代码。所以当别人问你函数有什么用时, 你就可以这么回答他。C语言是一个面向过程的语言, 即面向方法(函数)编程(C++/Java 是面向对象编程), 所以理解和应用函数的重要性不言而喻。

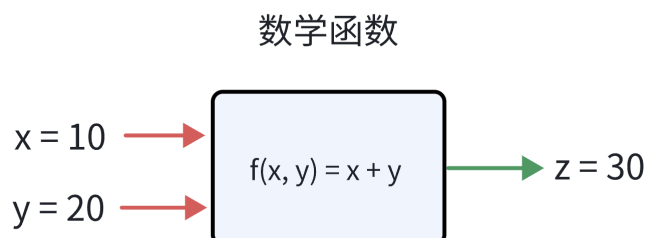
```

1 //取绝对值的逻辑
2 if (a < 0)
3     a = -a;
4 else
5     a = a;
6
7 int abs(int a)
8 {
9     if (a < 0)
10         a = -a;
11     else
12         a = a;
13     return a;
14 }

```

4.1 函数的世界

啥是函数？函数的英文是"function"，翻译过来就是功能。所以函数字面意思就是：封装一些逻辑，实现一个功能。举一个例子：



4.1.1 函数三大属性

把上面那个例子进行抽象，就可以得到函数的三大属性：输入参数、返回值、函数名。

```
1 1. 函数名（地址）
2 2. 输入参数（可多个）
3 3. 返回值（至多一个）
4
5 输出：函数名：输入
6 int function(int, char)
7 {
8     xxx
9 }
10
11 Tips：编译器只要看到有这三到属性，就判定为函数（即使函数体是空的）
```

- 函数名本质上是一个**地址标签**。如果知道函数的地址，就可以直接用()调过去

```
1 #include <stdio.h>
2
3 int function(int a, int b)
4 {
5     return (a+b);
6 }
7
8 void main(void)
9 {
10     function(1, 2);
11 }
12
13 通过命令：objdump -d bin > bin.s 进行反汇编得到：
14
15 0000000000001129 <function>:
16     1129:  f3 0f 1e fa      endbr64
17     112d:  55              push    %rbp
18     112e:  48 89 e5        mov     %rsp,%rbp
19     1131:  89 7d fc        mov     %edi,-0x4(%rbp)
20     1134:  89 75 f8        mov     %esi,-0x8(%rbp)
21     1137:  8b 55 fc        mov     -0x4(%rbp),%edx
22     113a:  8b 45 f8        mov     -0x8(%rbp),%eax
23     113d:  01 d0          add     %edx,%eax
24     113f:  5d              pop     %rbp
25     1140:  c3              retq
26
27 0000000000001141 <main>:
28     1141:  f3 0f 1e fa      endbr64
```

29	1145:	55	push	%rbp
30	1146:	48 89 e5	mov	%rsp,%rbp
31	1149:	be 02 00 00 00	mov	\$0x2,%esi
32	114e:	bf 01 00 00 00	mov	\$0x1,%edi
33	1153:	e8 d1 ff ff ff	callq	1129 <function>
34	1158:	90	nop	
35	1159:	5d	pop	%rbp
36	115a:	c3	retq	
37	115b:	0f 1f 44 00 00	nopl	0x0(%rax,%rax,1)

```

1 //函数名本质上是一个地址，体会下面的例子
2 #include <stdio.h>
3 #include <stdlib.h>
4 #include <string.h>
5 //extern int printf (const char *__restrict __format, ...)
6 int (*show)(const char *, ...);
7 void func(void)
8 {
9     show = printf;
10    show("func call \n");
11 }
12
13 int main(void)
14 {
15     func();
16     return 0;
17 }
18
19 #程序运行结果:
20 func call

```

4.1.2 参数传递

4.1.2.1 参数传递的本质

调用函数时，需要传入和返回参数。传入和返回的参数过程本质上是：**内存拷贝**。既然是拷贝，那一定存在两个对象：目的地（dest），源（src）。在C语言中，传入参数时，目的地叫**形参**、源叫**实参**；返回参数时，目的地和源都叫返回值。

```

1 #include <stdio.h>
2 int show(int a) //a 是形参，传递过程，就把num的值拷贝给了a
3 {
4     printf("%d \n", a);

```

```

5     a++;
6
7     return a; //a此时是返回值
8 }
9
10 void main(void)
11 {
12     int num = 10086;
13     int ret = 0;
14
15     //num 是实参
16     ret = show(num); // show函数返回a的值会拷贝给ret
17 }

```

4.1.2.2 值传递

因为存在拷贝的机制，值传递的时，不会对调用者的**源数据**进行破坏，所以值传递对数据起到保护和隔离的作用。体会一下下面的例子：

```

1  #include <stdio.h>
2
3  int show(int a)
4  {
5      a++;
6      printf("a in show %d \n", a);
7      return a;
8  }
9
10 void main(void)
11 {
12     int num = 10086; //源数据
13     int ret = 0; //返回值
14
15     ret = show(num);
16     printf("a in main %d , ret = %d \n", num, ret);
17 }
18
19 #程序运行结果:
20 a in show 10087
21 a in main 10086 , ret = 10087

```

另外，大家要养成一个思维习惯：当函数结束时，函数里的局部变量（形参也是局部变量）都会被“销毁”，即使内存中原来的值还存在，但已经不受到系统保护，数据随时可能被覆盖、改写等等。所以当我们返回一个局部变量的指针时，需要考虑指针指向的内存的生命周期。

- 恢复栈中的局部变量：在分析crash问题的时，通常需要分析调用栈，可以通过保存之前的栈针地址，查看原来内存的值。可参考：

```
1 #include <stdio.h>
2
3 static unsigned long addr = 0;
4
5 void show(void)
6 {
7     int a = 10086;
8     int b = 10010;
9
10    asm volatile("movq %rbp, addr(%rip)");
11 }
12
13 void main(void)
14 {
15     show();
16
17     int *p = (int*)addr;
18     printf("show bp = 0x%lx, a = %d, b = %d \n",addr, p[-1], p[-2]);
19
20 }
21 #程序运行结果:
22 show bp = 0x7fffffff2cacb0, a = 10010, b = 10086
```

4.1.2.3 地址传递

地址传递本质上跟值传递一样，只不过这个值有特殊的含义：代表了一个地址编号。地址传递一般用于**返回结果**和**连续空间传递**。

4.1.2.3.1 形参当做返回值

```
1 #include <stdio.h>
2
3 int func(void)
4 {
5     return 100 * 2;
6 }
7
8 void main(void)
9 {
10     int ret;
11
```

```

12     ret = func();
13
14     printf("ret = %d \n", r1);
15 }
16 #程序运行结果:
17 ret = 200

```

4.1.2.3.2 多值返回

我们知道，函数返回return后面只能带有一个值，所以如果有多个值需要返回的需求，可以利用值传递来做，比如，可以这样设计：

```

1  #include <stdio.h>
2
3  int func(int *r1, int* r2)
4  {
5      if (!r1 || !r2) {
6          return -1;
7      }
8
9      *r1 = 100 * 2;
10     *r2 = 200 * 2;
11
12     return 0;
13 }
14
15 void main(void)
16 {
17     int ret, r1, r2;
18
19     ret = func(&r1, &r2);
20     printf("ret = %d, a = %d, b = %d \n", ret, r1, r2);
21 }
22 #程序运行结果:
23 ret = 0, a = 200, b = 1000
24
25 Tips: 在这种情况下，这里的返回值ret代表的是func运行是否存在异常（linux 中 0 代表正常，非
    零代表异常），大家可以借鉴这个设计思想。

```

4.1.2.3.3 连续空间传递

因为参数传递是内存拷贝，所以如果传入的参数是一片连续的空间，那每次调用都会进行冗余的内存分配，造成内存不必要的浪费。所以连续空间，一般都是传这片空间的首地址。

```

1 #include <stdio.h>
2
3 struct abc {
4     int a;
5     int b;
6     int c;
7 };
8
9 int func(struct abc *p, int b[])
10 {
11     printf("a = %d, b = %d, c = %d \n", p->a, p->b, p->c);
12     printf("%d, %d,\n", b[0], b[1]);
13     return 0;
14 }
15
16 void main(void)
17 {
18     int ret = 0;
19     struct abc a = {
20         .a = 1,
21         .b = 2,
22         .c = 3,
23     };
24     int b[2] = {4,5};
25
26     ret = func(&a, b);
27 }
28
29 #程序运行结果:
30 a = 1, b = 2, c = 3
31 4, 5,

```

1. 非字符空间

对于非字符空间，在函数设计的时候，需要传入连续地址的长度，否则默认为是**返回结果**的设计。

```

1 #include <stdio.h>
2
3 int func(int *p, int len)
4 {
5     int i;
6
7     for(i = 0; i < len ; i++) {
8         printf("%d ", p[i]);

```

```

9     }
10    return 0;
11 }
12
13 void main(void)
14 {
15     int ret;
16     int arr[2] = {1,2};
17
18     ret = func(arr, 2);
19 }
20 #程序运行结果:
21 1 2
22
23 #系统函数
24 void *memcpy(void *dest, const void *src, size_t n);

```

2. 字符空间

在C语言中，字符空间的结束符是'\0'，所以默认情况下，传入字符空间可以不用传入字符传的长度。如：

```

1  #include <stdio.h>
2
3  int func(const char *s)
4  {
5      int i = 0;
6      while (s[i] != 0) {
7          printf("%c", s[i]);
8          i++;
9      }
10     printf("\n");
11     return 0;
12 }
13
14 void main(void)
15 {
16     int ret = 0;
17     ret = func("hello world");
18 }
19
20 #程序运行结果:
21 hello world
22
23 #系统函数

```

```
24 char *strcpy(char *dest, const char *src);
25 size_t strlen(const char *s)
```

4.2 C与面向对象

C语言是面向过程的语言，但我们可以借助C++/JAVA面向对象思想，实现类似 "继承"、"多态"、"封装"、"重载"的功能。从而服务于架构设计。

4.2.1 C与"继承"

继承是使用已存在的类作为基础建立新的类的技术。新的类可以增加新的数据类型或者方法，也可以使用父类的功能，但不能选择性的继承父类。通过继承可以很好的复用以前的代码，提高开发效率。

```
1  #include <iostream>
2
3  using namespace std;
4
5  // 基类
6  class shape {
7      protected:
8          int width;
9          int height;
10
11      public:
12          void setWidth(int w) {
13              width = w;
14          }
15          void setHeight(int h) {
16              height = h;
17          }
18 };
19
20 // 派生类
21 class rectangle : public shape
22 {
23     public:
24         int getArea() {
25             return (width * height);
26         }
27 };
28
29 int main(void)
30 {
31     rectangle rect;
```

```

32
33     rect.setWidth(5);
34     rect.setHeight(7);
35
36     // 输出对象的面积
37     cout << "Total area: " << rect.getArea() << endl;
38
39     return 0;
40 }

```

```

1  #include <stdio.h>
2  #include <string.h>
3
4  // 基类
5  struct shape {
6      int width;
7      int height;
8      void (*setWidth)(struct shape*s, int w);
9      void (*setHeight)(struct shape*s, int h);
10 };
11
12 void setWidth(struct shape*s, int w)
13 {
14     s->width = w;
15 }
16
17 void setHeight(struct shape*s, int h)
18 {
19     s->height = h;
20 }
21
22 // 派生类
23 struct rectangle {
24     struct shape s;
25     int (*getArea)(struct shape *s);
26 };
27
28 int getArea(struct shape *s)
29 {
30     return (s->width * s->height);
31 }
32
33 int main(void)
34 {
35     struct rectangle rect = {

```

```

36     .setWidth = setWidth,
37     .setHeight = setHeight,
38     .getArea = getArea,
39 };
40
41 rect.s.setWidth(&rect.s, 5);
42 rect.s.setHeight(&rect.s, 7);
43
44 // 输出对象的面积
45 printf("Total area: %d \n", rect.getArea(&rect.s));
46
47 return 0;
48 }
49

```

- linux设备驱动中的继承

1 //linux中设备模型的继承关系: platform_device : device : kobject

2 1、kobject: 它是Linux设备模型的核心之一, 用于表示内核中的各种实体, 如设备、驱动程序、总线、类别等

3 2、device: 用于表示内核中设备的各种实体, 如i2c、spi、platform设备等

4 3、platform_device: 用于表示内核中platform设备实体

```

5
6 struct kobject {
7     const char          *name;
8     struct list_head    entry;
9     struct kobject      *parent;
10    struct kset          *kset;
11    struct kobj_type      *ktype;
12    struct kernfs_node    *sd;
13    struct kref           kref;
14    ....
15 }
16
17 struct device {
18     struct device        *parent;
19
20     struct device_private *p;
21
22     struct kobject kobj;
23     .....
24 };
25
26 struct platform_device {
27     const char *name;

```

```

28     int id;
29     struct device dev;
30     ....
31 };

```

4.2.2 C与"封装"

封装的概念是指：将抽象的数据和行为（或者方法）相结合，形成一个有机的整体。是面向对象思想的核心，目的是增强安全性和简化编程，使用者不必理解具体的实现细节，通过调用接口使用数据成员。

```

1 class car {
2     protected:
3         int direction;
4         int throttle;
5         int brake;
6
7     public:
8         void setDirection(int d) {
9             direction = d;
10        }
11
12        void setThrottle(int t) {
13            throttle = t;
14        }
15
16        void setBrake(int b) {
17            brake = b;
18        }
19 };

```

体会以下的代码设计：

```

1 #include <stdio.h>
2
3 class person {
4     protected:
5         char name[12];
6         int age;
7
8     public:
9         void get_name(char *n) {
10             strcpy(n, name);

```

```

11     }
12     void get_age(int *a) {
13         *a = age;
14     }
15     person(const char *n, int a) {
16         strcpy(name, n);
17         age = a;
18     }
19 };
20
21 int main(void)
22 {
23     char name[12];
24     int age;
25     person p("xiaohua", 21);
26
27     p.get_age(&age);
28     p.get_name(name);
29
30     printf("name:%s, age:%d \n", name, age);
31     return 0;
32 }
33
34 //运行结果
35 name:xiaohua, age:18

```

```

1 #include <stdio.h>
2 #include <string.h>
3
4 struct person {
5     char name[12];
6     int age;
7     void (*get_name)(struct person *p, char *name);
8     void (*get_age)(struct person *p, int *age);
9 };
10
11 static void get_name(struct person *p, char *name)
12 {
13     strcpy(name, p->name);
14 }
15
16 static void get_age(struct person *p, int *age)
17 {
18     *age = p->age;
19 }

```

```

20
21 static struct person p1 = {
22     .name = "xiaohua",
23     .age = 21,
24     .get_age = get_age,
25     .get_name = get_name,
26 };
27
28 int main(void)
29 {
30     char name[12] = {0};
31     int age = 0;
32     struct person *p = &p1;
33
34     p->get_age(p, &age);
35     p->get_name(p, name);
36
37     printf("name:%s, age:%d \n", name, age);
38     return 0;
39 }
40
41 //运行结果
42 name:xiaohua, age:18

```

```

1 //linux 中文件系统中inode中inode_operations的设计
2 struct inode {
3     struct hlist_node    i_hash;           /* 哈希表 */
4     struct list_head     i_list;           /* 索引节点链表 */
5     struct list_head     i_dentry;        /* 目录项链表 */
6     unsigned long        i_ino;           /* 节点号 */
7     atomic_t             i_count;         /* 引用记数 */
8     umode_t              i_mode;          /* 访问权限控制 */
9     unsigned int         i_nlink;         /* 硬链接数 */
10    uid_t                 i_uid;           /* 使用者id */
11    gid_t                 i_gid;           /* 使用者id组 */
12    ...
13    struct inode_operations *i_op;         /* 索引节点操作表 */
14    ...
15 };
16
17 struct inode_operations {
18     struct dentry * (*lookup) (struct inode *, struct dentry *, struct
nameidata *);
19     void * (*follow_link) (struct dentry *, struct nameidata *);
20     int (*permission) (struct inode *, int);

```

```

21     struct posix_acl * (*get_acl)(struct inode *, int);
22
23     int (*readlink) (struct dentry *, char __user *,int);
24     void (*put_link) (struct dentry *, struct nameidata *, void *);
25
26     int (*create) (struct inode *,struct dentry *,int, struct nameidata *);
27     int (*link) (struct dentry *,struct inode *,struct dentry *);
28     int (*unlink) (struct inode *,struct dentry *);
29     int (*symlink) (struct inode *,struct dentry *,const char *);
30     int (*mkdir) (struct inode *,struct dentry *,int);
31     int (*rmdir) (struct inode *,struct dentry *);
32     int (*mknod) (struct inode *,struct dentry *,int,dev_t);
33     ...
34 };

```

4.2.3 C与"多态"

多态的概念是指：通过父类的指针来调用子类中的方法。作用是把不同的子类对象当做父类来看，屏蔽不同子类之间的差异，用于适应需求的不断变化，本质上是面向抽象编程，在C++中，多态的实现是通过**虚函数**来实现。体会一下以下这个例子：

```

1  #include <stdio.h>
2  #include <string.h>
3
4  class person {
5      protected:
6          char name[24];
7          int age;
8
9      public:
10         virtual void get_name(char *n) {
11             strcpy(n, name);
12         }
13         person(const char *n, int a) {
14             strcpy(name, n);
15             age = a;
16         }
17     };
18
19     class usa_person : public person {
20     public:
21         void get_name(char *n) {
22             sprintf(n, "%s", name);
23             sprintf(n+strlen(n), "%s", "-usa");
24         }

```

```

25     usa_person(const char *n, int a) : person(n, a) {
26     }
27 };
28
29 class china_person : public person {
30     public:
31     void get_name(char *n) {
32         sprintf(n, "%s", name);
33         sprintf(n+strlen(n), "%s", "-china");
34     }
35     china_person(const char *n, int a) : person(n, a) {
36     }
37 };
38
39 int main(void)
40 {
41     char name[24] = {0};
42     struct person *p = new usa_person("xiaohua", 21);
43
44     p->get_name(name);
45     printf("name:%s \n",name);
46     delete p;
47
48     p = new china_person("xiaohua", 21);
49     p->get_name(name);
50     printf("name:%s \n",name);
51     delete p;
52
53     return 0;
54 }
55

```

```

1  #include <stdio.h>
2  #include <string.h>
3
4  struct person {
5      char name[24];
6      int age;
7      void (*get_name)(struct person *p, char *name);
8  };
9
10 struct usa_person {
11     struct person p;
12     void (*get_name)(struct person *p, char *name);
13 };

```

```

14
15 static void get_name(struct person *p, char *name)
16 {
17     struct usa_person *child = (struct usa_person *)p;
18
19     child->get_name(p, name);
20 }
21
22 static void get_usa_name(struct person *p, char *name)
23 {
24     sprintf(name, "%s", p->name);
25     sprintf(name+strlen(p->name), "%s", "-usa");
26 }
27
28 static struct usa_person usa_p = {
29     .get_name = get_usa_name,
30     .p.get_name = get_name,
31     .p.name = "xiaohua",
32 };
33
34 int main(void)
35 {
36     char name[24] = {0};
37     struct person *p = (struct person *)&usa_p;
38
39     p->get_name(p, name);
40     printf("name:%s \n", name);
41     return 0;
42 }
43 //运行结果:通过基类person的对象, 调用派生类usa中的方法
44 name:usa-xiaohua

```

```

1 //linux的设备驱动模型就是典型的多态思想设计
2 struct platform_driver {
3     int (*probe)(struct platform_device *);
4     int (*remove)(struct platform_device *);
5     void (*shutdown)(struct platform_device *);
6     int (*suspend)(struct platform_device *, pm_message_t state);
7     int (*resume)(struct platform_device *);
8     struct device_driver driver;
9     ...
10 };
11
12 int platform_driver_register(struct platform_driver *drv)
13 {

```

```

14     ...
15     if (drv->probe)
16         drv->driver.probe = platform_drv_probe;
17     if (drv->remove)
18         drv->driver.remove = platform_drv_remove;
19     if (drv->shutdown)
20         drv->driver.shutdown = platform_drv_shutdown;
21     if (drv->suspend)
22         drv->driver.suspend = platform_drv_suspend;
23     if (drv->resume)
24         drv->driver.resume = platform_drv_resume;
25     return driver_register(&drv->driver);
26 }
27
28 static int platform_drv_probe(struct device *_dev)
29 {
30     struct platform_driver *drv = to_platform_driver(_dev->driver);
31     struct platform_device *dev = to_platform_device(_dev);
32     ...
33     return drv->probe(dev);
34 }
35
36 driver.probe -> platform_drv_probe->to_platform_driver(_dev->driver)-> drv-
>probe
37 (device_driver类)
  (platform_driver类)
38 //父类: driver, 派生类platform_driver, 通过driver的probe最终调用到platform_driver的
  probe

```

4.2.4 C与"重载"

重载的概念是指：在同一个范围中声明几个同名函数，但同名函数的形参不同（个数、类型、顺序），一般是用来处理功能类似但是数据类型不同的问题。严格意义上C的编译器不支持重载，但我们有一些方法达到相同的目的

```

1 //printf函数是C中典型有重载效果的函数
2 #include <stdio.h>
3
4 int main(void)
5 {
6     printf("%s \n", "hello world");
7     printf("%s %s \n", "hello world 1", "hello world 2");
8     printf("%d %s \n", 10, "hello world 1");
9     printf("%s %d \n", "hello world 1", 10);
10

```

```

11     return 0;
12 }

```

4.2.4.1 可变参数函数

```

1 void va_func(强制参数, 可变参数)
2 1. 强制参数有至少有一个, 代表以一种规则, 由函数定义者自行定义和解析。
3 2. 可变参数可以有多个, 函数定义者和调用者自行决定
4
5 va_list: 一个元素的结构体数组
6 ----/usr/lib/gcc/x86_64-linux-gnu/9/include-----
7 typedef __builtin_va_list __gnuc_va_list;
8 typedef __gnuc_va_list va_list; //一个元素的结构体数组
9
10 va_list = struct __va_list_tag {
11     unsigned int gp_offset;
12     unsigned int fp_offset;
13     void *overflow_arg_area;
14     void *reg_save_area;
15 } [1]
16
17 va_start: 定位到可变参数的地址
18 va_arg: 遍历可变参数
19 va_end: 终止va_list遍历
20
21 #define va_start(v,l)    __builtin_va_start(v,l)
22 #define va_end(v)        __builtin_va_end(v)
23 #define va_arg(v,l)      __builtin_va_arg(v,l)

```

```

1 #include <stdio.h>
2 #include <stdarg.h>
3
4 //这里约定n为后面参数的个数 (一种规则)
5 static void va_func(int n, ...)
6 {
7     va_list ptr;
8     va_start(ptr, n);
9     while (n-- > 0) {
10         printf("%d\n", va_arg(ptr, int));
11     }
12     va_end(ptr);
13 }
14

```

```

15 int main(void)
16 {
17     va_func(1, 100);
18     va_func(2, 200, 300);
19
20     return 0;
21 }
22
23 //运行结果:
24 100
25 200
26 300

```

```

1 //选自glibc-2.40
2 #include <libioP.h>
3 #include <stdarg.h>
4 #include <stdio.h>
5
6 #undef printf
7
8 /* Write formatted output to stdout from the format string FORMAT. */
9 /* VARARGS1 */
10 int
11 __printf (const char *format, ...)
12 {
13     va_list arg;
14     int done;
15
16     va_start (arg, format);
17     done = __vfprintf_internal (stdout, format, arg, 0);
18     va_end (arg);
19
20     return done;
21 }
22
23 #undef _IO_printf
24 ldbl_strong_alias (__printf, printf);
25 ldbl_strong_alias (__printf, _IO_printf);

```

4.2.4.2 回调函数+void*

```

1 #include <stdio.h>
2

```

```

3 class swap {
4     public:
5         void exchange(int *a, int *b) {
6             int tmp = *a;
7             *a = *b;
8             *b = tmp;
9         }
10        void exchange(double *a, double *b) {
11            double tmp = *a;
12            *a = *b;
13            *b = tmp;
14        }
15 };
16
17 int main(void)
18 {
19     swap s;
20     int a = 10;
21     int b = 20;
22     double c = 1.23;
23     double d = 4.56;
24
25     s.exchange(&a, &b);
26     s.exchange(&c, &d);
27
28     printf("a = %d, b = %d \n", a, b);
29     printf("c = %f, d = %f \n", c, d);
30     return 0;
31 }

```

```

1 //实现一个可以交换不同数据类型的swap函数
2 #include <stdio.h>
3 #include <stdarg.h>
4
5 typedef void (*func)(void*, void*);
6 void swap(func exchange, void *a, void *b)
7 {
8     exchange(a, b);
9 }
10
11 void exchange_int(void *a, void *b)
12 {
13     int *_a = (int*)a;
14     int *_b = (int*)b;
15     int tmp = 0;

```

```

16
17     tmp = *_a;
18     *_a = *_b;
19     *_b= tmp ;
20 }
21
22 void exchange_double(void *a, void *b)
23 {
24     double*_a = (double*)a;
25     double*_b = (double*)b;
26     double tmp = 0;
27
28     tmp = *_a;
29     *_a = *_b;
30     *_b= tmp;
31 }
32
33 void main(void)
34 {
35     int a = 10;
36     int b = 20;
37     double c = 1.23;
38     double d = 4.56;
39
40     swap(exchange_int, &a, &b);
41     swap(exchange_double, &c, &d);
42
43     printf("a = %d, b = %d \n", a , b);
44     printf("c = %f, d = %f \n", c , d);
45 }
46
47 //运行结果:
48 a = 20, b = 10
49 c = 4.560000, d = 1.230000

```

4.2.4.3 弱连接函数 (weak)

这个在驱动设计时，经常用到。weak修饰的函数属于弱连接函数，当前工程中如果有定义跟其相同的函数时，weak修饰的函数会被编译覆盖，达到override的效果。

```

1 //1.c
2 #include <stdio.h>
3
4 extern void config(void);
5

```

```

6 void __attribute__((weak)) config(void)
7 {
8     printf("config default\n");
9 }
10
11 void main(void)
12 {
13     config();
14 }
15
16 //2.c
17 #include <stdio.h>
18 void config(void)
19 {
20     //i2c bus no
21     //LDO
22     //hardware reg config
23     printf("config one \n");
24 }
25
26 //运行结果:
27 config two

```

```

1 //stm32中常用于设计中断回调函数，当用户自定义了相同的callback函数，就会跑用户自定义函数，
  表现出很好的拓展性设计
2
3 #ifndef __weak
4 #define __weak __attribute__((weak))
5 #endif
6
7 /**
8  * @brief EXTI line detection callbacks.
9  * @param GPIO_Pin: Specifies the pins connected EXTI line
10  * @retval None
11  */
12 __weak void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
13 {
14     /* Prevent unused argument(s) compilation warning */
15     UNUSED(GPIO_Pin);
16     /* NOTE: This function Should not be modified, when the callback is needed,
17      the HAL_GPIO_EXTI_Callback could be implemented in the user file
18      */
19 }

```

4.3 solid设计原则

solid设计是面向对象的原则，是由Gang of Four（四人帮），即Erich Gamma, Richard Helm, Ralph Johnson & John Vlissides四人）的《设计模式》（1995年出版）是第一次将设计模式提升到理论高度，并将之规范化。



这些原则属于建议性原则，大家在做架构设计的时候要从实际的业务角度出发，切记不要为了设计而设计（冗余设计）。

4.3.1 单一职责原则

一个类只负责一件事情。价值：高内聚，低耦合，拓展性和健壮性强。

```
1  #include <stdio.h>
2
3  class Game {
4  public:
5      DataCompute();
6      ~DataCompute();
7
8      void setShapen();
9      void setPlace();
10
11     void showViewCcon(); //游戏方块显示
12 private:
13     int mPlace[4];        //位置
14     int mShape[4];        //形状
15
16     DataCompute *mDataCompute; //游戏逻辑
17     int mStartButton;        //开始按钮
18     int mEndButton;         //结束按钮
19 };
```

```

1  #include <stdio.h>
2
3  //游戏逻辑
4  class DataCompute {
5  public:
6      DataCompute();
7      ~DataCompute();
8
9      void setShapen();
10     void setPlace();
11 private:
12     int mPlace[4];           //位置
13     int mShape[4];          //形状
14 };
15
16 //游戏界面
17 class GameUI
18 {
19 public:
20     GameUI();
21     ~GameUI();
22
23     void showViewCcon();     //游戏方块显示
24 private:
25     DataCompute *mDataCompute; //游戏逻辑
26     int mStartButton;          //开始按钮
27     int mEndButton;           //结束按钮
28 };

```

在C语言中，就是一个结构体在设计和封装的时候只负责一个事情。以一个imu器件的驱动结构体为例：

```

1  struct imu_operations {
2      int (*init)(struct int_param_s *int_param);
3      int (*set_bypass)(unsigned char bypass_on);
4      int (*enable_sensor)(unsigned char sensors);
5      int (*configure_fifo)(unsigned char sensors);
6      int (*set_sample_rate)(unsigned short rate);
7      int (*set_mag_sample_rate)(unsigned short rate);
8      int (*get_sample_rate)(unsigned short *rate);
9      int (*set_gyro_fsr)(unsigned short fsr);
10     int (*set_accel_fsr)(unsigned char fsr);
11     int (*get_gyro_fsr)(unsigned short *fsr);
12     int (*get_accel_fsr)(unsigned char *fsr);

```

```

13     int (*get_compass_fsr)(unsigned short *fsr);
14     int (*get_gyro_sens)(float *sens);
15     int (*get_mag_sens)(float *sens);
16     int (*get_accel_sens)(unsigned short *sens);
17     int (*read_gyro_data)(short *data, unsigned long *timestamp);
18     int (*read_accel_data)(short *data, unsigned long *timestamp);
19     int (*read_mag_data)(short *data, unsigned long *timestamp);
20 };
21
22 struct imu_chip {
23     struct imu_operations *ops;
24     struct imu_data accel_data[CALI_DATA_LEN];
25     struct imu_data gyro_data[CALI_DATA_LEN];
26     struct imu_data compass_data[CALI_DATA_LEN];
27     ...
28 }
29
30

```

4.3.2 开闭原则

软件中的对象（类、模块、函数等）应该满足对于拓展是开放的，对修改是封闭的。体会一下模块化编程的例子：

```

1 //keil stm32 方案
2 //module_init.h
3 typedef void (*init_t)(void);
4
5 #define module_init(fn) \
6 const init_t __embedi_##fn __attribute__((section("embedi_init"))) \
7 __attribute__((used)) = fn
8
9 //module_init.c
10 extern int embedi_init$$Base;
11 extern int embedi_init$$Length;
12
13 void embedi_module_init(void)
14 {
15     init_t* init_call = (init_t*)&embedi_init$$Base;
16     int count = 0;
17
18     count = (int)&embedi_init$$Length / sizeof(init_t);
19     while (count-->0) {
20         (*init_call)();
21         init_call++;
22     }
23 }

```

```

21     }
22     osLog("module init \n");
23 }
24
25 //1.c
26 static void module_one_init(void)
27 {
28     // do something
29 }
30 module_init(module_one_init);
31
32 //2.c
33 static void module_two_init(void)
34 {
35     // do something
36 }
37 module_init(module_two_init);

```

```

1 //GCC 方案
2 #include <stdio.h>
3
4 typedef void (*init_t)(void);
5
6 #define module_init(fn) \
7 const init_t __embedi_##fn __attribute__((section(".embedi_init"))) \
8 __attribute__((used)) = fn
9
10 //module_init.c
11 extern init_t __init_start;
12 extern init_t __init_end;
13
14 void embedi_module_init(void)
15 {
16     init_t* init_call = (init_t*)&__init_start;
17
18     for ( ; init_call < &__init_end; init_call++){
19         (*init_call)();
20     }
21 }
22
23 static void module_one_init(void)
24 {
25     printf("%s call \n", __FUNCTION__);
26 }
27 module_init(module_one_init);

```

```

27
28 static void module_two_init(void)
29 {
30     printf("%s call \n", __FUNCTION__);
31 }
32 module_init(module_two_init);
33
34 //1.lds
35 __init_start = .;
36 .embedi_init : {*(.embedi_init)}
37 __init_end = .;
38
39 //gcc -o bin 1.c -T 1.lds

```

4.3.3 里氏替换原则

里氏替换原则：简单来讲就是子类可以去拓展父类的功能，但不能改变父类原有的功能。技术上，所有引用基类的地方必须能够透明的使用其子类的对象。价值：继承一定程度上破坏了封装，此原则用来包含封装性。但嵌入式中应用相对较少，可以感受一下下面这个例子：

```

1 #include <iostream>
2
3 using namespace std;
4
5 // 基类
6 class birds {
7     protected:
8         int flySpeed;
9         int runSpeed;
10
11     public:
12         void setFlySpeed(int s) {
13             flySpeed = s;
14         }
15         void setRunSpeed(int s) {
16             runSpeed = s;
17         }
18         int getFlySpeed(void) {
19             return flySpeed;
20         }
21         int getRunSpeed(void) {
22             return runSpeed;
23         }
24         int calculateFlyTime(int distance) {
25             return distance / flySpeed;

```

```

26     }
27 };
28
29 // 派生类
30 class swallow : public birds
31 {
32
33 };
34
35 class ostrich : public birds
36 {
37     public:
38         void setFlySpeed(int s) {
39             flySpeed = 0;
40         }
41 };
42
43 int main(void)
44 {
45     birds *b1 = new swallow;
46     birds *b2 = new ostrich;
47     int distance = 100;
48
49     b1->setFlySpeed(10);
50     cout << "b1 flySpeed: " << b1->getFlySpeed() << endl;
51     cout << "b1 flyTime: " << b1->calculateFlyTime(distance) << endl;
52
53     b2->setFlySpeed(0);
54     cout << "b2 flySpeed: " << b2->getFlySpeed() << endl;
55     cout << "b2 flyTime: " << b2->calculateFlyTime(distance) << endl;
56
57     delete b1;
58     delete b2;
59
60     return 0;
61 }
62

```

```

1 #include <iostream>
2
3 using namespace std;
4
5 // 基类
6 class animal {
7     protected:

```

```

8      int runSpeed;
9
10     public:
11         void setRunSpeed(int s) {
12             runSpeed = s;
13         }
14         int getRunSpeed(void) {
15             return runSpeed;
16         }
17 };
18 class birds : public animal {
19     protected:
20         int flySpeed;
21
22     public:
23         void setFlySpeed(int s) {
24             flySpeed = s;
25         }
26         int getFlySpeed(void) {
27             return flySpeed;
28         }
29         int calculateFlyTime(int distance) {
30             return distance / flySpeed;
31         }
32 };
33
34 // 派生类
35 class swallow : public birds
36 {
37 };
38
39 class ostrich : public animal
40 {
41
42 };

```

```

1  #include <stdio.h>
2  #include <string.h>
3
4  struct birds {
5      int flySpeed;
6      int runSpeed;
7      void (*setFlySpeed)(struct birds *b, int s);
8      void (*setRunSpeed)(struct birds *b, int s);
9      int (*getFlySpeed)(struct birds *b);

```

```

10     int (*getRunSpeed)(struct birds *b);
11     int (*calculateFlyTime)(struct birds *b, int distance);
12 };
13
14 void setFlySpeed(struct birds *b, int s) {
15     b->flySpeed = s;
16 }
17 void setRunSpeed(struct birds *b, int s) {
18     b->runSpeed = s;
19 }
20 int getFlySpeed(struct birds *b) {
21     return b->flySpeed;
22 }
23 int getRunSpeed(struct birds *b) {
24     return b->runSpeed;
25 }
26 int calculateFlyTime(struct birds *b, int distance) {
27     return distance / b->flySpeed;
28 }
29
30 // 派生类
31 struct swallow {
32     struct birds b;
33 };
34
35 struct ostrich {
36     struct birds b;
37 };
38
39 struct swallow s = {
40     .b.setFlySpeed = setFlySpeed,
41     .b.setRunSpeed = setRunSpeed,
42     .b.getFlySpeed = getFlySpeed,
43     .b.getRunSpeed = getRunSpeed,
44     .b.calculateFlyTime = calculateFlyTime,
45 };
46
47 struct ostrich o = {
48     .b.setFlySpeed = setFlySpeed,
49     .b.setRunSpeed = setRunSpeed,
50     .b.getFlySpeed = getFlySpeed,
51     .b.getRunSpeed = getRunSpeed,
52     .b.calculateFlyTime = calculateFlyTime,
53 };
54
55 int main(void)
56 {

```

```

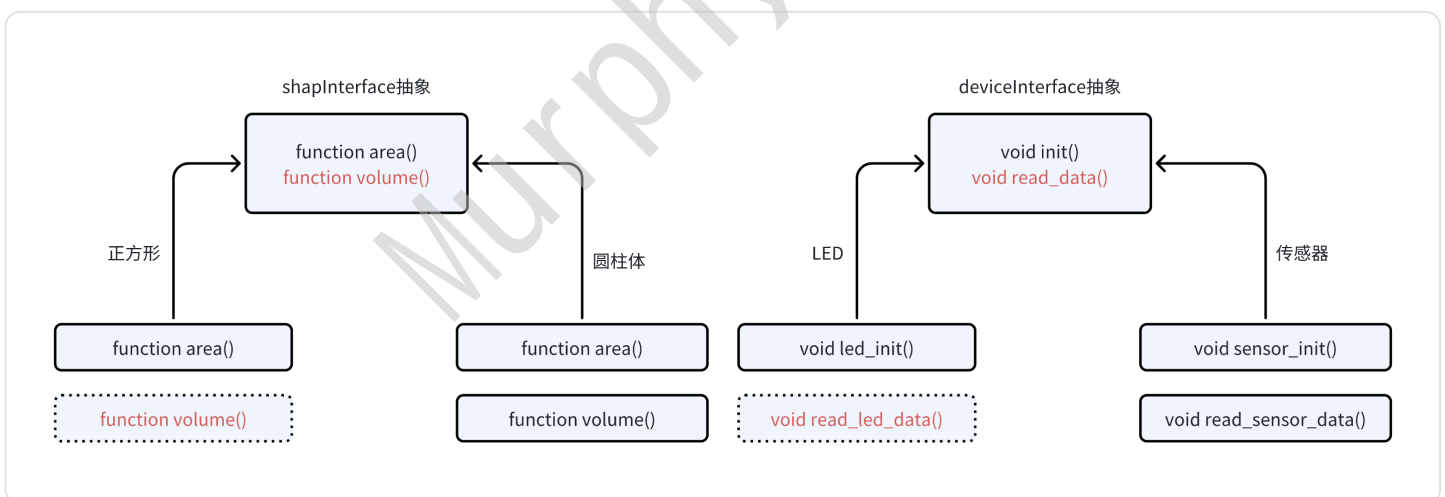
57     struct birds *b1 = (struct birds*)&s;
58     struct birds *b2 = (struct birds*)&o;
59     int distance = 100;
60
61     b1->setFlySpeed(b1, 10);
62     printf("b1 flySpeed: %d \n", b1->getFlySpeed(b1));
63     printf("b1 flyTime: %d \n", b1->calculateFlyTime(b1, distance));
64
65     b2->setFlySpeed(b2, 0);
66     printf("b2 flySpeed: %d \n", b2->getFlySpeed(b2));
67     printf("b2 flyTime: %d \n", b2->calculateFlyTime(b2, distance));
68
69     return 0;
70 }

```

4.3.4 接口隔离原则

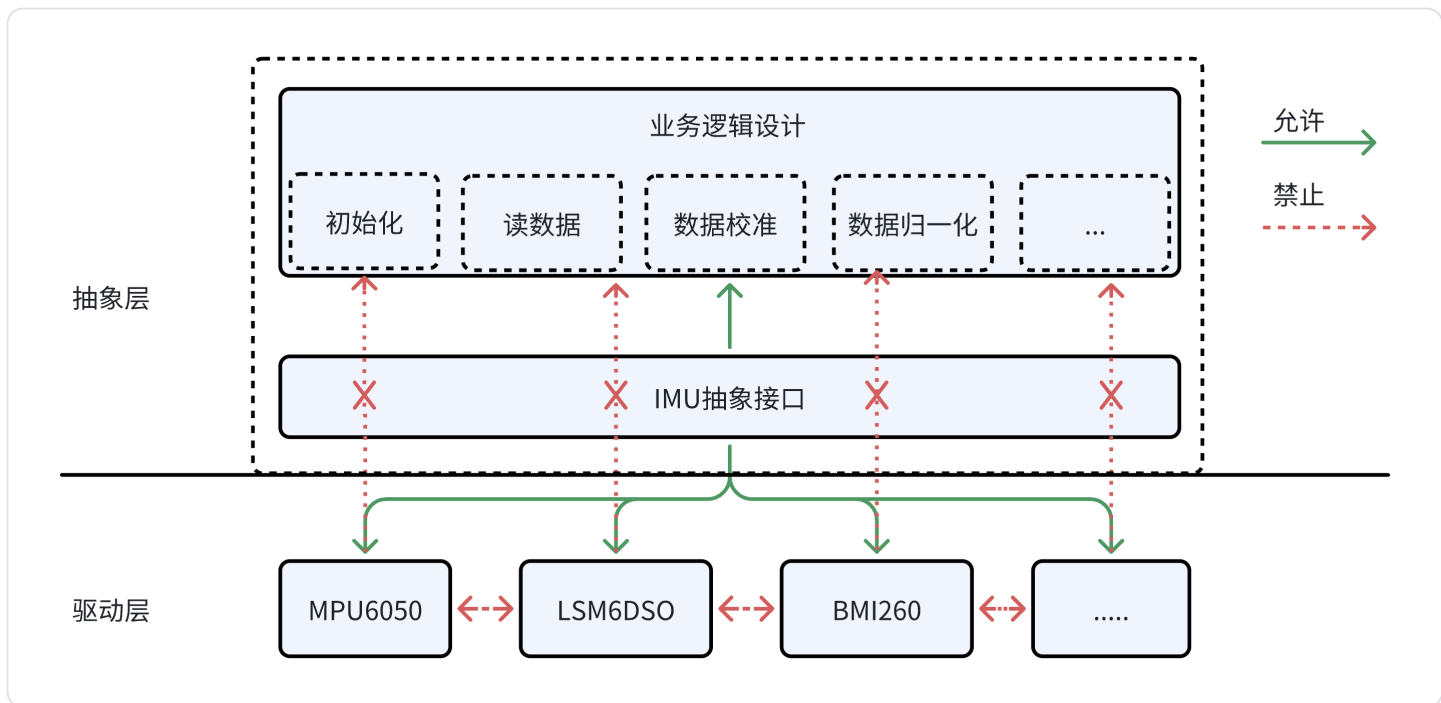
定义：不应该强迫用户端实现一个它用不上的接口。具体两个的例子：

1. 抽象了面积和体积的两个接口，客户端在使用的这个接口的时候会发现，对于正方形来说没有体积，所以实现体积的函数对它来说就是冗余的。
2. 抽象了器件初始化和读取数据的接口。对于LED灯来说，一般都是控制型，没有必要实现一个读LED数据的接口，所以对LED来说，读取数据就是冗余的。



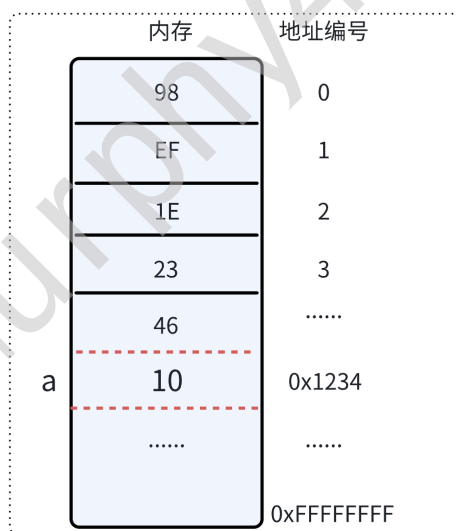
4.3.5 依赖倒置原则

高层次模块不应该依赖低层次的模块，实际上是面向抽象编程思想，这样可以降低代码间的耦合度，使其看扩展、易维护。举一个驱动的例子：在这个架构设计中，IMU有很多公共的业务逻辑：初始化、读取数据、数据校准等等，这些业务逻辑不应该耦合具体驱动接口，而是依赖于抽象接口；同时驱动与驱动之间也不能互相调用，而是调用抽象接口。后续的器件增加拓展，只需要修改驱动部分，抽象层都不会改动。



5. 内存视角

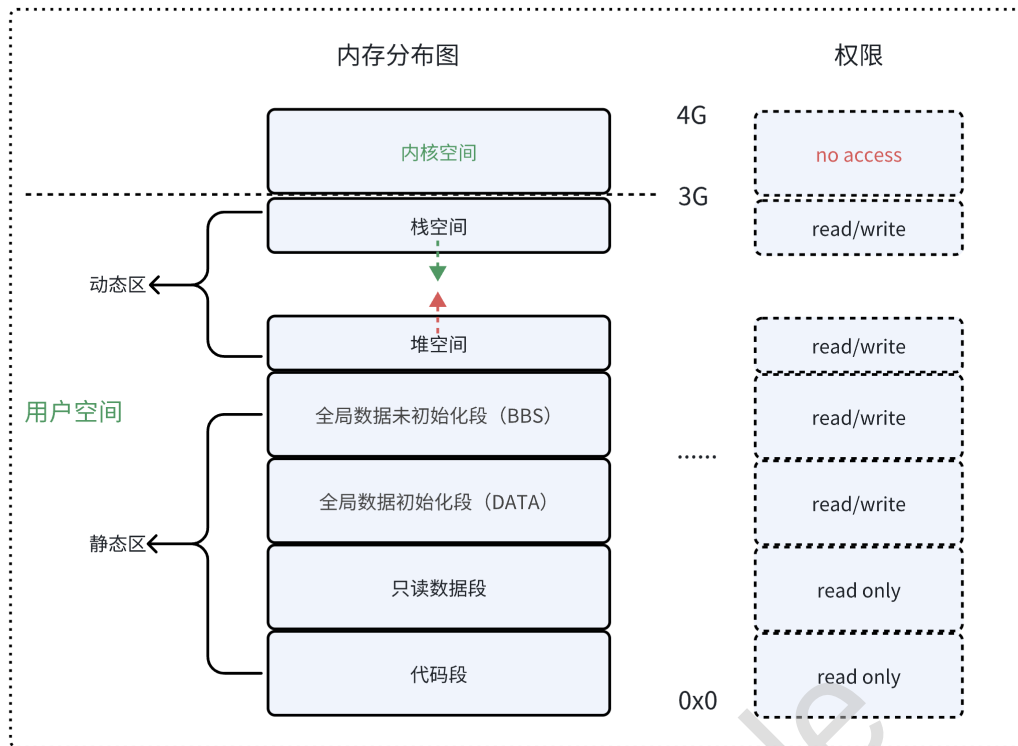
前面在学习数据类型关键字的时候我们了解到，定义一个变量的本质是在内存中圈定一块特定大小的内存。本章将会对内存的位置、生命周期、操作权限等做更多的探讨。



5.1 内存基础概念

5.1.1 内存空间分布图

内存分布分成静态区和动态区，静态区在编译时已经决定了内存分配大小，动态区是运行时分配。



```

1 #include <stdio.h>
2
3 int global_initied = 10; // 全局初始化变量
4 int global_uninitied; // 全局未初始化变量
5 int main(void)
6 {
7     int local_var = 20; // 局部变量
8     char *str = "Hello"; // 字符串常量
9     static int static_var = 30; // 静态变量
10    int *heap_var = (int *)malloc(sizeof(int)); // 动态分配的内存
11
12    printf("address of code: %p\n", main);
13    printf("address of str: %p\n", str);
14    printf("address of global_initied : %p\n", (void *)&global_initied );
15    printf("address of static_var: %p\n", (void *)&static_var);
16    printf("address of global_uninitied: %p\n", (void *)&global_uninitied);
17    printf("address of heap_var : %p\n", (void *)heap_var);
18    printf("address of local_var : %p\n", (void *)&local_var);
19
20    // 释放动态分配的内存
21    free(heap_var);
22
23    return 0;
24 }
25 //运行结果:
26 address of code: 0x55e737e1f1a9

```

```
27 address of str: 0x55e737e20008
28 address of global_inited : 0x55e737e22010
29 address of static_var: 0x55e737e22014
30 address of global_uninited: 0x55e737e2201c
31 address of heap_var : 0x55e738cb12a0
32 address of local_var : 0x7ffe44b03c14
```

5.1.1.1 代码段

代码段只能读不可以写，写会触发Segmentation fault (core dumped)，俗称"段错误"。在整个程序生命周期内合法有效。

```
1 #include <stdio.h>
2
3 void func(void)
4 {
5     printf("hello func \n");
6 }
7
8 int main(void)
9 {
10     printf("func addr %p \n", func);
11
12     void (*p)(void) = func;
13     //try to access
14     p();
15     int *p1 = (int *)func;
16     //try to read
17     printf("func addr %d \n", *p1);
18     //try to write
19     *p1 = 1;
20     return 0;
21 }
22 //运行结果:
23 func addr 0x561398e14169
24 hello func
25 func addr -98693133
26 Segmentation fault (core dumped)
```

5.1.1.2 只读数据段

只读数据段存放字符传常量，地址比代码段更高。权限是：只能读不可以写，写会触发Segmentation fault (core dumped)，在整个程序生命周期内合法有效。

```

1 #include <stdio.h>
2
3 int main(void)
4 {
5     printf("%s read only data addr: %p \n", "hello world", "hello world");
6     char *s = "hallo world";
7     s[1] = 'e';//try to write rodata Segmentation
8     return 0;
9 }
10 //运行结果:
11 read only data addr: 0x56014766f008
12 Segmentation fault (core dumped)

```

5.1.1.3 全局数据段

全局变量和局部static修饰的变量会放在全局数据段（data/bbs），允许读写，在整个程序生命周期内合法有效。

```

1 #include <stdio.h>
2 int a = 10;
3 int b;
4
5 int func(void)
6 {
7     static int d = 10;
8     b++;
9     return ++d;
10 }
11
12 int main(void)
13 {
14     static int c;
15     printf("%p , %p, %p \n", &a, &b, &c);
16     //try to write data Segmentation
17     a = 20;
18     b = 30;
19     c = 40;
20     printf("a = %d , b = %d, c = %d \n", a, b, c);
21
22     printf("b = %d , d = %d, d = %d\n", b, func(), func());
23     return 0;
24 }
25 //运行结果:
26 0x5556e9e9b010 , 0x5556e9e9b020, 0x5556e9e9b01c
27 a = 20 , b = 30, c = 40

```

```
28 b = 32 , d = 12, d = 11
29
```

5.1.1.4 堆空间

堆空间在运行时由程序员来分配（malloc）和释放（free），可读可写，生命周期是程序员来决定。值得注意的是，如果在一个函数中分配堆内存，函数执行结束后malloc空间不会被释放，如果不手动进行释放的话，会造成内存泄漏。

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 void func(void)
5 {
6     int *heap_var = (int *)malloc(sizeof(int)); // 动态分配的内存，单位是B，默认反
        馈的类型是void*
7     if (!heap_var) {
8         return;
9     }
10    //try to write heap Segmentation
11    *heap_var = 10;
12    printf("heap addr = %p , heap_var = %d\n", heap_var, *heap_var);
13    free(heap_var); //如果没有free动作， heap_var指向的内存就存在泄漏：不会再使用但又不
        能分配给别人
14 }
15
16 int main(void)
17 {
18     func();
19     return 0;
20 }
21
22
23 //运行结果:
24 heap addr = 0x55bf2b8db2a0 , heap_var = 10
```

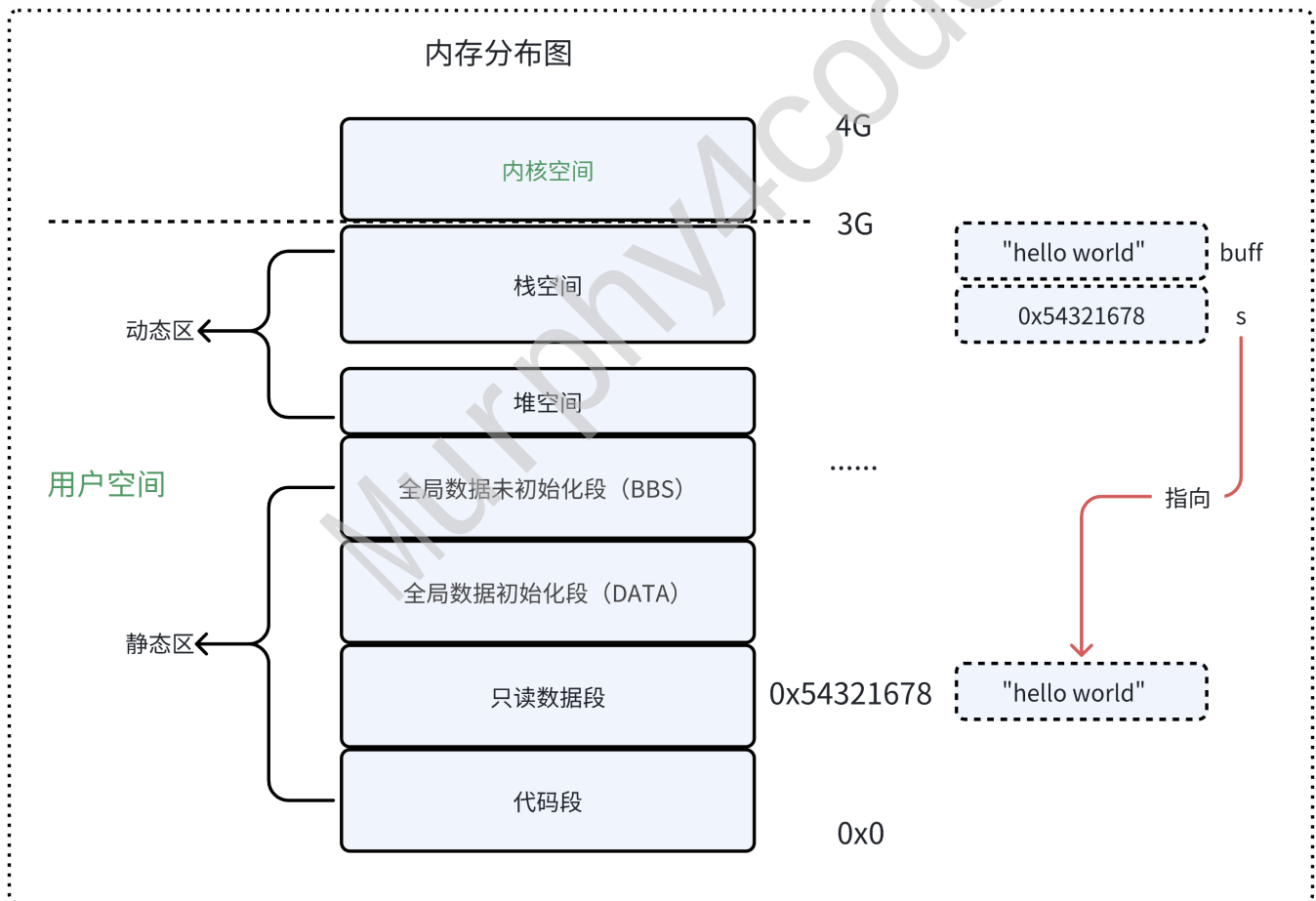
5.1.1.5 栈空间

栈空间指的是在函数运行时的上下文分配的空间，可读可写，生命周期在函数执行结束后结束。看下面的一个例子：buff和s都是局部变量，系统会给buff分配sizeof("hello world")栈空间，而s的栈空间是4byte，它指向只读数据段中的字符串"hello world"地址编号。所以这里返回buff就会有问题，返回指针没有问题。

```

1 #include <stdio.h>
2
3 char* func(void)
4 {
5     char *s = "hello world";
6     char buff[] = "hello world";
7
8     printf("func:%s \n", buff);
9     return s;
10 }
11 void main(void)
12 {
13     char *p = func();
14     printf("main:%s \n", p);
15 }

```



堆栈的生长方向问题:

```

1 #include <stdio.h>
2 #include <stdlib.h>
3

```

```

4 void func(int *a)
5 {
6     int *p1 = (int *)malloc(sizeof(int));
7     int *p2 = (int *)malloc(sizeof(int));
8     if (p2 > p1) {
9         printf("heap upward \n");
10    } else {
11        printf("heap downward \n");
12    }
13    free(p1);
14    free(p2);
15
16    int a = 1;
17    int b = 2;
18    if (&b > a) {
19        printf("stack upward \n");
20    } else {
21        printf("stack downward \n");
22    }
23 }
24
25 int main(void)
26 {
27     int a;
28
29     func(&a);
30     return 0;
31 }

```

5.1.2 内存溢出问题

内存溢出指的是程序运行过程中，访问超过其分配空间范围的内存区域。

5.1.2.1 栈溢出

- 递归函数：递归函数如果没有正常的退出条件，最后会把整个栈空间消耗殆尽，最后出现段错误。

```

1 #include <stdio.h>
2
3 void try_overflow(void)
4 {
5     int a = 10;
6     try_overflow();
7 }
8
9 void main(void)

```

```

10 {
11     try_overflow();
12 }
13 //运行结果:
14 Segmentation fault (core dumped)
15
16 //汇编
17 try_overflow:
18 .LFB0:
19     endbr64
20     pushq   %rbp
21     movq    %rsp, %rbp
22     subq    $16, %rsp //分配栈空间的内存
23     movl    $10, -4(%rbp)
24     call    try_overflow //反复调用
25     nop
26     leave
27     ret

```

- 很大的局部变量

```

1  #include <stdio.h>
2  #include <string.h>
3
4  #define N 8192*1024
5  void main(void)
6  {
7      char arr[N];
8      memset(arr, 'c', sizeof(char)*N);
9
10     for (int i = 0; i < N; i++) {
11         printf("%c", arr[i]);
12     }
13     puts("");
14 }
15
16 //运行结果:
17 Segmentation fault (core dumped)
18
19 murphy@ubuntu:/home/workspace/course$ ulimit -a
20 core file size          (blocks, -c) 0
21 data seg size           (kbytes, -d) unlimited
22 scheduling priority      (-e) 0
23 file size                (blocks, -f) unlimited
24 pending signals         (-i) 15152

```

```

25 max locked memory      (kbytes, -l) 65536
26 max memory size        (kbytes, -m) unlimited
27 open files              (-n) 1024
28 pipe size               (512 bytes, -p) 8
29 POSIX message queues    (bytes, -q) 819200
30 real-time priority      (-r) 0
31 stack size             (kbytes, -s) 8192
32 cpu time                (seconds, -t) unlimited
33 max user processes      (-u) 15152
34 virtual memory          (kbytes, -v) unlimited
35 file locks              (-x) unlimited
36

```

- 栈缓冲区溢出

```

1  #include <stdio.h>
2  #include <string.h>
3
4  #define N 10
5  void main(void)
6  {
7      char arr[N];
8      char arr1[] = "hello world";
9
10     strcpy(arr, arr1); //工程上建议不要用strcpy, 用strncpy
11     //strncpy(arr, arr1, N);
12     printf("arr:%s \n", arr);
13     printf("arr1:%s \n", arr1);
14 }
15 //strcpy运行结果
16 arr:hello world
17 arr1:d
18
19 //strncpy运行结果
20 arr:hello worlhello world //原因是arr长度不够, copy N个字符中缺少结束符'\0'
21 arr1:hello world

```

5.1.2.2 堆缓冲区溢出

```

1  #include <stdio.h>
2  #include <string.h>
3  #include <stdlib.h>
4

```

```

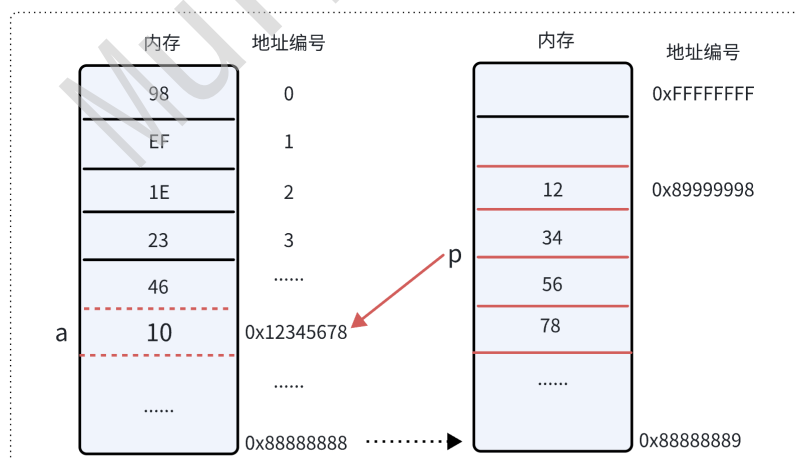
5 #define N 5
6 void main(void)
7 {
8     char *str = (char*)malloc(sizeof(char)*N);
9     char *str1 = (char*)malloc(sizeof(char)*N);
10
11     strcpy(str1, "hello world");
12     strcpy(str, "hello world");
13
14     printf("str:%s \n",str);
15     printf("str1:%s \n",str1);
16 }
17
18 //运行结果：溢出没有显示出异常，但已经污染了堆的其他区域
19 str:hello world
20 str1:hello world

```

5.2 指针

前面在介绍指针类型时，已经知道指针变量圈定的内存大小**跟编译器有关或者说跟CPU的地址总线有关**。在32位的系统中，指针类型占用4byte，在64位的系统中，指针类型占用8byte。指针变量的值有特殊意义，代表了一个地址编号，本章讲重点讲指针访问内存的规则和技巧。另外，希望大家对这两个概念：指针和指针变量，有清晰的区分。我们一般说指针，代表就是这个**指针变量的值**，即内存地址；指针变量就是一个变量，用于存放内存地址。

32位系统下指针访问



5.2.1 指针访问内存

要访问指针指向的地址，那就必然需要回答两个问题：1、内存的可读可写性是什么？2、内存的访问规则是什么？

5.2.1.1 指针变量的初始化

大家要养成思维习惯：每当看到一个指针变量，要对其值的合法性保持敬畏。

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int b = 20;
5 void main(void)
6 {
7     int a = 10;
8     int *p1 = &a;
9     int *p2 = &b;
10    int *p3 = (int*)malloc(sizeof(int));
11    if (!p3) {
12        return;
13    }
14    *p3 = 30;
15
16    int *p;
17    p = (int *)100; //从复制语法上可行，但内存不可读，不可写
18
19    printf("*p1 = %d, *p2 = %d, *p3 = %d \n", *p1, *(&b), p3[0]);
20    printf("p = %p \n", p);
21    printf("*p = %d \n", *p);
22
23    free(p3);
24 }
25
26 //运行结果:
27 *p1 = 10, *p2 = 20, *p3 = 30
28 p = 0x64
29 Segmentation fault (core dumped)
```

5.2.1.2 空指针和野指针

空指针很好理解，就是值为0（NULL）的指针变量，如果访问了0地址就会出现非法访问的错误。野指针是指值为非法地址的指针变量。野指针在实际工程中有很大的危害，会造成代码的稳定问题（不可预知的bug）。要形成free后，把指针变量置空（NULL）的习惯。

```
1 #include <stdio.h>
2 #include <string.h>
3 #include <stdlib.h>
4
5 void main(void)
6 {
```

```

7     int *ptr = NULL;
8     printf(" *p = %d \n", *ptr);
9
10    ptr = (int*)malloc(sizeof(int));
11    *ptr = 102;
12    printf(" *p = %d \n", *ptr);
13
14    free(ptr);
15 }

```

```

1  #include <stdio.h>
2  #include <string.h>
3  #include <stdlib.h>
4
5  void main(void)
6  {
7      int *ptr = (int*)malloc(sizeof(int));
8      printf(" *ptr = %d \n", *ptr);
9      free(ptr);
10     //ptr = NULL;
11     if (NULL != ptr) {
12         *ptr = 102;
13     }
14
15     int *p = (int*)malloc(sizeof(int));
16     printf(" *p = %d \n", *p);
17     printf(" ptr = %p \n", ptr);
18     printf("  p = %p \n", p);
19     // *ptr = 101;
20 }
21
22 //运行结果:
23 *ptr = 0
24 *p = 102 //指针执行的内容被别的指针 (ptr) 修改了
25 ptr = 0x55bf1f2cc2a0 //ptr就是野指针, 指向的这块内存已经不属于它, 但是它仍可以非法访问
26 p = 0x55bf1f2cc2a0

```

5.2.1.3 指针访问内存的规则

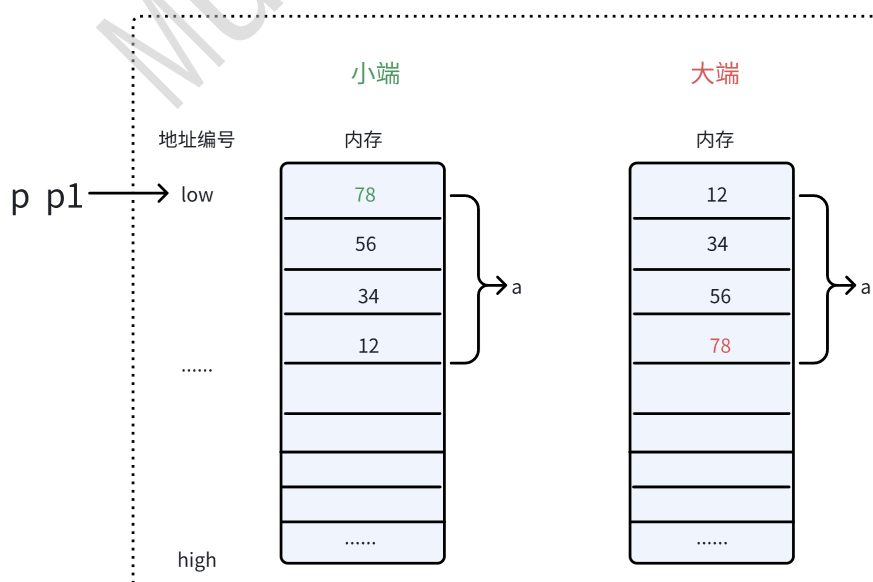
我们前面学过几种指针访问内存的方式: $*p$ 、 $p[x]$ 、 $p \rightarrow x$ 。那每次访问的大小是多少呢? 我们知道指针变量的定义的形式是: 数据类型 $*p$; 所以每次访问的大小由前面修饰的数据类型决定。

5.2.1.3.1 标准数据类型指针

```

1 #include <stdio.h>
2
3 void main(void)
4 {
5     int a = 0x12345678;
6
7     int *p = &a;
8     char *p1 = (char*)&a;
9
10    printf(" *p = 0x%x \n", *p);
11    printf(" *p1 = 0x%x \n", *p1);
12
13    printf(" p1[0] = 0x%x \n", p1[0]);
14    printf(" p1[1] = 0x%x \n", p1[1]);
15    printf(" p1[2] = 0x%x \n", p1[2]);
16    printf(" p1[3] = 0x%x \n", p1[3]);
17 }
18 //运行结果
19 *p = 0x12345678
20 *p1 = 0x78
21 p1[0] = 0x78
22 p1[1] = 0x56
23 p1[2] = 0x34
24 p1[3] = 0x12
25
26 Tips:
27 1、记住，不管大端还是小端，指针p和p1都是指向低地址。

```



5.2.1.3.2 连续空间类型指针

```

1 #include <stdio.h>
2
3 struct abc {
4     int a;
5     int b;
6     char c;
7 };
8
9 int arr[3] = {1,2,3};
10
11 int main(void)
12 {
13     struct abc data = {
14         .a = 1,
15         .b = 2,
16         .c = 3,
17     };
18     printf(" size of struct abc = %ld\n", sizeof(struct abc));
19
20     struct abc *p = &data;
21     printf(" a = %d, b = %d, c = %d\n", p->a, p->b, p->c);
22
23     int* p1 = (int *)&data;
24     printf(" struct p1[0] = %d \n", p1[0]);
25     printf(" struct p1[1] = %d \n", p1[1]);
26     printf(" struct p1[2] = %d \n", p1[2]);
27
28     int* p2 = arr;
29     printf(" arr p2[0] = %d \n", p2[0]);
30     printf(" arr p2[1] = %d \n", p2[1]);
31     printf(" arr p2[2] = %d \n", p2[2]);
32     return 0;
33 }
34
35 //运行结果:
36 a = 1, b = 2, c = 3
37 p1[0] = 1
38 p1[1] = 2
39 p1[2] = 3

```

```

1 //linux container_of 宏的思想
2 #define offsetof(TYPE, MEMBER) ((size_t) &((TYPE *)0)->MEMBER)
3 #define container_of(ptr, type, member) ({ \
4     const typeof( ((type *)0)->member ) *__mptr = (ptr); \
5     (type *) ( (char *)__mptr - offsetof(type,member) );})

```

```

6
7 #include <stdio.h>
8 struct abc {
9     int a;
10    int b;
11    char c;
12 };
13
14 void find_struct(int *member)
15 {
16     unsigned long offset = 0;
17     struct abc* p = NULL;
18
19     offset = (unsigned long)&((struct abc*)0)->b;
20     printf(" member offset: %ld \n", offset);
21
22     p = (struct abc *)((char*)member - offset);
23
24     printf(" a:%d, b:%d c: %d \n", p->a, p->b, p->c);
25 }
26
27 int main(void)
28 {
29     struct abc data = {
30         .a = 1,
31         .b = 2,
32         .c = 3,
33     };
34     printf(" size of struct abc = %ld\n", sizeof(struct abc));
35
36     struct abc *p = &data;
37     printf(" a = %d, b = %d, c = %d\n", p->a, p->b, p->c);
38
39     find_struct(&data.b);
40     return 0;
41 }
42

```

5.2.1.3.3 函数类型指针

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4
5 int (*show)(const char *, ...);

```

```

6 void func(void)
7 {
8     show = printf;
9     show("func call \n");
10 }
11
12 int main(void)
13 {
14     func();
15     return 0;
16 }
17
18 tips: 函数指针结合typedef来用, 代码更容易读懂:
19 typedef void(*func)(void);
20 func call;//定义一个指针变量
21 call();//调用

```

5.2.1.3.4 指针修改const变量

```

1 #include <stdio.h>
2
3 int main(void)
4 {
5     int a = 0x12345678;
6     const int b = 0x11111111;
7
8     int *p = &a;
9     p[1] = 0x22222222;//越界访问
10
11     printf("a = 0x%x, b = 0x%x \n", a, b);
12
13     return 0;
14 }
15
16 //运行结果: 编译连警告都没有
17 a = 0x12345678, b = 0x22222222

```

5.2.2 指针运算

5.2.2.1 指针+、-、++、--运算

指针变量本质也是一个变量, 所以进行算数运算语法上是可以的。但是实际的工程应用中, 指针变量的乘除法没有意义, 更多的是加减法。总的来说, + 运算符用于指针的算术运算, 允许将指针移动

到任意偏移位置，而 ++ 运算符是 + 运算符的特例，它递增指针指向的位置，移动一个对象的大小。看一个例子：

```
1 #include <stdio.h>
2
3 int main(void)
4 {
5     int array[] = {1, 2, 3, 4, 5};
6     int *p = array;
7
8     p = p + 2; //p[2]
9     printf("*p = %d \n", *p);
10
11    p++; //p[3]
12    printf("*p = %d \n", *p);
13
14    return 0;
15 }
16
17 //运行结果
18 *p = 3
19 *p = 4
```

5.2.2.2 指针逻辑运算

逻辑运算中，判断两个指针是否相等比较常用，其他的不常用。另外，只有两个相同类型的指针比较才有意义，一般编译也会包警告，如果类型不同。

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main(void)
5 {
6     int *p = (int*)malloc(sizeof(int));
7     if (p == NULL) {
8         return -1;
9     }
10
11    char *p1 = NULL;
12    if (p1 == p) { // warning
13        //do something
14    }
15    free(p);
16    return 0;
```

```

17 }
18
19 //运行结果
20 1.c: In function 'main':
21 1.c:12:12: warning: comparison of distinct pointer types lacks a cast
22     12 |         if (p1 == p) {
23         |

```

```

1 #参考linux下MAX的实现(3.18内核)
2 #define MAX(x, y) ({
3     typeof(x) _max1 = (x);
4     typeof(y) _max2 = (y);
5     (void) (&_max1 == &_max2);
6     _max1 > _max2 ? _max1 : _max2; })

```

5.2.3 多级指针

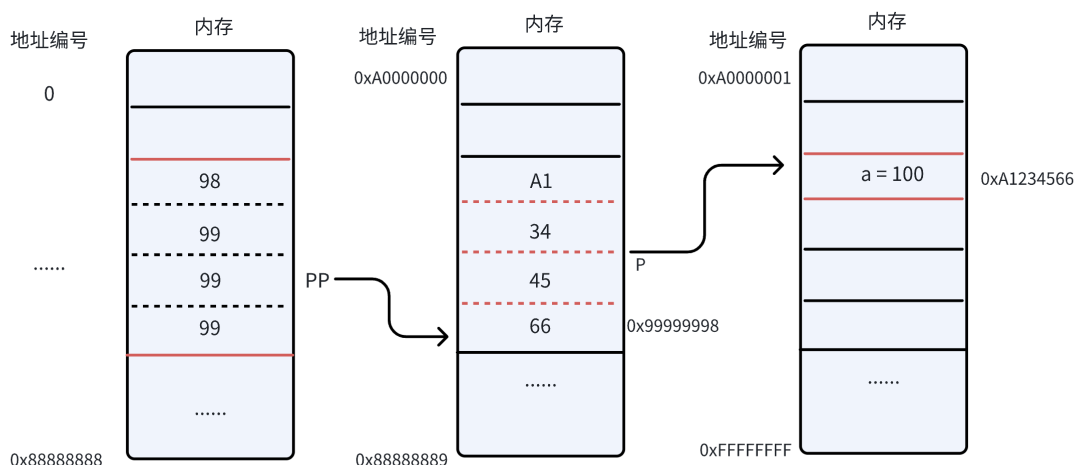
多级指针本质上也是一个指针，也需要一个指针变量来存放。这个指针变量的内存大小跟一级指针一样，跟系统有关。

```

1 int **p
2
3 1. *p 第一"*" 决定 变量p 是一个指针变量
4 2. **p 第二"*" 决定 指针p 访问内存规则是以 指针类型 进行访问，所以*p的值还是一个指针(地址)
5 3. int 决定 指针(*p) 访问内存规则是以 int类型 进行访问，所以 *( *p) 是 int类型

```

32位系统下多级指针访问



```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main(void)
5 {
6     int a = 100;
7     int *p = &a;
8     int **pp = &p;
9
10    printf("**pp = %d, *pp[0] = %d \n", **pp, *pp[0]);
11    printf("size of pp %ld \n", sizeof(pp));
12    return 0;
13 }
14
15
16 //运行结果
17 **pp = 100, *pp[0] = 100 //建议用*pp[]的方式访问
18 size of pp 8

```

- 用于指针变量地址传递

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 void func1(char * s)
5 {
6     printf("func1: %s \n", s);
7
8     s = "hello linux";
9 }
10
11 void func2(char **s)
12 {
13     *s = "hello linux";
14 }
15
16 int main(void)
17 {
18     char * s = "hello world";
19
20     func1(s);
21     printf("s1 = %s \n", s);
22
23     func2(&s);
24     printf("s2 = %s \n", s);

```

```

25
26     return 0;
27 }
28
29 //运行结果
30 func1: hello world
31 s1 = hello world
32 s2 = hello linux
33

```

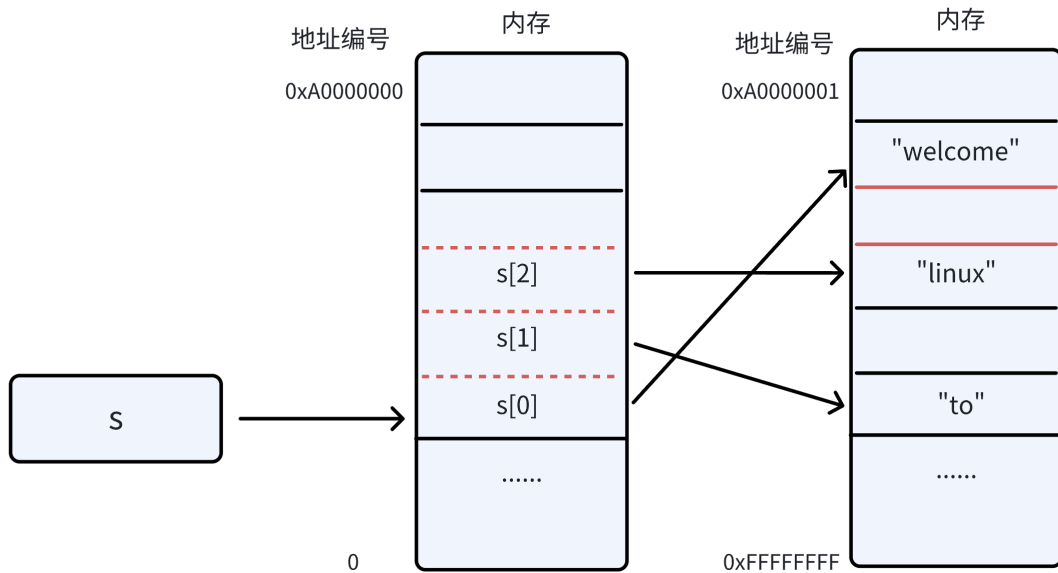
- 多级指针用于**物理无序**映射到**逻辑有序**的数据结构设计：

```

1  #include <stdio.h>
2  #include <stdlib.h>
3
4  int main(void)
5  {
6      char *arr[3] = {"welcome", "to", "linux"};
7      char **s = arr;
8
9      printf("%s:%p\n", s[0], s[0]);
10     printf("%s:%p\n", s[1], s[1]);
11     printf("%s:%p\n", s[2], s[3]);
12
13     printf("&s[0]:%p\n", &s[0]);
14     printf("&s[1]:%p\n", &s[1]);
15     printf("&s[2]:%p\n", &s[2]);
16
17     return 0;
18 }
19
20 //运行结果
21 welcome:0x5557a265a004
22 to:0x5557a265a00c
23 linux:0x75e4a3ccff45c900
24 &s[0]:0x7ffd01eae80
25 &s[1]:0x7ffd01eae88
26 &s[2]:0x7ffd01eae90

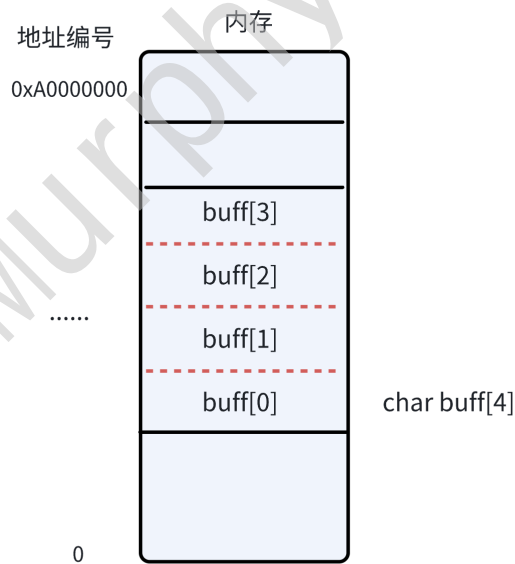
```

32位系统下多级指针访问



5.3 数组

数组本质上不算是一个新的数据类型，而是一种用于存储相同类型数据元素的数据结构。数组中的每个元素都有唯一的索引（位置），通过索引可以访问和操作数组中的元素。



5.3.1 数组的定义

数组定义需要回答：分配多少空间？读取的方式是什么？

- 1 数据类型 数组名[n] : `int array[10]`
- 2
- 3 tips:

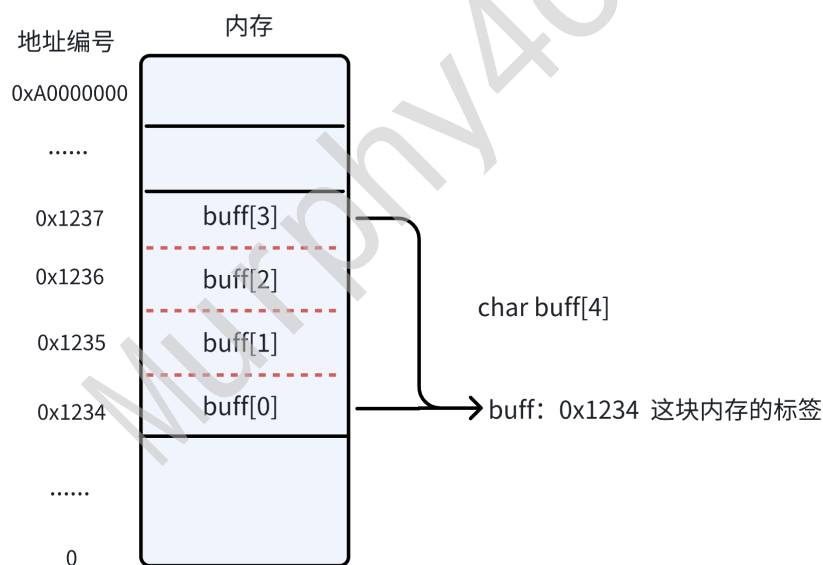
```

4 n的作用域是在申请空间的时候生效，代表圈定了n*sizeof(数据类型)字节的空间
5
6 #include <stdio.h>
7
8 int main(void)
9 {
10     int arr[5] = {1,2,3,4,5};
11     printf("size of arr is: %ld byte, len is %ld \n", sizeof(arr),
12           sizeof(arr) / sizeof(arr[0]));
13
14     return 0;
15 }
16 //运行结果:
17 size of arr is: 40 byte, len is 10

```

5.3.1.1 数组名的本质

数组名是一个常量标签（类似函数名），所以数组名不能做左值，是圈定的内存的别名。数组名的值是地址常量（指针常量），是一个地址编号，指向数组的首元素。



- 数组名是常量标签

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 int main(void)
4 {
5     char arr[20];
6     arr = "hello wolrd"; //编译报错
7
8     return 0;

```

```

9 }
10
11 1.c: In function 'main':
12 1.c:7:9: error: assignment to expression with array type
13     7 |     arr = "hello world";
14       |
15
16 Tips: 啥是指针常量? 假设0x12345678合法, 则 (int*) 0x12345678 就是一个指针常量, 指针常量
    当然不能被赋值: (int*) 0x12345678 = 123; //不符合语法, 类似 (int) 123 = 456; 一样

```

- arr 和 &arr 的区别

arr的值和&arr值一样, 但是意义不同: arr的值是指向数组的首元素的地址常量(指针常量), &arr的值是指向整片数组内存的地址常量(指针常量)。所以都是指针常量, 但是指针读取内存规则不一样。

```

1 #include <stdio.h>
2 #include <stdlib.h>
3
4 char arr[12];
5 int main(void)
6 {
7     printf("arr      = %d\n", arr);
8     printf("&arr     = %d\n", &arr);
9     printf("size of arr is: %ld\n", sizeof(arr));
10    printf("arr + 1  = %d\n", arr + 1);
11    printf("&arr + 1 = %d\n", &arr + 1);
12
13    return 0;
14 }
15
16 //运行结果
17 arr      = 738533400
18 &arr      = 738533400
19 size of arr is: 12
20 arr + 1  = 738533401
21 &arr + 1 = 738533412

```

5.3.1.2 一维数组的访问

```

1 #include <stdio.h>
2 #include <stdlib.h>
3

```

```

4 int main(void)
5 {
6     char arr[12] = "hello world";
7
8     printf("%c \n", arr[4]);
9     printf("%c \n", *(arr+4));
10
11     return 0;
12 }
13
14 //运行结果
15 o
16 o

```

5.3.1.3 二维数组的访问

```

1 #include <stdio.h>
2
3 int main()
4 {
5     int arr[3][5] = {{1, 2, 3, 4, 5}, {6, 7, 8, 9, 10}, {11, 12, 13, 14, 15}};
6
7     // 打印数组指针指向的数组中的元素
8     for (int i = 0; i < 3; i++) {
9         for (int j = 0; j < 5; j++) {
10             printf("%d ", arr[i][j]);
11         }
12         printf("\n");
13     }
14     printf("%p %p %d\n", arr, *arr, **arr);
15     // 打印数组指针指向的数组中的元素
16     for (int i = 0; i < 3; i++) {
17         for (int j = 0; j < 5; j++) {
18             printf("%d ", (*(arr+i)+j));
19         }
20         printf("\n");
21     }
22
23     return 0;
24 }
25
26
27 //运行结果
28 1 2 3 4 5
29 6 7 8 9 10

```

```

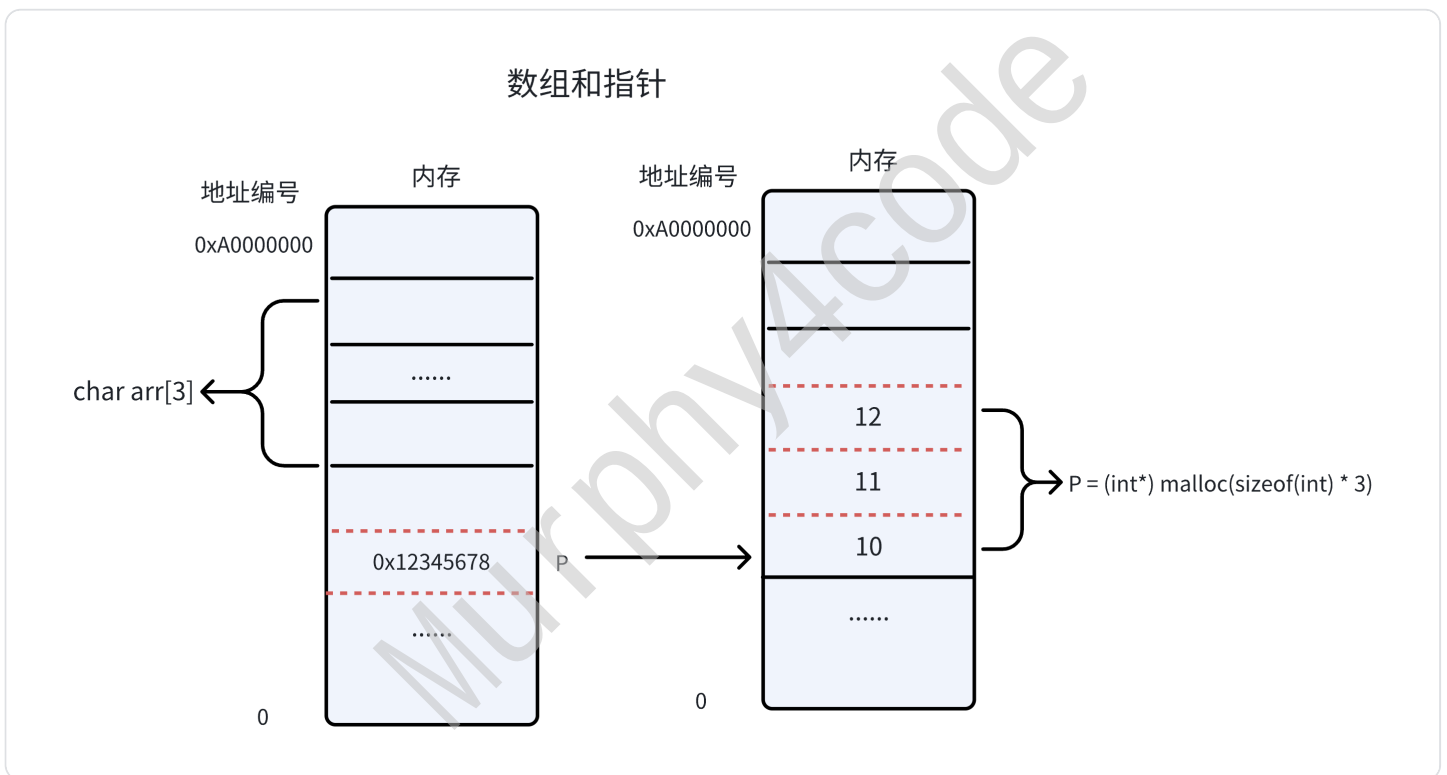
30 11 12 13 14 15
31
32 0x7ffc6fb525d0 0x7ffc6fb525d0 1
33
34 1 2 3 4 5
35 6 7 8 9 10
36 11 12 13 14 15

```

5.3.2 数组与指针

5.3.2.1 数组与指针区别

数组和指针本质上都可以代表一个连续的空间。只是数组名是这段连续空间的标签（指针常量），而指针需要一个指针变量来存放。在访问内存时，数组名和指针可以画等号。



5.3.2.2 指针数组与数组指针

- 指针数组：指针数组是一个数组，其中的每个元素都是一个指针。

示例： `int *array[3];` 声明了一个包含3个元素的指针数组，每个元素都是一个指向整数的指针。

用途： 指针数组常用于存储一组指针，每个指针可以指向不同类型的数据或不同位置的数组。

```

1 int main()
2 {
3     int a = 1, b = 2, c = 3;
4     int *array[3] = {&a, &b, &c};
5

```

```

6    // 打印指针数组中每个元素指向的值
7    for (int i = 0; i < 3; i++) {
8        printf("%d ", *array[i]);
9    }
10
11    return 0;
12 }

```

- 数组指针：数组指针是一个指针，它指向一个数组

示例：int (*ptr)[3]; 声明了一个指针，指向包含3个元素的整数数组

用途：数组指针常用于处理二维数组或作为指向动态分配数组的指针

```

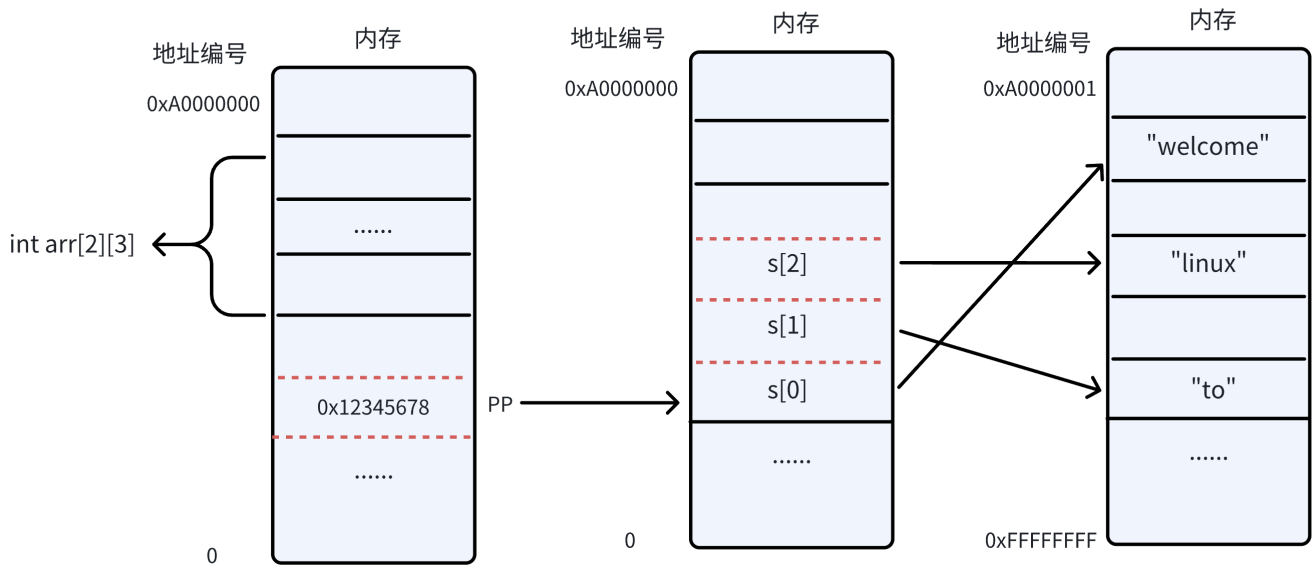
1  #include <stdio.h>
2
3  int main()
4  {
5      int arr[3][5] = {{1, 2, 3, 4, 5}, {6, 7, 8, 9, 10}, {11, 12, 13, 14, 15}};
6      int (*p)[5] = arr;
7
8      // 打印数组指针指向的数组中的元素
9      for (int i = 0; i < 3; i++) {
10         for (int j = 0; j < 5; j++) {
11             printf("%d ", p[i][j]);
12         }
13         printf("\n");
14     }
15
16     return 0;
17 }

```

5.3.2.3 二维数组和二级指针的关系

先说结论：没有关系。二维数组本质上是一段连续空间的圈定，跟二级指针在概念上就不是一个维度，无法比较。但二维数组名是一个**二级指针常量**。在内存访问时，**二维数组名和二级指针可以画等号**。

二维数组和二级指针



6. 常见面试题

6.1 预处理-#define

```
1 Q: 用预处理指令#define声明一个常数，用以表明1年中有多少秒(忽略闰年问题)
2
3 #define 宏名 宏体
4 宏名：大写字母表示
5
6 #define SECOND_OF_YEAR (365 * 24 * 3600) UL
7
8 L表示长整型//因为可能CPU位数不一样，至少保证为4字节
9 U表示无符号
10
11 int a = SECOND_OF_YEAR;//编译阶段已经处理为常数
12
13 1、#define语法的基本知识（例如：不能以分号结束、括号的使用，等等）
14 2、懂得预处理器将为你计算常数表达式的值，因此，直接写出你是如何计算一年中有多少秒而不是计算出实际的值，是更清晰而没有代价的。
15 3、意识到这个表达式将使一个16位机的整型数溢出，因此要用到长整型符号L，告诉编译器这个常数是长整型数。
16 4、如果你在你的表达式中用到UL（表示无符号长整型）。
17
```

```

1 //嵌入式程序员面试题：下面这个MIN宏有哪些问题？
2 #define MIN(A,B) A <= B ? A : B
3 问题1: 3 * MIN2(10, 20) 结果不符合预期
4 解决方案: #define MIN(A,B) (A <= B ? A : B)
5
6 问题2: MIN2(10, 20 < 30? 20 : 30)结果不符合预期
7 解决方案:
8 #define MIN(A,B) ((A) <= (B) ? (A) : (B))
9
10 问题3: MIN2(a++, 20 < 30? 20 : 30)结果不符合预期
11 解决方案:
12 #define MIN(A,B) ({ \
13     typeof(A) _min1 = (A); \
14     typeof(B) _min2 = (B); \
15     _min1 < _min2 ? _min1 : _min2;\
16 })
17
18 int main(void)
19 {
20     int a = 10;
21
22     printf("    min = %d \n", 3 * MIN(a++, 20 < 30? 20 : 30));
23     printf("    a = %d \n", a);
24     return 0;
25 }

```

6.2 修饰符-const

```

1 C:只读，建议性，不具备强制性 !=常量
2 const意味着“只读”
3 const int a = 100; // a是变量，尽量保持100不变，可通过指针修改
4
5 const int a;
6 int const a;
7 前两个的作用是一样，a是一个常整型数。
8
9 const int *a;
10 意味着a是一个指向常整型数的指针（也就是，整型数是不可修改的，但指针可以）。
11
12 int * const a;
13 a是一个指向整型数的常指针（也就是说，指针指向的整型数是可以修改的，但指针是不可修改的）。
14
15 int const * const a;
16 a是一个指向常整型数的常指针（也就是说，指针指向的整型数是不可修改的，同时指针也是不可修改的）。

```

17

18 合理地使用关键字`const`可以使编译器很自然地保护那些不希望被改变的参数，防止其被无意的代码修改。简而言之，这样可以减少bug的出现。

6.3 signed与unsigned隐式转换

```
1 #include <stdio.h>
2 //嵌入式程序员面试题：下面的代码输出是什么，为什么？
3
4 int main(void)
5 {
6     unsigned int a = 9;
7     int b = -10;
8
9     printf("    a+b = %d \n", a+b);
10    (b+a > 0) ?
11        puts("    a+b > 0"):
12        puts("    a+b <= 0");
13    return 0;
14 }
15
16 //输出结果
17    a+b = -1
18    a+b > 0
```

6.4 复杂类型定义

右左原则。

```
1 Q:用变量a给出下面的定义
2
3 1. 一个整型数 (An integer) ;
4
5 int a;
6
7 2. 一个指向整型数的指针 (A pointer to an integer) ;
8
9 int *a;
10
11 3. 一个指向指针的指针，它指向的指针是指向一个整型数 (A pointer to a pointer to an
    integer) ;
12
13 int **a;
```

```
14
15 4. 一个有10个整型数的数组 (An array of 10 integers) ;
16
17 int a[10];
18
19 5. 一个有10个指针的数组，该指针是指向一个整型数的 (An array of 10 pointers to
    integers) ;
20
21 int *a[10];
22
23 6. 一个指向有10个整型数数组的指针 (A pointer to an array of 10 integers) ;
24
25 int (*a)[10];
26
27 7. 一个指向函数的指针，该函数有一个整型参数并返回一个整型数 (A pointer to a function
    that takes an integer as an argument and returns an integer) ;
28
29 int (*a)(int)
30
31 8. 一个有10个指针的数组，该指针指向一个函数，该函数有一个整型参数并返回一个整型数 ( An
    array of ten pointers to functions that take an integer argument and return an
    integer )
32
33 int (*a[10]) (int)
```

7. 附录

7.1 推荐的学习网站

<https://c-cpp.com/c/language>

https://www.geeksforgeeks.org/c-programming-language/?ref=gcse_ind

7.2 ASCII码表

ASCII可显示字符（共95个）

二进制	十进制	十六进制	图形	二进制	十进制	十六进制	图形	二进制	十进制	十六进制	图形
0010 0000	32	20	(space)	0100 0000	64	40	@	0110 0000	96	60	`
0010 0001	33	21	!	0100 0001	65	41	A	0110 0001	97	61	a
0010 0010	34	22	"	0100 0010	66	42	B	0110 0010	98	62	b
0010 0011	35	23	#	0100 0011	67	43	C	0110 0011	99	63	c
0010 0100	36	24	\$	0100 0100	68	44	D	0110 0100	100	64	d
0010 0101	37	25	%	0100 0101	69	45	E	0110 0101	101	65	e
0010 0110	38	26	&	0100 0110	70	46	F	0110 0110	102	66	f
0010 0111	39	27	'	0100 0111	71	47	G	0110 0111	103	67	g
0010 1000	40	28	(	0100 1000	72	48	H	0110 1000	104	68	h
0010 1001	41	29	)	0100 1001	73	49	I	0110 1001	105	69	i
0010 1010	42	2A	*	0100 1010	74	4A	J	0110 1010	106	6A	j
0010 1011	43	2B	+	0100 1011	75	4B	K	0110 1011	107	6B	k
0010 1100	44	2C	,	0100 1100	76	4C	L	0110 1100	108	6C	l
0010 1101	45	2D	-	0100 1101	77	4D	M	0110 1101	109	6D	m
0010 1110	46	2E	.	0100 1110	78	4E	N	0110 1110	110	6E	n
0010 1111	47	2F	/	0100 1111	79	4F	O	0110 1111	111	6F	o
0011 0000	48	30	0	0101 0000	80	50	P	0111 0000	112	70	p
0011 0001	49	31	1	0101 0001	81	51	Q	0111 0001	113	71	q
0011 0010	50	32	2	0101 0010	82	52	R	0111 0010	114	72	r
0011 0011	51	33	3	0101 0011	83	53	S	0111 0011	115	73	s
0011 0100	52	34	4	0101 0100	84	54	T	0111 0100	116	74	t
0011 0101	53	35	5	0101 0101	85	55	U	0111 0101	117	75	u
0011 0110	54	36	6	0101 0110	86	56	V	0111 0110	118	76	v
0011 0111	55	37	7	0101 0111	87	57	W	0111 0111	119	77	w
0011 1000	56	38	8	0101 1000	88	58	X	0111 1000	120	78	x
0011 1001	57	39	9	0101 1001	89	59	Y	0111 1001	121	79	y
0011 1010	58	3A	:	0101 1010	90	5A	Z	0111 1010	122	7A	z
0011 1011	59	3B	;	0101 1011	91	5B	[	0111 1011	123	7B	{
0011 1100	60	3C	<	0101 1100	92	5C	\	0111 1100	124	7C	
0011 1101	61	3D	=	0101 1101	93	5D	]	0111 1101	125	7D	}
0011 1110	62	3E	>	0101 1110	94	5E	^	0111 1110	126	7E	~
0011 1111	63	3F	?	0101 1111	95	5F	_				

ASCII控制字符（共33个）

二进制	十进制	十六进制	缩写	Unicode 表示法	脱出字符 表示法	名称 / 意义
0000 0000	0	00	NUL	NUL	^@	空字符 (Null)
0000 0001	1	01	SOH	SOH	^A	标题开始
0000 0010	2	02	STX	STX	^B	文本开始

0000 0010	2	02	STX	STX	^B	本文开始
0000 0011	3	03	ETX	ETX	^C	本文结束
0000 0100	4	04	EOT	EOT	^D	传输结束
0000 0101	5	05	ENQ	ENQ	^E	请求
0000 0110	6	06	ACK	ACK	^F	确认回应
0000 0111	7	07	BEL	BEL	^G	响铃
0000 1000	8	08	BS	BS	^H	退格
0000 1001	9	09	HT	HT	^I	水平定位符号
0000 1010	10	0A	LF	LF	^J	换行键
0000 1011	11	0B	VT	VT	^K	垂直定位符号
0000 1100	12	0C	FF	F _F	^L	换页键
0000 1101	13	0D	CR	C _R	^M	CR (字符)
0000 1110	14	0E	SO	S _O	^N	取消变换 (Shift out)
0000 1111	15	0F	SI	S _I	^O	启用变换 (Shift in)
0001 0000	16	10	DLE	D _{LE}	^P	跳出数据通讯
0001 0001	17	11	DC1	D _{C1}	^Q	设备控制一 (XON 激活软件速度控制)
0001 0010	18	12	DC2	D _{C2}	^R	设备控制二
0001 0011	19	13	DC3	D _{C3}	^S	设备控制三 (XOFF 停用软件速度控制)
0001 0100	20	14	DC4	D _{C4}	^T	设备控制四
0001 0101	21	15	NAK	N _{AK}	^U	确认失败回应
0001 0110	22	16	SYN	S _{YN}	^V	同步用暂停
0001 0111	23	17	ETB	E _{TB}	^W	区块传输结束
0001 1000	24	18	CAN	C _{AN}	^X	取消
0001 1001	25	19	EM	E _M	^Y	连线介质中断
0001 1010	26	1A	SUB	S _{UB}	^Z	替换
0001 1011	27	1B	ESC	E _{SC}	^[	退出键
0001 1100	28	1C	FS	F _S	^\ ^]	文件分割符
0001 1101	29	1D	GS	G _S	^]	组群分隔符
0001 1110	30	1E	RS	R _S	^^	记录分隔符
0001 1111	31	1F	US	U _S	^_ ^?	单元分隔符
0111 1111	127	7F	DEL	D _{EL}	^?	Delete字符